

OpenFrame TJESガイド

OpenFrame/Batch for MVS 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame TJESガイド

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/Batch for MVS 7.1

ガイドバージョン: v3.1.3

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	ix
第1章 TJESの紹介	1
1.1. 概要	1
1.2. 特徴	2
1.3. TJESの構成	2
1.4. マルチノードのTJES構成	6
1.5. TJESのブートとシャットダウン	7
1.5.1. ブート	7
1.5.2. ウォームブートとコールドブート	7
1.5.3. シャットダウン	8
1.6. TJESシステム・テーブル	9
1.6.1. JESST	9
1.6.2. JOBQ	9
1.6.3. OUTPUTQ	10
第2章 ジョブの管理	11
2.1. ジョブの状態	11
2.1.1. コマンドによるジョブ状態の変更	13
2.2. JCLの管理	14
2.3. ジョブの処理手順	15
2.3.1. サブミット	15
2.3.2. スケジューリング	17
2.3.3. 実行	18
2.3.4. 出力	18
2.3.5. 削除	19
2.4. ジョブの検索	19
2.5. ログ	19
2.5.1. サブミット・ログ	20
2.5.2. ジョブ・ログ	20
2.6. ジョブのバックアップ	25
2.7. ジョブの終了コード	26
2.7.1. ジョブ・ステップ終了コード	26
2.7.2. ジョブ終了コード	27
第3章 ジョブの実行	29
3.1. 概要	29
3.2. ジョブの実行	29
3.2.1. 実行の初期化	30
3.2.2. ジョブ・ストリームの解析	32
3.2.3. 実行プロセス	33
3.2.4. EXEC PGM処理	34

3.2.5.	プロシージャ	37
3.2.6.	DD処理	38
3.2.7.	スペシャルDD	39
3.2.8.	出力処理	40
3.2.9.	ジョブ・スプール	41
3.2.10.	ジョブ制御	43
3.2.11.	ジョブ・レベルのレポート	44
3.2.12.	ステップ・レベルのレポート	44
3.2.13.	セキュリティ	45
3.2.14.	tjclrunの呼び出し	48
3.2.15.	NICEによるPERFORM処理	49
第4章	スプール	51
4.1.	概要	51
4.2.	スプール・データセット	51
4.2.1.	デフォルト・データセット	52
4.2.2.	SYSOUTデータセット	61
4.3.	スプールのバックアップ	62
第5章	出力処理	63
5.1.	概要	63
5.2.	出力	63
5.3.	出力処理	64
5.3.1.	出力の状態	64
5.3.2.	出力クラス	65
5.3.3.	出力の処理方式	66
5.3.4.	内部リーダー	68
5.3.5.	外部ライター	68
第6章	TJESMGRコマンド	71
6.1.	概要	71
6.2.	ジョブ実行コマンド	77
6.2.1.	CANCEL	77
6.2.2.	HOLD	77
6.2.3.	INQUEUECLASS	77
6.2.4.	INQUEUEPRIORITY(IP)	77
6.2.5.	JEM	78
6.2.6.	NICE	78
6.2.7.	OUTQUEUEPURGE	79
6.2.8.	OUTQUEUERELEASE	79
6.2.9.	PSTART	79
6.2.10.	REMOVE	80
6.2.11.	RESUME	80
6.2.12.	RUN	81
6.2.13.	SCAN	81

6.2.14.	START	82
6.2.15.	STOP	82
6.2.16.	SUSPEND	82
6.3.	ジョブ出力コマンド	83
6.3.1.	PO(Print Output status)	83
6.3.2.	PODD(Print Output DD Information)	83
6.3.3.	POJOB(Print Output of JOB)	84
6.3.4.	POLIST(Print Output detail LIST)	85
6.3.5.	POSPPOOL(Print Output SPOOL of JOB)	85
6.3.6.	PS(Print Screen)	86
6.3.7.	PSHISTORY(PSH)	88
6.3.8.	PSIO(Print Screen Input/Output information)	88
6.3.9.	PSJOB(Print Screen JOB detail information)	89
6.3.10.	PSSTEP(Print Screen STEP)	91
6.4.	イニシエーター・コマンド	91
6.4.1.	BOOT	91
6.4.2.	CHANGECLASS(CC)	92
6.4.3.	JCLRUNACTIVE(RA)/JCLRUNINACTIVE(RI)	93
6.4.4.	JCLRUNCLASS(RC)	94
6.4.5.	NODECLEAR	95
6.4.6.	NODESTATUS(NODEST)	95
6.4.7.	PSCLASS(PSC)	95
6.4.8.	PSJCLRUN (PSR)	95
6.4.9.	SHUTDOWN	98
6.4.10.	SHOWCLASS(SC)	98
6.5.	データセット・コマンド	99
6.5.1.	DATASETCOPY(DSCOPY)	99
6.5.2.	DATASETCREATE(DSCREATE)	100
6.5.3.	DATASETDELETE(DSDELETE)	102
6.5.4.	DATASETEXPORT(DSEXPORT)	102
6.5.5.	DATASETIMPORT(DSIMPORT)	103
6.5.6.	DATASETLIST(DSLIST)	104
6.5.7.	DATASETMOVE(DSMOVE)	104
6.5.8.	DATASETRECATALOG(DSRECAT)	105
6.5.9.	DATASETUNCATALOG(DSUNCAT)	106
6.5.10.	DATATOUCH(DSTOUCH)	106
6.5.11.	GDGCREATE	107
6.5.12.	GDGDELETE	108
6.5.13.	PSCATALOG(PSCAT)	108
6.5.14.	PSDATASET(PSDS)	109
6.5.15.	PSEENTRY(PSENT)	111
6.5.16.	PSLOCK(PSL)	111
6.5.17.	PSVOLUME(PSV)	112

6.5.18.	SETCATALOG(SETCAT)	113
6.6.	スプール・バックアップ・コマンド	113
6.6.1.	SPOOLBACKUP(SPBA)	114
6.6.2.	SPOOLBACKUPLIST(SPBL)	115
6.6.3.	SPOOLBACKUPSTATUS(SPBASTAT)	115
6.6.4.	SPOOLBACKUPSTOP(SPBASTOP)	116
6.6.5.	SPOOLCLEAR(SPCL)	116
6.6.6.	SPOOLPODD(SPPODD)	116
6.6.7.	SPOOLPOSPPOOL(SPPOSPPOOL)	117
6.6.8.	SPOOLPS(SPPS)	117
6.6.9.	SPOOLPSJOB(SPPSJ)	118
6.6.10.	SPOOLPSIO(SPPSIO)	118
6.6.11.	SPOOLRESTORE(SPRE)	119
6.7.	その他のコマンド	120
6.7.1.	CLS	120
6.7.2.	CURRENTUSER	120
6.7.3.	EDIT	120
6.7.4.	ERROR	121
6.7.5.	HELP	121
6.7.6.	LICENSE	121
6.7.7.	NOTIFY(N)	122
6.7.8.	PSPRINT(PSP)	123
6.7.9.	SWITCHUSER	124
6.7.10.	TSOLIST(TSL)	124
6.7.11.	VERSION	125
第7章	環境設定	127
7.1.	Tmaxの設定	127
7.1.1.	Tmaxサーバー	127
7.1.2.	マルチノードの設定	130
索引		133

図目次

[図 1.1]	TJESの構成	2
[図 1.2]	マルチノード環境	6
[図 2.1]	ジョブ状態のフロー	11
[図 5.1]	ジョブの出力処理方式	66
[図 5.2]	SMTPの動作構造	70

このガイドについて

対象読者

本書は、リホスト・ソリューションのTmax OpenFrame[®](以下、OpenFrame)/Batchシステムのエンジンとして機能するTJESを使用してジョブを実行および管理するユーザーを対象としています。ジョブの管理と実行、スプールの管理、出力処理について説明し、TJESのコマンドとログ、エラーへの対応方法およびTJES関連の環境設定について説明します。

前提知識

本書を理解するには、OpenFrame/Batchについての知識が必要です。本書を読む前に『OpenFrame Batchガイド』をお読みになることをお勧めします。

TJES関連の環境設定については、『OpenFrame 環境設定ガイド』を参照してください。

本書の構成

本書は、計7章で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: TJESの紹介

TJESの特徴、コンポーネント、マルチノードのTJES構成、TJESシステムのブートとシャットダウン、およびTJESシステム・テーブルについて説明します。

- 第2章: ジョブの管理

ジョブの状態と状態の変更、クラスの概要、JCLの管理、およびジョブの処理、検索、ログ、バックアップ、終了コードについて説明します。

- 第3章: ジョブの実行

実際にジョブを実行するランナーであるtjclrunの動作方式と機能について説明します。また、JCL解析、ジョブの実行とその結果について説明します。

- 第4章: スプール

スプール・データセットとスプール・バックアップについて説明します。

- 第5章: 出力処理

出力の概念および処理方法について説明します。

- 第6章: TJESMGRコマンド

ジョブの実行コマンド、ジョブ出力コマンド、イニシエーター・コマンド、データセット・コマンド、スプール・バックアップ・コマンド、およびその他のTJESMGRコマンドについて説明します。

- 第7章: 環境設定

TJESに必要なTmax環境ファイルについて説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上 (32ビット, 64ビット)
	Linux x86 2.6以上 (32ビット, 64ビット)
	Linux ia64 2.6以上 (32ビット, 64ビット)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 (Fixset07)
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame インストールガイド	OpenFrame製品のインストールと環境設定について説明しています。
OpenFrame Batchガイド	OpenFrame/Batchの紹介および機能について説明しています。
OpenFrame Baseガイド	OpenFrame/Baseを構成する全般的な機能について説明しています。
OpenFrame TACF管理者ガイド	TACFの機能と構造、ユーザーとグループの関連付け、リソースとデータセットへの認証、および多様なTACFコマンドについて説明しています。
OpenFrame データセットガイド	OpenFrameのデータセットを紹介し、データセットの種類やカタログ方法などについて説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameエンジンと一緒に提供されるユーティリティ・プログラムについて説明しています。
OpenFrame エラーメッセージ リファレンスガイド	OpenFrame製品の使用中に発生し得るエラーのタイプおよびその解決方法について説明しています。
OpenFrame ツールリファレンスガイド	OpenFrameシステムの運用に必要な各種ツール・プログラムについて説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

参考文献

製品	ガイド
メインフレーム	JES2 Introduction
	MVS JCL Reference
	MVS JCL User's Guide
	JCL Reference Guide

第1章 TJESの紹介

本章では、TJESの特徴、コンポーネント、マルチノードのTJES構成、TJESシステムのブートとシャットダウン、およびTJESシステム・テーブルについて説明します。

1.1. 概要

OpenFrame TJES (Tmax Job Entry Subsystem、以下TJES) は、メインフレームのJESに対応するOpenFrameシステムのバッチ・ジョブ管理モジュールです。TJESは、マルチノード構成や自動化されたエラー復旧をサポートするために、TmaxSoftのTPモニター製品であるTmax上に実装されています。

TJESは、MVS JCLをIBMメインフレームと同様にUNIXでサポートします。また、実証されたミドルウェア、Tmaxのマルチノード・クラスタリング機能を使用するので、大規模なメインフレーム・システムを問題なくリホストすることができます。

TJESは、JCLを使用してユーザーからのジョブを受け取り、システムのリソース状況に合わせてスケジューリングしてランナーを使用して実行します。また、ジョブの実行状況の確認や実行結果の出力など、OpenFrameで実行されるジョブのすべてを管理します。

以下は、TJESの機能と特性です。

- JCLを使用してジョブをサブミットします。
 - IBMメインフレームのMVS JCLをサポート
 - CONTROL-M、A-AUTOなどの外部スケジューラーとの連携をサポート
 - インターナル・リーダー(内部読み取りプログラム)をサポート
 - エクスターナル・ライター(外部書き出しプログラム)をサポート
- サブミットされたジョブをスケジューリングします。
 - ジョブ・クラス、ジョブの優先順位によるスケジューリング
 - マルチノード・スケジューリングをサポート
- ジョブを出力します。
 - IBM印刷フォーマットをサポートするINFOPRINTを提供

本書は、ジョブの管理と実行、スプールの管理、ジョブの出力処理について説明します。また、TJESのコマンドとログ、エラーへの対応方法、TJES関連の環境設定についても説明します。

1.2. 特徴

TJESは、メインフレームのバッチ・プログラム（ユーティリティ、COBOLプログラム、PL/Iプログラム）の起動や、UNIXシェル・スクリプト、Cプログラムなど、UNIXで実行可能なすべてのプログラムを起動することができます。UNIXでシェル・スクリプトとcronを介してバッチ処理を実行する際、体系化されたジョブ・スケジューリングや容易な管理を提供します。

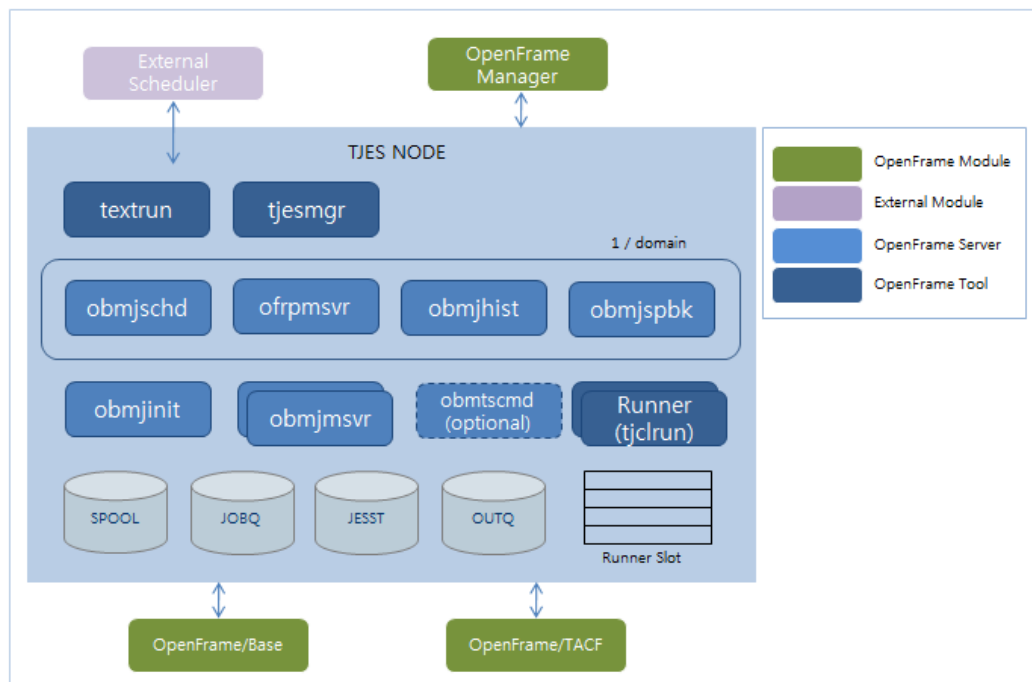
以下は、TJESが提供するバッチ・システムです。

- ジョブ・クラスとランナー・クラスによるスケジューリング
- ランナー・スロットの数を制限し、同時に実行できるバッチ・ジョブの数を制御
- ジョブの進行状況と結果を確認
- ジョブの一時停止、再開、停止およびジョブの属性を変更
- 出力の管理
- データセット・ロックによるデータ整合性を保証
- TACFによるセキュリティの強化

1.3. TJESの構成

以下の図は、TJESの構成を示しています。

[図 1.1] TJESの構成



TJESのコンポーネント

- obmjschd

ジョブ・スケジューラーです。TJESのドメイン全体で1つのみ起動されます。TJESのジョブ・スケジューリングおよびジョブIDの発行や各ノードのブート状態を管理します。

- ofrpmsvr

プリンター管理サーバーです。TJESのドメイン全体で1つのみ起動されます。OUTPUTQに登録された出力を条件を満たすプリンターで出力します。

- obmjhist

ジョブ履歴サーバーです。TJESのドメイン全体で1つのみ起動されます。ジョブの状態を変更するすべてのアクション情報を保存します。

- obmjmsvr

ジョブ管理サーバーです。TJESのジョブと出力を管理および表示することができます。

- obmjspbk

スプール・バックアップ・サーバーです。TJESの1つのノードでのみ起動されます。実行済みのジョブをTJESから削除する際、そのジョブのスプールをバックアップします。また、バックアップされたスプールをTJESと同じ形式で表示することができます。

- obmjinit

ランナーとランナー・スロットの管理サーバーです。TJESの各ノードに1つずつ起動され、自身のノードに割り当てられたランナーとランナー・スロットを管理し、ジョブをランナーに割り当てます。

参考

障害発生により、Tmaxのフェイルオーバー機能によって自動的に再起動された場合、obmjinitは、WORKING状態またはSUSPEND状態のジョブをSTOP状態に変更し、該当のランナー・プロセスを強制終了します。ただし、ユーザーが手動でobmjinitサーバーを再起動した場合は、ジョブの状態は変更されますが、再起動された時点とランナーのPIDが明確ではないため、ランナー・プロセスは自動的に終了されません。

- obmtscmd (optional)

ISPFサービスでCLISTを実行する場合に、CLISTに記述されたTSOコマンドを処理するサーバーです。

参考

ISPFサービスを使用する環境でのみ提供されるサーバーです。ISPFサービスとJSON形式のデータをやり取りするので、Linux環境にjansson (jansson.h、libjansson.so)がインストールされている必要があります。

- tjclrun

JCLの実行モジュールです。JCLジョブをステップ順に実行します。

その他のOpenFrame製品

- OpenFrame/Base

OpenFrame(Batch、OSC、OpenFrame Managerなど) を実行するための必須製品です。

OpenFrame/Baseは、非VSAM、TSAMなどのデータセットを提供し、カタログやデータセット・ロックなどを管理することができます。

参考

OpenFrame/Baseの詳細については、『OpenFrame Baseガイド』を参照してください。

- OpenFrame/TACF

OpenFrameのセキュリティをサポートします。OpenFrame TJESはOpenFrame/TACFと連携して、ユーザー認証、データセットやジョブ名などのリソースにアクセスするための権限チェックなどにおいて強力なセキュリティ機能を提供します。

参考

OpenFrame/TACFの詳細については、『OpenFrame TACF管理者ガイド』を参照してください。

TJESのリソース

- ランナー・スロット

ランナー・スロットは、ランナーの運用情報を保存する領域であり、1つのランナーに1つのランナー・スロットが割り当てられます。ランナーとobmjinitは、ランナー・スロットを介してデータを送受信します。ランナー・スロットは、obmjinitの起動時に生成され、終了時に削除されます。システム・テーブルのOFM_BATCH_RUNNERで実装されています。

- スプール

TJESが使用する特別なデータセットです。ジョブの実行に必要なリソースとジョブの進行状況や結果を保存するために使用されます。ジョブをサブミットすると、ジョブIDと同じ名前のディレクトリが生成されますが、そのディレクトリのサブディレクトリが使用されます。スプールの構成する情報は、設定に従ってスプール・ディレクトリのメンバーまたはシステム・テーブルで管理されます。ジョブ・スプールは、REMOVEコマンドまたはCANCELコマンドで削除できます。

- TJESシステム・テーブル

TJESが内部情報を保存するために使用するシステム・テーブルです。JOBQ、JESST、OUTPUTQがあります。詳細については、[「1.6. TJESシステム・テーブル」](#)を参照してください。

インターフェース

- textrun

サードパーティー・スケジューラーによってTJESにサブミットされたジョブの進行状況と結果をモニタリングできるモジュールです。UNIXでのサードパーティー・スケジューラーは、ジョブの開始と終了をプロセスの開始と終了とみなします。したがって、textrunは自身がサブミットしたジョブの実行が完了するまで終了されずに実行され、ジョブが終了すると、その結果を返して終了します。

- tjesmgr

システム管理者向けのコマンド・ベースのユーザー・インターフェースです。BOOTコマンドとSHUTDOWNコマンドを含むTJESのすべての機能を使用することができます。

- OpenFrame Managerの[Batch]メニュー

一般ユーザー向けのGUIのユーザー・インターフェースです。TJESシステムのブート、シャットダウンなどTJESの管理機能を除く、ジョブのサブミット、表示、管理機能を提供します。

外部製品

- 外部スケジューラー

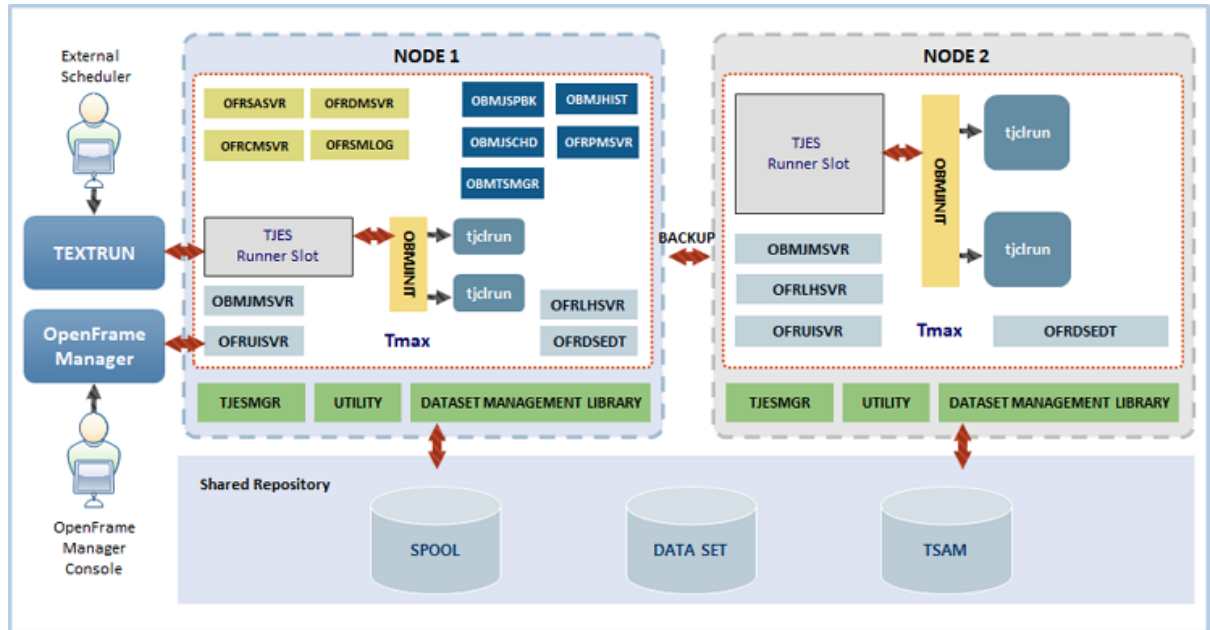
TJESは、ジョブ・クラスとジョブの優先順位によるスケジューリングのみサポートしており、ジョブ間の相関によるスケジューリングはサポートしていません。ジョブ間の相関によるスケジューリング、日間/週間/月間のバッチ処理など自動化されたジョブ・サブミットを実行する際には、外部スケジューラーを使用します。外部スケジューラーには、CONTROL-M、A-AUTOなどがあります。

1.4. マルチノードのTJES構成

TJESは、ジョブの処理性能やサービスの可用性を高めるために、複数のUNIXマシンを1つのマシンであるかのように使用できるマルチノード環境をサポートしています。本節では、TJESをマルチノードで構成する際、各コンポーネントの配置と動作方式について説明します。

以下の図は、2つのノードでTJESを構成した場合のコンポーネント・ダイアグラムです。

[図 1.2] マルチノード環境



TJESをマルチノードで構成するには、スプール、データセット、TSAMなどのデータ・ストレージ・リソースを共有する必要があります。TJESは、共有リソースを介して、複数のノードで同じデータセットを使用できるようにしたり、他のノードで実行されたジョブの結果を確認できるようにします。

NODE1のofrlmsvr、ofrsasvr、ofrpmsvr、obmjmschd、obmjhist、obmjspbk、obmtsmgrは、ドメイン全体で1つのみ存在できるTmaxサーバーです。NODE1に問題が発生した場合は、Tmaxによって自動的にNODE2が起動(フェイルオーバー)されるため、サービスの可用性を高めることができます。

NODE1とNODE2の両方に存在するobmjmsvr、ofrdsedt、ofrlhsvr、ofruijsvrは、マルチノード・サービスを提供するために各ノードに1つ以上存在する必要があります。obmjinitは、各ノードに1つのみ存在する必要があります。

tjclrunは、オンデマンド方式で実行されるプロセスなので、ノードごとに現在実行中のジョブ数と同数のtjclrunが存在し、実行中のジョブがない場合はtjclrunも存在しません。

obmjinitとtjclrunは、TJESランナー・スロットを介して情報を交換します。TJESランナー・スロットは、obmjinitの起動時に生成され、終了時に削除されます。

外部UIであるtextrun、OpenFrame Manager、tjesmgrは、Tmaxを介して全ノードに接続できます。

1.5. TJESのブートとシャットダウン

ジョブを実行するには、TJESがブート状態である必要があります。システム管理者は、tjesmgrのBOOTコマンドとSHUTDOWNコマンドを使用してTJESのブートとシャットダウンを制御することができます。

参考

詳しい使用方法については、[「6.4. イニシエーター・コマンド」](#)の[「6.4.1. BOOT」](#)と[「6.4.9. SHUTDOWN」](#)を参照してください。

1.5.1. ブート

BOOTコマンドは、TJESがシャットダウンしている状態でのみ実行でき、すでにブートされている場合は無視されます。BOOTは、TJESの各ノードで自身のランナー・スロットを使用できる状態に初期化した後、TJESのobmjschdにランナー・スロットの現在の状態をレポートして、ジョブがスケジューリングされるようにします。

TJESが正常にブートされると、SYS1.JESSTシステム・データセットに当該ノードがブートされたことが記録されます。これにより、データベースへの接続失敗などの障害が発生した場合、TJESの自動復旧プロセスでブート状態が維持されます。

マルチノードのTJES環境では、TJESのブート時にTJESの全ドメインに属するすべてのノードにBOOTコマンドを送信します。また、各ノードごとにブートすることもできます。

ブート時に該当のノードのランナー・スロットは、DOWENDからDOWN状態になる前のACTIVEまたはINACTIVE状態で復旧されます。ブートされたことがない場合は、tjesサブジェクトのINITDEFセクションに設定されたデフォルト状態で復旧されます。

1.5.2. ウォームブートとコールドブート

- ウォームブート(WarmBoot)

システムを初期化した後、運用中に使用する一般的な起動方法です。TJESランナー・スロット・テーブルを初期化し、スケジューラーにランナー・スロットの現在の状態をレポートしてジョブ・スケジューリングが実行されるようにします。tjesmgrのBOOTコマンドを使用します。

- コールドブート(ColdBoot)

OpenFrame TJESを初期化するための特殊な起動方法です。tjesinitツールを使用してシステム・テーブルを初期化します。コールドブートを実行した後、実際にジョブを実行するためにはウォームブートを実行する必要があります。

以下の場合に、コールドブートを実行します。

- OpenFrame TJESのインストール後
- システム・テーブル内容の変更など、主要システムのアップグレード後

- OpenFrame環境設定の**tjes**サブジェクトの次のような変更をシステムに適用する際
 - **JOBDEF**セクションのJOBNUMの範囲の変更後
 - **JOBCLASS**セクションの変更後
 - **OUTDEF**セクションのOUTNUMの範囲の変更後

参考

tjesinitツールの詳細については、『OpenFrame ツールリファレンスガイド』の「4.5. tjesinit」を参照してください。

1.5.3. シャットダウン

SHUTDOWNコマンドは、**obmjinit**がブートされている状態でのみ実行できます。すでにシャットダウンしている状態では無視されます。**SHUTDOWN**コマンドが実行されると、TJESは該当のノードの状態を「Not booted」に変更し、システムのシャットダウン後に発生される追加的なジョブのスケジューリングを制限します。

シャットダウンされる前にスケジューリングされ実行中のジョブは、一応終了されるまで正常に実行されますが、ユーザーが強制終了を明示した場合は、SHUTDOWNコマンドの実行と同時に中止されます。

実行中のジョブをすべて終了してからシステムをシャットダウンさせる場合は、STOPコマンドを使用して実行中のジョブを終了した後、SHUTDOWNコマンドを使用してTJESシステムをシャットダウンします。

TJESが正常にシャットダウンしたら、SYS1.JESSTシステム・データセットに当該ノードがシャットダウンされたことを記録します。これにより、データベースへの接続失敗などの障害が発生した場合、TJESの自動復旧プロセスでシャットダウン状態が維持されます。

マルチノードのTJES環境では、TJESがシャットダウンされるとき、TJESの全ドメインに属するすべてのノードにSHUTDOWNコマンドを送信します。また、各ノードごとにシステムをシャットダウンすることもできます。

- システム全体をシャットダウンするコマンド

```
shutdown
```

- 各システムごとにシャットダウンするコマンド

```
shutdown node=nodename
```

SHUTDOWNコマンドが実行されたノードの各ランナー・スロットはDOWEND状態に変更されます。実行中のジョブがある場合はランナー・スロットをWORKING状態で維持し、ジョブの実行が終了されたらDOWEND状態に変更します。

1.6. TJESシステム・テーブル

TJESでは、ノード間でシステム設定と各種リソースの情報を共有するため、**システム・テーブル**に保存しています。

1.6.1. JESST

TJESのシステム・レベル情報(JESST)は、そのタイプに応じて3つのシステム・テーブルに保存されます。

システム・レベル情報	説明
ノード情報	システムに存在するノードの名前と各ノードのブート状態をシステム・テーブルのOFM_BATCH_NODESTに保存します。
JOBQ情報	JOBQの情報(ジョブIDの範囲など)をシステム・テーブルのOFM_BATCH_JOBQに保存します。
OUTPUTQ情報	OUTPUTQの情報(OUTPUTQのサイズなど)をシステム・テーブルのOFM_BATCH_OUTPUTQに保存します。
JOB CLASS情報	各ジョブ・クラスの基本属性情報をシステム・テーブルのOFM_BATCH_JCLSSTに保存します。

参考

1. JESSTの表には、OFM_BATCH_JESST、OFM_BATCH_JCLSST、OFM_BATCH_NODESTがあります。この3つの表は、OpenFrameのインストール時にインストールされる必要があります。
2. コールドブート中に記録されるJESSTシステム・テーブルのJOBQ、OUTPUTQ情報は、**tjesinit**ツールによってのみ更新されます。以降、OpenFrame環境設定の**tjes**サブジェクトを変更した場合は、**tjesinit**を使用してシステム・テーブルに変更内容を反映する必要があります。

1.6.2. JOBQ

TJESでジョブを管理するために必要な情報を保存するシステム・テーブルです。JOBID、JOBNAME、JOBCLASS、JOBPRTY、JOBSTATUS、JCLPATH、USER ACCOUNTなどがJOBQに保存されます。JOBIDに対してデフォルト・インデックスが設定されています。

参考

JOBQの実際の名前はOFM_BATCH_JOBQであり、OpenFrameのインストール時にインストールされる必要があります。

1.6.3. OUTPUTQ

TJESでのジョブの実行結果である出力(OUTPUT)を管理するために必要な情報を保存するシステム・テーブルです。

システム・テーブルは、OpenFrameのインストール時に作成されます。tmbootを実行する前にOpenFrameが提供するコールドブート用のUNIXツールである**tjesinit**を使用してシステム・テーブルを初期化する必要があります。

参考

OUTPUTQの実際の表名はOFM_BATCH_OUTPUTQであり、OpenFrameのインストール時にインストールされる必要があります。各システム・テーブルは、batchinitツールを使用して作成します。batchinitの実行方法については、『OpenFrame ツールリファレンスガイド』の「1.3. batchinit」を参照してください。

第2章 ジョブの管理

本章では、ジョブの状態、ジョブの実行手順と実行結果の確認、およびジョブの管理方法について説明します。

OpenFrameでは、tjesmgrとOpenFrame Managerを使用してジョブを管理することができます。本書では、tjesmgrを使用してジョブを管理する方法についてのみ説明します。

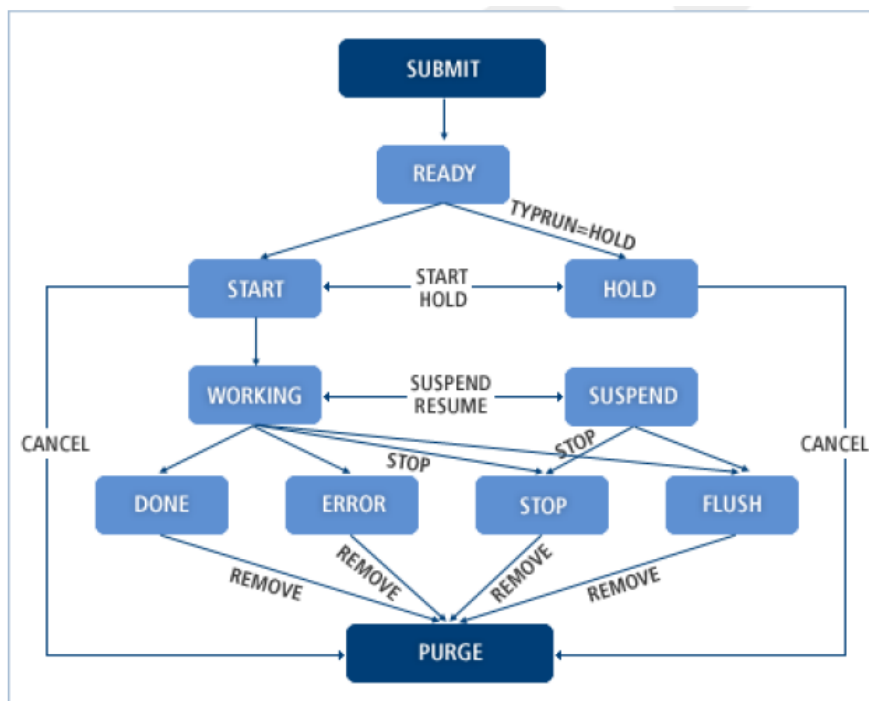
参考

OpenFrame Managerを使用してジョブを管理する方法については、『OpenFrame Manager ユーザーガイド』を参照してください。

2.1. ジョブの状態

TJESでは、ジョブをSTART、HOLD、WORKING、SUSPEND、DONE、ERROR、STOP、FLUSHの8つの状態に区別して管理しています。

[図 2.1] ジョブ状態のフロー



参考

READYは、ジョブがサブミット中であることを示しており、PURGEは、ジョブがTJESから削除されたことを示します。

以下は、TJESでのジョブ状態についての説明です。

状態	内容
START	JCLによってサブミットされたジョブが実行される前までの状態です。ジョブ・スケジューリングの対象であり、ランナー・スロットに割り当てられていない状態です。 JOB CLASSを変更することが可能であり、HOLDコマンドを使用してジョブをHOLD状態に変更したり、CANCELコマンドを使用してJOBQから削除したりすることができます。
HOLD	JCLによってサブミットされたジョブが実行される前までの状態です。ジョブ・スケジューリングの対象外です。HOLD状態に該当する場合は以下のとおりです。 – JCLにTYPRUN=HOLDを記述してサブミットした場合 – HOLDとして設定されたジョブ・クラスでサブミットした場合 – START状態のジョブをHOLDコマンドを使用して変更した場合 HOLD状態のジョブは、STARTコマンドを使用してSTART状態に変更したり、CANCELコマンドを使用してJOBQから削除したりすることができます。
WORKING	tjclrunがジョブを実行している状態です。実行中のステップの特性に応じて、複数のUNIXプロセスとして動作することができます。SUSPENDコマンドを使用して実行中のジョブを一時停止したり、STOPコマンドを使用してジョブの実行を終了し、STOP状態に変更したりすることができます。
SUSPEND	ジョブの実行が一時停止された状態です。RESUMEコマンドを使用してWORKING状態に戻したり、STOPコマンドを使用してジョブの実行を終了させ、STOP状態に変更したりすることができます。 サードパーティー・ユーティリティを使用する場合、ユーティリティ内でセッションを新しく設定すると、該当のプロセスとそのサブプロセスは一時停止(SUSPEND)されないことに注意してください。
DONE	JCL記述に従ってジョブが正常に実行された状態です。 ユーザー・プログラムが意図した結果を出しているかどうかは、スプール・データを表示(PODD)するか、各ステップのリターンコード(PSJOB)

状態	内容
	を表示して別途確認する必要があります。REMOVEコマンドを使用してJOBQからジョブを削除することができます。 DONE状態は、JCL記述に従ってジョブの全ステップが実行され、リターンコードがCOND文またはOpenFrame環境設定のrcサブジェクトなどで指定された有効範囲内にあることを示しているだけであり、各ステップに指定されたプロシージャまたはプログラムが正常に実行されたことを示しているわけではありません。
ERROR	ジョブの実行結果がCOND文、またはOpenFrame環境設定のrcサブジェクトで指定されたリターンコードの有効範囲外の場合です。REMOVEコマンドを使用してJOBQからジョブを削除することができます。
STOP	WORKING状態のジョブをSTOPコマンドを実行して強制終了するか、障害復旧計画に基づいてTJESが自動的に再起動され、ジョブが強制終了された状態です。ジョブの実行に使用されたすべてのリソースを返して終了されます。 REMOVEコマンドを使用してJOBQから該当するジョブを削除することができます。
FLUSH	ジョブの実行中にtjclrunでエラーが発生し、ジョブを実行できなくなった状態です。JCLランタイム解析エラーまたはDD割り当てエラーなどがFLUSH状態の主な原因です。 REMOVEコマンドを使用してJOBQからジョブを削除することができます。

2.1.1. コマンドによるジョブ状態の変更

ユーザーはコマンドを使用してジョブの状態を変更することができます。

次のtjesmgrコマンドを使用して、現在のジョブ状態を変更後のジョブ状態にすることができます。

現在のジョブ状態	tjesmgrコマンド	変更後のジョブ状態
-	RUN	START, HOLD
START	CANCEL	ジョブの実行前にJOBQからジョブが削除されます。
HOLD	HOLD	
HOLD	CANCEL	ジョブの実行前にJOBQからジョブが削除されます。
START	START	
WORKING	SUSPEND	SUSPEND
STOP	STOP	

現在のジョブ状態	tjesmgrコマンド	変更後のジョブ状態
SUSPEND	STOP	STOP
RESUME	WORKING	
DONE	REMOVE	ジョブの実行後にJOBQからジョブが削除されます。
ERROR	REMOVE	ジョブの実行後にJOBQからジョブが削除されます。
STOP	REMOVE	ジョブの実行後にJOBQからジョブが削除されます。
FLUSH	REMOVE	ジョブの実行後にJOBQからジョブが削除されます。

JOBQは、OpenFrame環境設定の**tjes**サブジェクトの**JOBDEF**セクションに設定されたサイズの分だけ保存できるため、不要なジョブはJOBQから削除する必要があります。

JOBQからジョブを削除する際、CANCELコマンドはジョブが実行される前に、REMOVEコマンドはジョブが実行された後に削除します。

HOLD状態のジョブはスケジューリングされないため、HOLD状態のジョブを実行したい場合は、STARTコマンドを使用してジョブをSTART状態に変更する必要があります。

参考

1. **tjes**サブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
 2. 各コマンドの使用方法については、『[第6章 TJESMGRコマンド](#)』を参照してください。
-

2.2. JCLの管理

OpenFrameでは既存のメインフレームで使用していたJCLをそのまま実行させることができます。

JCLは一般ディレクトリで管理しても構いませんが、OpenFrame環境設定の**tjes**サブジェクトの**PROCLIB**セクションの**JCLLIB**キーに登録されているデータセットのメンバーとして登録することをお勧めします。JCLが多い場合は、別のPDSを作成してメンバーとして管理することをお勧めします。

OpenFrameでは、PDSが実際のディレクトリとして作成されているため、そのディレクトリにJCLをコピーしておくと、PDSのメンバーになります。

JCLをPDSのメンバーとして登録しておくと、JCLをサブミットする際、データセット名とメンバー名だけで対象JCLを検索することができます。特にJCLがOpenFrame環境設定の**tjes**サブジェクトの**PROCLIB**セクションの**JCLLIB**キーに登録されているデータセットのメンバーとして登録された場合は、メンバー名だけで対象JCLを検索できます。JCLLIBに指定されたPDSの順でデータセットのメンバーを検索します。

参考

1. **tjes**サブジェクトの**PROCLIB**セクションの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

2.3. ジョブの処理手順

TJESを介して実行されるすべてのジョブは、以下の手順で処理されます。

1. サブミット (Submit) : TJESからJCLを受け取ります。
2. スケジューリング (Schedule) : 条件を満たすランナー・スロットにジョブを割り当てます。
3. 実行 (Execution) : ランナーがジョブを実行します。
4. 出力 (Output) : ジョブの実行が終わり、その出力が処理されます。
5. 削除 (Remove) : 実行済みのジョブをTJESから削除します。

2.3.1. サブミット

サブミットでは、受け取ったJCLを分析してジョブ単位でTJESに送信します。

JCLがサブミットされると、TJESはそのJCLを分析してジョブ・エントリを構成し、JOBID、SPOOLなど、ジョブの管理に必要なリソースを割り当てた後、JOBQに保存します。サブミットされたジョブの情報はスケジューラーに通知され、スケジューラーによってランナー・スロットに割り当てられます。

以下は、TJESによってジョブに割り当てられるリソースです。

リソース	説明
JOBID	JOBIDは、TJESがジョブを識別するために割り当てる一意のIDです。TJESによって管理されるJOBIDは「JOBnnnnnn」または「J0nnnnnn」形式であり、OpenFrame環境設定のtjesサブジェクトのJOBDEFセクションのENDNUMキーで指定します。ジョブに指定できるJOBIDの有効範囲は、00001から99999、または000001から999999です。 JOBIDはサブミット時に発行され、ジョブがCANCELまたはREMOVEコマンドによって削除されるときに返されます。返されたJOBIDは再使用されます。 そのほか、CICSで生成される「STCnnnnnn」、TSOで生成される「TSOnnnnnn」形式のJOBIDがありますが、これらはTJESが管理していません。
SPOOL	TJESは、各ジョブごとに独立したスプール領域を割り当てます。スプール・ボリュームに作成され、JOBIDをディレクトリ名とします。 SPOOLには、サブミットされたJCLを保存するためにコピーしたINPJCL、カタログをコピーしたCATPROC、ジョブの実行結果であるSYSOUT、TJESがジョブの実行状態をレポートするために内部で使用するファイルなどが格納されます。内部で管理するファイルは、設定によりシステム・テーブルに格納することもできます。 SPOOLは、tjesinitツールを使用して初期化します。

リソース	説明
JOBQ	TJESは、ジョブの検索や変更などを容易にするため、システム・テーブルにジョブの基本情報を保存します。 JOBQに保存されたジョブの情報は、tjesinitツールを使用して初期化することができます。

以下は、TJESでJCLがサブミットされる手順です。

1. JCL構文エラーを確認します。プロシージャの有無と構文エラーはこの時点では確認できず、tjclrunによってJCLが解析されるときに確認されます。
2. JOBIDを発行します。
3. SPOOLを作成します。
4. サブミットされたJCLをコピーしたINPJCLをSPOOLに格納します。
5. JCLのジョブに関する情報をJOBQに格納します。
6. JCLのJOB文にTYPRUN=HOLDが記述されているか、OpenFrame環境設定の**tjes**サブジェクトの**JOBCLASS**セクションの特定のCLASSキー値にHOLD属性が指定されていない場合は、obmjschdに通知してスケジューリングします。

サブミット中にエラーが発生すると、OpenFrame環境設定の**ofsys**サブジェクトの**DIRECTORY**セクションの**LOG_DIR**キーに指定されたパスのジョブのサブディレクトリに、submit_YYYYMMDD.logという名前でファイルが保存されます。正常にサブミットされたジョブは、STARTまたはHOLDのいずれかの状態になります。

tjesmgrのRUNコマンドを使用してJCLを正常にサブミットすると、それによって作成されたジョブがJOBQに登録されます。

参考

1. サブミット・ログの形式の詳細については、[「2.5.1. サブミット・ログ」](#)を参照してください。
 2. JOBQの出力の詳細については、[「6.3.6. PS\(Print Screen\)」](#)を参照してください。
 3. **ofsys**サブジェクトと**tjes**サブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

以下は、OpenFrameがサポートするJCLのサブミット方法です。

- tjesmgrのRUNコマンドを使用します。詳細については、[「第6章 TJESMGRコマンド」](#)の[「6.2.12. RUN」](#)を参照してください。
- textrunツールを使用します。詳細については、『OpenFrame ツールリファレンスガイド』の「4.4 textrun」を参照してください。

- **OpenFrame Manager**の[Batch]メニューを使用します。詳細については、『OpenFrame Manager ユーザーガイド』を参照してください。

2.3.2. スケジューリング

スケジューリングでは、以下の条件を満たすランナー・スロットに現在START状態のジョブを割り当てます。

- ランナー・スロットがACTIVE状態(有効な状態)であり、別のジョブが実行中でない場合に限りま
- RUNNER CLASSとJOB CLASSが一致する必要があります。
- ジョブのサブミット時に、特定のノードでジョブが実行されるように明示した場合は、そのノードとランナー・スロットが属するノードが一致する必要があります。

以下は、スケジューリング・メソッドを疑似コードで示しています。

```
Priority aging
For i = 1 to 8 / * to iterate RUNNER CLASSES */
  For each idle runner {
    Get the list of Job which matches the ith class of the runner
    For each Job in descending priority order {
      Check node affinity. If fails, try next Job
      Check Jobname duplication. If fails, try next Job
      Assign the Job to the runner and break
    }
  }
}
```

JOB CLASS

JOB CLASSは、ジョブの属性です。JCL JOB文のクラス・パラメータにA～Zまたは0～9のいずれかの値を記述します。

JCLにクラスが記述されていない場合は、OpenFrame環境設定の**tjclrun**サブジェクトの**JOB**セクションの**CLASS**キーに記述されたデフォルトのJOB CLASSを使用します。JOB CLASSは、ジョブ・スケジューリングの際に、ジョブを実行できるランナー・スロットを制限するために使用されます。AというJOB CLASSを持つジョブは、RUNNER CLASSにCBAまたはABCのようにAが含まれたランナー・スロットにのみ割り当てられます。

また、JOB CLASSのサブ属性であるHOLDによってTJESの動作方式が変わります。HOLD属性が指定されたJOB CLASSでサブミットされたジョブは、JCL JOB文にTYPRUN=HOLDが指定されていなくてもHOLD状態でサブミットされます。

参考

tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

RUNNER CLASS

RUNNER CLASSは、ランナー・スロットの属性です。OpenFrame環境設定のtjesサブジェクトのINITDEFセクションに記述されます。

RUNNER CLASSは、JOB CLASSと一致する必要があるため、A～Zまたは0～9のJOB CLASSと同じ値を持ちます。1つのランナー・スロットは、最大8つのクラスを持つことができます。

複数のRUNNER CLASSが設定された場合は、先に記述されたクラスのジョブがない場合にのみ後述されたクラスを使用することができます。つまり、最初に設定されたクラスが最優先され、最後に設定されたクラスが最も低い優先順位を持ちます。また、i番目に記述されたクラスのジョブを実行するには、TJESのすべてのランナー・スロットがi番目以前に記述されたクラスのジョブを実行できない状態である必要があります。

たとえば、TJES内の1番のランナー・スロットはRUNNER CLASSがABCであり、2番ランナー・スロットはBCAの場合、RUNNER CLASSがCのジョブは、Bのジョブがない場合に、2番ランナー・スロットの2番目のクラス・ジョブとして実行できます。また、Cのジョブを1番ランナー・スロットの3番目のクラス・ジョブとして実行するには、AとBのジョブが存在せず、2番ランナー・スロットが別のジョブを実行中であるか、無効状態であるため、2番目のクラス・ジョブを実行できない状態である必要があります。

参考

tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

2.3.3. 実行

ランナーがJCL記述に基づいてジョブを実行する段階です。

ランナーはスプールにコピーされているJCLを解析し、その結果に従ってジョブを実行します。1つのジョブは1つ以上のジョブ・ステップで構成されており、ランナーは、JCLに記述されたジョブ・ステップに従ってジョブを順次実行します。また、ジョブの実行時に必要なデータセットの割り当てもこの段階で行われます。

2.3.4. 出力

ジョブの実行中に生成されたSYSOUTを処理します。SYSOUTは出力という単位で処理されます。また、出力クラスによって処理できるプリンターと基本的な後処理が決まります。

出力はプリンターに出力されるか、インターナル・リーダーを介してTJESにサブミットされます。詳細については、[「第5章 出力処理」](#)を参照してください。

2.3.5. 削除

ユーザー・コマンドによって出力段階にあるジョブがTJESから削除されます。

ジョブに割り当てられたJOBID、SPOOL、JOBQなどのすべてのリソースがシステムに返されます。TJESから削除したジョブに再接続するためには、スプール・バックアップ・コマンドを使用してバックアップする必要があります。バックアップに成功すると、スプールからジョブが自動削除されます。

2.4. ジョブの検索

サブミットされたジョブは、JCLのパラメータに記述された値によって、START状態またはHOLD状態でJOBQに登録されます。

JOBQに登録されたすべてのジョブは、tjesmgrのPSコマンドでジョブ状態を確認することができます。START状態のジョブはスケジューリングによってWORKING状態に切り替わり、実行後は、ERROR、DONE、またはFLUSH状態で終了します。

以下のtjesmgrコマンドを使用して、ジョブの詳細情報を表示できます。

コマンド	説明
PSJOB	ジョブの詳細情報を表示します。
PSIO	ジョブで使用したデータセットの入出力情報を表示します。
POSPPOOL	ジョブで生成されたスプール情報を表示します。
PODD	ジョブで生成されたスプールの内容を表示します。
POJOB	ジョブの出力の詳細情報を表示します。

参考

ジョブ情報の表示についての詳細は、[「6.3. ジョブ出力コマンド」](#)を参照してください。

2.5. ログ

TJESでは、実行中に発生するイベントをログに記録しています。

以下は、OpenFrame/BatchシステムでTJESが記録するログです。

- JCLのサブミットに関する情報が保存されるサブミット・ログ
- すべてのジョブ状態の変更が記録されるジョブ・ログ

tjes.confファイルの[LOG]セクションにログ・ファイル・パスを設定する際、ファイル名に「&SYSDATE」を挿入すると、「&SYSDATE」がYYYYMMDD形式の日付に置き換えられ、日付ごとにログが生成されます。

2.5.1. サブミット・ログ

サブミット・ログは、OpenFrame環境設定の**ofsys**サブジェクト、**DIRECTORY**セクションの**LOG_DIR**キーに設定されたファイル・パスのjobサブディレクトリに、submit_YYYYMMDD.logという名前でファイルが保存されます。サブミット・ログは、成功または失敗に関係なく、obmjmsvrが検知したすべてのJCLのサブミット試行を記録します。

以下は、サブミット・ログの基本形式です。

```
>> timestamp    jclpath
submit_result
```

項目	説明
timestamp	yyyymmddHHMMSS形式の14桁の数字です。
jclpath	サブミットされたJCLのUNIX絶対パスです。
submit_result	サブミットの結果を示します。

2.5.2. ジョブ・ログ

ジョブ・ログは、OpenFrame環境設定の**ofsys**サブジェクトの**DIRECTORY**セクションの**LOG_DIR**キーに設定されたファイル・パスのジョブのサブディレクトリに、job_YYYYMMDD.logという名前でファイルが保存されます。

以下は、ジョブ・ログの基本形式です。

```
[YYYY-MM-DDTHH:MI:SS.FFFFFFFF][MODULE(PID)][MSG_LEVEL][MSG_CODE]
CMD=operation,NODE=nodename,USER=userid,JOBID=jobid,JOBNAME=jobname,additional
info
```

以下は、共通項目についての説明です。

項目	説明
CMD	操作のタイプを指定します。 – SUBMIT : ジョブをサブミットします。 – EXECUTE : ジョブをランナーに割り当てます。 – CHANGE : ジョブの属性を実行前に変更します。 – FINISH : ジョブの実行を終了します。 – REMOVE : TJESからジョブを削除します。 – SUSPEND : ジョブを一時停止します。 – RESUME : 一時停止されたジョブを再実行します。 – NICE : ジョブのCPU使用時間に対する相対的な優先順位を指定します。

項目	説明
	– STEP : ジョブの特定のステップの実行情報です。 – CANCEL : ジョブを取り消します。
NODE	ジョブが実行されるノードの名前です。 JCLにジョブが実行されるexecution_nodeを指定していないか、OpenFrame環境設定にtjesmgrサブジェクト、DEFAULT_OPTIONセクションのDEFAULT_RUNNING_NODEキーのVALUE項目を「MY」に設定していない場合は、すべてのノードで実行できるようにアスタリスク(*)が設定されます。
SER	ジョブを実行したユーザーの名前です。CMD=FINISHの場合は、TJESが出力されます。
JOBID	サブミットされたジョブのジョブIDです。
JOBNAME	ジョブの名前です。
additional info	この値はCMD項目の値によって異なります。

SUBMIT

ジョブをサブミットします。

- 構文

```
CLASS=class, STATUS=status, JCL=path
```

項目	説明
CLASS	サブミットされたジョブのクラスです。
STATUS	ジョブのサブミット状態によって設定されます。 – S : START状態 – H : HOLD状態 – D : DONE状態
JCL	コピーされたINPJCLのパスではなく、サブミットされたJCLソース・ファイルのパスです。

EXECUTE

ジョブをランナーに割り当てます。

- 構文

INDEX=*index*, PID=*pid*, CLASS=*class*

項目	説明
INDEX	ジョブを実行しているランナー・スロットのインデックスです。実行中のノードでのみ有効です。
PID	ジョブを実行しているランナーのプロセスIDです。実行中のノードでのみ有効です。
CLASS	ランナー・スロットに割り当てられた複数のクラスのうち、ジョブを実行するために使用しているクラスです。

CHANGE

ジョブの属性を実行前に変更します。

- 構文

CLASS=*class*, PRIORITY=*priority*, STATUS=*status*

項目	説明
CLASS	変更後のジョブのクラスです。
PRIORITY	変更後のジョブの優先順位です。
STATUS	変更後のジョブ状態を示します。 – S : START状態 – H : HOLD状態

FINISH

ジョブの実行を終了します。

- 構文

INDEX=*index*, PID=*pid*, STATUS=*status*, RCODE=*exitcode*

項目	説明
INDEX	ジョブを実行しているランナー・スロットのインデックスです。実行中のノードでのみ有効です。
PID	ジョブを実行しているランナーのプロセスIDです。実行中のノードでのみ有効です。
STATUS	ジョブの終了状態です。

項目	説明
	– D : DONE状態 – E : ERROR状態 – T : STOP状態 – F : FLUSH状態
RCODE	ジョブの終了コードです。詳細については、 「2.7. ジョブの終了コード」 を参照してください。

REMOVE

TJESからジョブを削除します。

- 構文

```
STATUS=status
```

項目	説明
STATUS	削除されたジョブの終了状態です。 – D : DONE状態 – E : ERROR状態 – T : STOP状態 – F : FLUSH状態

SUSPEND

ジョブを一時停止します。

- 構文

```
INDEX=index, PID=pid, STATUS=status
```

項目	説明
INDEX	ジョブを実行しているランナー・スロットのインデックスです。実行中のノードでのみ有効です。
PID	ジョブを実行しているランナーのプロセスIDです。実行中のノードでのみ有効です。
STATUS	一時停止されたジョブの状態です。 – P : SUSPEND状態

RESUME

一時停止されたジョブを再実行します。

- 構文

```
INDEX=index,PID=pid,STATUS=status
```

項目	説明
INDEX	ジョブを実行しているランナー・スロットのインデックスです。実行中のノードでのみ有効です。
PID	ジョブを実行しているランナーのプロセスIDです。実行中のノードでのみ有効です。
STATUS	再実行されたジョブの状態です。 – W : WORKING状態

NICE

ジョブのCPU使用時間に対する相対的な優先順位を指定します。

- 構文

```
INDEX=index,PID=pid,STATUS=status,NICE=nice
```

項目	説明
INDEX	ジョブを実行しているランナー・スロットのインデックスです。実行中のノードでのみ有効です。
PID	ジョブを実行しているランナーのプロセスIDです。実行中のノードでのみ有効です。
STATUS	NICEコマンドを実行したジョブの状態です。 – P : SUSPEND状態 – W : WORKING状態
NICE	0 ~ 20の数字で、数字が低いほど優先順位が高いです。

STEP

ジョブの特定のステップの実行情報です。

- 構文

STEPNAME=stepname, SKIP=skip, STEPSEQ=seqnum, RCODE=exitcode

項目	説明
STEPNAME	ジョブ・ステップ名です。
SKIP	ジョブ・ステップの実行をスキップするかどうかです。 – Y: ジョブ・ステップが実行されません。 – N: ジョブ・ステップが実行されます。
STEPSEQ	ジョブ・ステップの順番です。1から始まります。
RCODE	ジョブの終了コードです。詳細については、「 2.7. ジョブの終了コード 」を参照してください。

CANCEL

ジョブを取り消します。

- 構文

STATUS=status

項目	説明
STATUS	取り消されたジョブの状態です。 – S: START状態 – H: HOLD状態

2.6. ジョブのバックアップ

多くのジョブを実行すると、ジョブをサブミットできなくなる状態までJOBQにジョブが蓄積する可能性があります。そのため、古いジョブをJOBQから削除する必要がありますが、削除するジョブ情報を残したい場合は、スプールをバックアップします。

以下の表は、スプールのバックアップに関連するコマンドとその機能です。

コマンド	説明
SPOOLBACKUP	JOBQからジョブを削除し、削除されたジョブのスプールをバックアップします。
SPOOLBACKUPLIST	スプールがバックアップされた日付を表示します。
SPOOLPS	該当の日付にバックアップされたジョブを表示します。
SPOOLRESTORE	バックアップされたジョブを一時ディレクトリに復元します。

コマンド	説明
SPOOLPSJOB	復元されたジョブの詳細情報を表示します。(PSJOBと同一情報を出力)
SPOOLPSIO	復元されたジョブの入出力情報を表示します。(PSIOと同一情報を出力)
SPOOLPOSPool	復元されたジョブのプール一覧を表示します。(POSPoolと同一情報を出力)
SPOOLPODD	復元されたジョブのプール内容を表示します。(PODDと同一機能)
SPOOLCLEAR	一時ディレクトリに復元されたジョブを削除します。

参考

詳細については、[「6.6. スプール・バックアップ・コマンド」](#)を参照してください。

2.7. ジョブの終了コード

以下では、TJESでのジョブ終了コードとジョブ・ステップ終了コードを定義します。

2.7.1. ジョブ・ステップ終了コード

ジョブ・ステップが終了する際に、次の形式でリターンコードを返します。

XNNNN

• X

終了状態を示します。R、S、U、A、F、Mのいずれかが指定されます。

項目	説明
R	標準状態です。アプリケーションが正常終了したことを示します。
S	システムABENDです。アプリケーションがシグナルで終了したことを示します。 SIGPIPE、SIGSEGV、SIGABEND、SIGBUGなどのシグナルのうち、プロセスを終了させるシグナルによってプロセスが異常終了した場合です。
U	ユーザーABENDです。アプリケーションがABEND状態であることをレポートし、WAASABNDによって終了したことを示します。
A	アプリケーションABENDによって該当のアプリケーションは正常終了したものの、そのRC値を呼び出したプログラム(IKJEFT01など)が条件を満たしていないと判断したため、ジョブを終了することを示します。
F	FLUSH状態です。ランナーの内部エラーによって、アプリケーションが正常に実行されなかったことを示します。

項目	説明
M	JOB文にTYPRUN=JEMと記述されているか、tjesmgr JEMコマンドで実行したジョブの結果でのみ返されるリターンコードです。

- NNNN

NNNN: 0~4096までの数字で、アプリケーションの戻り値です(255以上は外部RCモジュールを介してリターンコードがレポートされます)。ただし、「F」状態の場合はランナーの戻り値です。

2.7.2. ジョブ終了コード

ジョブ終了コードは、ジョブの終了状態に関する付加情報であり、XNNNNN形式で表示されます。
(X: R、S、U、A、F、Mの6文字と2つの記号文字(+、-)、N: 0-9の数字)

以下は、各ジョブの状態と形式についての説明です。

- DONE

XNNNNN

最後のジョブ・ステップの終了コードです。(Xは、ジョブ・ステップ終了コードと同じです)

- ERROR

XNNNNN

エラーを発生させたジョブ・ステップの終了コードです。(Xは、ジョブ・ステップ終了コードと同じです)

- STOP

ユーザーがSTOPコマンドを実行した場合であり、終了コードは無効です。

- FLUSH

[+-]NNNNN

区分	説明
+	ランナーが異常を検知して終了した場合です。NNNNNは、ランナーからの戻り値です。OpenFrameのエラー・コードとして定義されたエラーについてはエラー・コードを返し、その他の定義されていないエラーに対しては、1が返されます。
-	ランナーが、UNIXシグナルによって異常終了した場合です。NNNNNは、UNIXシグナル番号です。

第3章 ジョブの実行

本章では、実際にジョブを実行するランナーであるtjclrunの動作方式と機能について説明します。また、JCL解析、ジョブの実行とその結果について説明します。

3.1. 概要

ユーザーがサブミットしたジョブがスケジューラーによってスケジューリングされ、実際にオペレーティング・システム上で実行されることをジョブの実行といいます。ジョブは、**tjclrun**によって実行されます。

以下は、tjclrunによってジョブが実行される手順です。

1. スケジューラーがサブミットされたジョブのジョブ・クラスと優先順位などのスケジュール・パラメータおよび現在のシステムで使用可能なランナー・スロットの状態をチェックして、特定のランナー・スロットにジョブを割り当て(スケジューリング)ます。
2. ランナー・スロットは、tjclrunを実行するためのデータをジョブ・スプールに記録し、tjclrunを呼び出します。
3. tjclrunは、ジョブ・スプールを介して渡されたジョブのJCLに従ってジョブを実行します。
4. ジョブの実行が終了したら、tjclrunは自身を呼び出したランナー・スロットにジョブの実行結果をレポートします。

ジョブの実行結果は、TJESが管理するJOBQに格納されます。実行済みジョブの出力およびJOBQに格納されたジョブの情報は、TJESによって管理されます。ユーザーは、実行済みジョブの実行結果を確認または出力することが可能であり、特定のジョブ情報が不要になった場合は削除することができます。

本章では、ジョブを実行するtjclrunの動作と機能について説明します。関連するJCLステートメントと構文については、メインフレームの『JCL Reference Guide』を参照してください。

3.2. ジョブの実行

ジョブを実行するためにtjclrunが呼び出されると、tjclrunはいくつかの初期化を行った後、ジョブ・スプールを介して渡されたJCLを解析し、その結果に基づいてジョブを実行します。

tjclrunは、JCL記述に従ってジョブの各ジョブ・ステップを順に実行します。各ジョブ・ステップでは、データセットの割り当てやバッチ・アプリケーションの実行などが行われます。

EXEC文のPGMパラメータを使用してジョブ・ステップに指定されたバッチ・プログラムを実行し、そのモニタリング情報をTJESに定期的にレポートします。また、実行されたバッチ・プログラムにSYSINデータセットの内容を送信し、SYSOUTデータセットを受け取ってジョブ・スプールに記録

します。バッチ・プログラムの実行が終了したら、その実行結果をTJESにレポートします。その後、ジョブ・ステップの終了状態に応じて次のジョブ・ステップへ進むか、例外に該当する場合はジョブの実行を中止します。

すべてのジョブ・ステップが終了すると、tjclrunは、ジョブの実行結果をTJESにレポートし、割り当てられたデータセットなどのリソースを解放した後終了します。

以下は、tjclrunがジョブの実行中に行われる処理です。

ジョブ	段階
実行	実行の初期化
	ジョブ・ストリームの解析
	実行
	EXEC PGM処理
	EXEC PROC処理
リソース	DD処理
	スペシャルDD
	出力処理
	ジョブ・スプール
制御とモニタリング	ジョブ制御
	ジョブ・レベルのレポート
	ステップ・レベルのレポート
その他	セキュリティ(TACFサポート)
	tjclrunの呼び出し(tjclrunコマンドの使用)
	NICEによるPERFORM処理

3.2.1. 実行の初期化

ランナー・スロットまたはサブミッターによってランナーが呼び出される前に、以下の必須リソースを準備する必要があります。

- JOBID
- JOBQのジョブ項目
- ランナー・スロット
- ジョブ・スプール(INPJCL)

以下は、初期化を実行する手順です。

1. コマンドラインの引数

ランナー・スロットは、tjclrunの呼び出し時に指定した引数(args)を解析し、ジョブIDや実行するJCLファイルの場所、その他のパラメータを受け取ります。

注

デバッグなどの特別な場合は、シェルを介してtjclrunを実行できますが、一般的にはTJESのランナー・スロットを通じて実行します。TJESを介さずにtjclrunを実行した場合、そのジョブはTJESの管理対象から除外されるだけでなく、正常な手順で実行されない可能性があります。

2. 環境設定の読み込み

OpenFrame環境設定のtjclrunサブジェクトの各セクションとキーを読み込み、設定に適したtjclrunの実行方法を決定します。

3. 実行アカウント

ジョブの実行時に、tjclrunプロセスの所有者はTJES管理者のオペレーティング・システムのアカウントと同一です。つまり、オペレーティング・システムのobmというユーザー・アカウントでTJESシステムを起動した場合は、tjclrunプロセスと各ジョブ・ステップで実行されるバッチ・プログラムのユーザーIDもobmになります。

ただし、OpenFrame環境設定のtjclrunサブジェクトのACCOUNTセクションのSETUIDキーの値をYESに設定すると、JCL JOB文のUSERに指定されたユーザーでtjclrunのプロセスを実行することができます。詳細については、[「3.2.13. セキュリティ」](#)の[「setuid root tjclrun」](#)を参照してください。

4. SYSLIBの初期化

tjclrunは、OpenFrame環境設定のtjclrunサブジェクトのSYSLIBセクションに設定されたディレクトリから、ジョブ・ステップに指定されたプログラムを検索します。SYSLIBセクションを設定していない場合は、ユーザーの環境変数に指定されたディレクトリからプログラムを検索します。

JCLにJOBLIB DDまたはSTEPLIB DDが存在する場合は、SYSLIBよりJCLに記述されたJOBLIBおよびSTEPLIBが優先されます。

一部のUNIXシステムでは、tjclrunにルート権限を付与して運用すると、環境変数が削除される問題が発生するため、OpenFrame環境設定のtjclrunサブジェクトのSYSLIBセクションを必ず設定する必要があります。設定しない場合は、tjclrunがJCLに指定されたプログラムの実行またはロードに失敗します。詳細については、[「3.2.13. セキュリティ」](#)の[「setuid root tjclrun」](#)を参照してください。

5. SYSMMSG DDのオープン

tjclrunがジョブを実行する際に発生するエラーおよび詳細についての結果レポートは、ジョブ・スプールのSYSMMSGに保存されます。tjclrunは、初期化段階からログ・メッセージを保存するためにSYSMMSGを予め開いておきます。

以下の場合にログがSYSMMSGに保存されます。

- tjclrunの内部エラーやシステム関数の失敗などの問題を分析するために使用されるtjclrunの内部ログ

tjclrunの問題分析に使用するためのログを出力する場合は、OpenFrame環境設定の**tjclrun**サブジェクトの**DEBUG**セクションの**PROFILE**キーの値をYESに指定します。

- OpenFrameライブラリから出力されるstderr

OpenFrameライブラリから出力されるstderrは、OpenFrameの一般的なログ形式に従うため、SYSMMSGとは出力形式が異なります。

6. ジョブ開始のレポート

tjclrunが指定されたジョブの実行を開始したことをTJESにレポートします。

7. ジョブ・スプールのオープン

ジョブの実行プロセスで使用するジョブ・スプールを開きます。

参考

tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

3.2.2. ジョブ・ストリームの解析

tjclrunは、ジョブ・スプールを介して渡されたJCLファイル(INPJCL)を解析してジョブを実行します。

参考

tjclrunによって実行されるJOB文が含まれた(ジョブ・スプールに格納された)JCLファイルをジョブ・ストリームといいます。

● JCLプロシージャの解析

tjclrunは、ジョブの実行中にJCLファイルを複数回解析します。

JOB文が含まれたINPJCLファイルを解析して順に実行します。そのとき、JCLプロシージャが呼び出されると、該当のJCLプロシージャを解析して実行します。プロシージャの実行が終了したら、INPJCLの実行を続行します。

● JCLLIB/JOBPARM PROCLIB

tjclrunは、JCLLIB文に指定されたJCLライブラリ(ライブラリ、PDS)からCLプロシージャを検索します。

また、JOBPARMコマンドのPROCLIBパラメータに指定されたddnameに該当するJCLライブラリから、JCLプロシージャまたはINCLUDE JCLを検索します。JCLライブラリからも、解析対象が見つからなかった場合は、最後にデフォルト・ディレクトリのSYS1.PROCLIBから検索します。

このディレクトリからも見つからなかった場合は、ジョブを異常終了(FLUSH状態)として処理します。

参考

INPJCL(ジョブ・ストリーム)にJOBPARMコマンドが指定された場合、このコマンドのPROCLIBパラメータに指定されたddnameは、**tjes**サブジェクトの**PROCLIB**セクションにキーとして設定されている必要があります。

● INCLUDE

JCLLIB文またはJOBPARMコマンドのPROCLIBパラメータに指定されたJCLライブラリからインクルードされるJCLを検索します。

3.2.3. 実行プロセス

実行するジョブのINPJCL(ジョブ・ストリーム)を解析した後、tjclrunはJCLに記述された手順に従ってジョブ・ステップを実行します。

参考

JCLのJOB文にRESTARTパラメータが指定されている場合、このパラメータで指定されたジョブ・ステップから実行します。

実行手順

以下は、解析後のジョブの実行手順です。

1. ロック段階(Lock phase)

該当のジョブで使用されるデータセットにロックを設定する段階です。

2. 実行段階(Exec phase)

実際にジョブ・ステップを順序どおり実行する段階です。

tjclrunは、INPJCLやJCLプロシージャを分析して、該当のジョブが使用するデータセット・リストを事前 to 取得します。そして、ジョブの実行前に当該データセットにロックを一括要求することにより、複数のジョブが同時に同一データセットを排他的に(exclusive)使用するとき to 発生するデッドロックを防止することができます。

ジョブで使用されるデータセットへのロック要求が正常に実行されたら、ジョブの実行段階に進みます。実行段階(Exec phase)は以下のように処理されます。

● EXEC PGM STEP (プログラム・ステップ)

処理するジョブ・ステップがバッチ・プログラムを実行する場合、指定されたバッチ・プログラムを実行します。tjclrunは、バッチ・プログラムを自身の子プロセスとして実行し、そのプログラ

ムの実行が終了されるまで待ちます。バッチ・プログラムの終了を待っている間、バッチ・プログラムが実行したデータセット入出力統計などの情報を定期的にTJESにレポートします。

- EXEC PROC STEP（プロシージャ・ステップ）

処理するジョブ・ステップがJCLプロシージャを呼び出す場合、指定されたJCLプロシージャを解析し、その結果に基づいて該当のプロシージャを実行します。呼び出されたJCLプロシージャから別のJCLプロシージャを呼び出すこともできます。これをネストされたJCLプロシージャの呼び出しといいます。ネストされたJCLプロシージャは、最大15レベルまで呼び出すことができます。

JCLプロシージャ・ステップの実行後、そのJCLプロシージャを呼び出した元のステップに戻り、その次のステップから続行します。

条件実行

以下のように、ジョブ・ステップを条件付きで実行することができます。

- EXEC COND

前のジョブ・ステップの終了状態に応じて次のジョブ・ステップに進むか、あるいはスキップするかを決めます。このような条件付きのジョブ・ステップの実行は、EXEC文のCONDパラメータを使用して指定します。

- JOB COND

バッチ・プログラムが終了すると、プログラムの終了状態がJOB文のCOND(例外)を満たすかどうかをチェックします。JOB CONDを満たす場合は、ジョブ・ステップの実行を中止します。

JOB文のCONDを満たさない場合は、次のジョブ・ステップを続行します。

- IF-THEN/ELSE/ENDIF

IF-THEN/ELSE/ENDIF文に記述された条件を評価し、それに基づいてジョブ・ステップを実行します。

3.2.4. EXEC PGM処理

tjclrunの最も重要な機能は、ユーザー指定のバッチ・プログラムを実行することであり、以下のようなプロセスで行われます。

EXEC COND

EXEC PGM STEPを実行する前にEXEC文に記述されたCONDパラメータを評価して、前のステップの実行結果がEXEC CONDを満たす場合(例外の場合)は、EXEC PGM STEPを実行せずにスキップします。

ジョブで実行される最初のジョブ・ステップは、常にEXEC CONDを満たさないものと評価されます。

EXEC CONDを満たさないジョブ・ステップ(EXEC PGM STEP)は、以下の手順で実行されます。

1. ステップの進行状況をレポート

EXEC PGM STEPの処理開始をTJESにレポートします。

2. プログラムの実行権限

EXEC PGMに指定されたバッチ・プログラムが主要なユーティリティである場合、それに対するジョブ・ユーザーの実行権限をチェックします。OpenFrame環境設定の**tjclrun**サブジェクト、**TACF**サブジェクトの**CHECK_UTAUTH**キーの値がYESに設定されている場合にのみ権限をチェックします。

3. STEP DDの割り当て

バッチ・プログラムが使用するデータセットを指定したDD文を処理してデータセットを割り当てます。特定のステップ内で同じDDが見つかった場合は、後に指定されたDDはtjclrunによって変更され、バッチ・プログラムで使用されないようにします。ジョブ・ステップ・レベルのスペシャルDD文に対する付加的な処理もこのステップで行われます。

ジョブ・ステップ・レベルのスペシャルDD文には、STEP CAT DD文、STEPLIB DD文があります。

4. プログラム・パラメータ

EXEC PGMに指定されたバッチ・プログラムに渡される実行パラメータを作成します。バッチ・プログラムは、同パラメータをUNIXプログラムのargcまたはargvを使用して取得できます。

参考

プロシージャ・ステップに指定されたPARMが、呼び出されるJCLプロシージャ内に含まれたEXEC PGMのPARM値を上書きします。

5. 呼び出しと実行

システム関数を使用して、EXEC PGMに指定されたバッチ・プログラムを呼び出し(fork)ます。TJESでのバッチ・プログラムは、tjclrunプロセスの子プロセスとして実行されます。最初に呼び出し(fork)を実行して子プロセスを作成し、子プロセスでEXECを実行して指定されたバッチ・プログラムを開始します。

6. SYSIN/SYSOUTパイプ

OpenFrameでは、バッチ・プログラムが使用するSYSIN DDとSYSOUT DDをUNIXプログラムのstdinとstdoutとして実装しています。

tjclrunは、シェルのコマンドラインを使用してバッチ・プログラムを実行するユーザーと同じ役割をします。tjclrunは、SYSIN DDの内容を読み込んでアプリケーションのstdinに書き込み、stdinからSYSINの内容を読み込むことができますようにします。

また、tjclrunは、アプリケーションのstdoutの内容を読み込んで、SYSOUT DDに該当するデータセットまたはジョブ・スプールに出力します。

SYSIN DDとSYSOUT DDをUNIXシステムのプログラムが共通して使用するstdinとstdoutにマッピングすることにより、OpenFrameのデータセットAPIを使用せずに作成されたUNIXアプリケー

ションを、OpenFrame/Batchシステムで別途処理することなく、バッチ・プログラムとして実行することができます。

参考

1. UNIXアプリケーションのstderrは、SYSOUT DDにstdoutの内容と一緒に記録されます。stdoutとstderrは、アプリケーションの基本的な出力ストリームです。メインフレームにはstderrに該当するDDがないため、stderrをSYSOUT DDと一緒に保存しています。
 2. tjclrunは、JCLに記述されたSYSOUT DDとSYSPRINT DDをシノニムとして処理します。1つのジョブ・ステップにSYSOUT DDとSYSPRINT DDが両方とも指定された場合、SYSPRINT DDは無視され、SYSOUT DDのみ使用されます。
-

7. 監視レポートと待機

tjclrunは、EXEC PGMに指定されたバッチ・プログラムを実行し、終了を待ちながら定期的にアプリケーションで実行されたデータセットのI/O数をTJESにレポートします。

8. STEP DDの後処理

ジョブ・ステップで実行されたバッチ・プログラムが終了したら、その終了状態に応じてDD文のDISPパラメータに指定された後処理を行います。

ジョブ・ステップで実行されたバッチ・プログラムが正常終了した場合は、Normal Dispositionに指定された後処理を、異常終了した場合は、Abnormal Dispositionに指定された後処理を行います。DISPのタイプには、PASS、KEEP、CATLG、UNCATLG、DELETEがあります。

JOB COND

バッチ・プログラムの終了時に返された終了状態(Exit status)がJOB CONDを満たすかどうかを評価し、満たす場合はジョブ・ステップを実行せずに終了します。プログラムの終了状態がJOB CONDを満たさない場合は、次のジョブ・ステップに進みます。

リターンコードのチェック

プログラムの終了状態がJOB CONDを満たす場合、tjclrunは実行したバッチ・プログラムの終了時に返されたリターンコードに応じて、該当のステップを正常として処理するか、あるいはエラーとして処理するかを決めます。正常と判断された場合は、後続するステップを実行し、エラーと判断された場合はそのステップを異常終了し、後続するステップのうち、STEP CONDの異常時に実行するように指定されたステップのみ実行します。異常終了の場合に実行させる条件には、EVENとONLYパラメータがあります。

参考

リターンコードは、OpenFrame環境設定のrcサブジェクトに設定します。詳細については、『OpenFrame 環境設定ガイド』を参照してください。

3.2.5. プロシージャ

EXEC PROCステップによってJCLプロシージャが呼び出された場合、tjclrunは指定されたJCLプロシージャを解析して実行します。そのプロシージャの最終ステップまで実行したら、次のジョブ・ステップを続行します。

JCLプロシージャは、プロシージャが定義された場所に応じて、次の2つに分けられます。

- **入カストリーム・プロシージャ(INSPROC)**

入カストリーム・プロシージャは、そのプロシージャを呼び出すINPJCLにプロシージャの内容が定義されています。入カストリーム・プロシージャはそのプロシージャが定義されているINPJCLでのみ呼び出すことができます。

- **カタログ・プロシージャ(CATPROC)**

カタログ・プロシージャは、JCLプロシージャをメンバーとして持つJCLライブラリに登録されているJCLプロシージャです。カタログ・プロシージャを呼び出す場合は、JCLライブラリに登録されているメンバー名をプロシージャに指定します。PROCLIBは頻繁に使用されるJCLプロシージャを集めたJCLライブラリです。カタログ・プロシージャは異なる複数のJCLから呼び出すことができます。

tjclrunは、最初にINPJCLからEXEC PROC文に指定されたプロシージャを検索し、見つからなかった場合はカタログ・プロシージャとみなして、JCLLIB文またはJOBPARMコマンドのPROCLIBパラメータに指定されたJCLライブラリから該当のプロシージャを検索します。

特定のJCLライブラリから指定されたプロシージャが見つかった場合は、残りのJCLライブラリからは検索しません。

JCLLIB文が存在しない場合のみJOBPARMコマンドのPROCLIBパラメータによって指定されたDDからプロシージャを検索します。(JCLLIB文が優先します)

JOBPARM PROCLIB=ddnameに指定できるddnameは、OpenFrame環境設定の**tjes**サブジェクトの**PROCLIB**セクションにキーとして設定されている必要があります。1つのddnameに複数のJCLライブラリ名を指定することができます。また、JCLLIBと同様に、複数のJCLライブラリの最初から順番に指定されたプロシージャを検索するように設定することができます。

参考

1. カatalog・プロシージャが見つかった場合、tjclrunは、カatalog・プロシージャを解析する前に、そのカatalog・プロシージャの内容をジョブ・スプールのCATPROC DDに保存します。これにより、ジョブの実行結果を確認したり、問題点を分析したりすることができます。
 2. 入カストリーム・プロシージャ(INSPROC)は、INPJCLに含まれているため、別途保存しません。
 3. tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

3.2.6. DD処理

tjclrunは、JCLのDD文に記述されたデータセットをバッチ・プログラムが使用できるように割り当てます。一般的なバッチ・プログラムは自身が使用するデータセットを自ら割り当ててではなく、JCL DD文に記述されたデータセットをtjclrunが割り当てるようにし、その結果を継承して使用します。

データセットの割り当ては、EXEC PGM STEPでバッチ・プログラムを実行する前に行われます。バッチ・プログラムの終了後には、tjclrunによって割り当てられたデータセットへの後処理が実行されます。割り当てられたデータセットは、ジョブの終了時(tjclrunの終了時)に解除されます。

データセットの割り当ては、DD文に記述されたデータセットの特性に応じて以下のように分けられます。

● NORMALデータセット

最も一般的なデータセットであり、割り当てるデータセット名がDSNAMEパラメータで指定された場合です。OpenFrameのデータセット割り当てモジュールを呼び出してデータセットを割り当てし、アプリケーションがそのデータセットを使用できるように、割り当て結果をアプリケーションに渡します。

● SYSOUTデータセット

DD文にSYSOUTパラメータが指定されたデータセットです。SYSOUTデータセットは、ジョブの実行結果として生成された出力データをジョブ・スプールに保存します。

参考

ddnameがSYSOUTの場合と、DD文にSYSOUTパラメータが指定されたデータセットは、異なるデータセットです。後者の方がSYSOUTデータセットであり、SYSOUT DDはDD名がSYSOUTである場合です。通常、SYSOUT DD SYSOUT=*形式でコーディングされるため、注意が必要です。

● 一時データセット

DD文のDSNAMEパラメータで指定されたデータセット名が「&&」で始まるか、またはDSNAMEパラメータが指定されていないSYSOUTデータセット以外のデータセットは一時データセットです。tjclrunは、内部的に一時データセットに一意的な名前を与えます。一時データセットは、ジョブの実行時に生成され、ジョブが終了すると削除されます。

tjclrunは、ユーザーが指定した一時データセット以外にも、いくつかの内部用の一時データセットを使用しています。以下はその例です。

– 入力ストリーム・データセット

INPJCL(ジョブ・ストリーム)に「DD *」または「DD DATA」形式で記述されたデータセットです。

tjclrunは、入力ストリーム・データセットに内部用の名前を与えてジョブ・スプールに保存します。このデータセットは、一般的なデータセットと同様に割り当てられ、バッチ・プログラム

によって使用されます。tjclrunは、ジョブ・スプールに一時的に保存された入カストリーム・データセットがジョブの終了時に削除されるよう、一時データセットとして作成しています。

3.2.7. スペシャルDD

DD文は以下の形式でJCLにコーディングされます。

```
//ddname DD [parameters,...]
```

システムで特殊な用途で使用するために予約されたDD名を持つDD文をスペシャルDD文といいます。

参考

特殊な用途で使用するために予約されたスペシャルDDのDD名を約束された用途以外で使用することはできません。

TJESは以下のようなスペシャルDDをサポートします。

● **SYSIN DD**

現行のジョブ・ステップにが指定されている場合、tjclrunはSYSIN DD文に指定されたデータセットの内容を読み込んで、tjclrunが実行したバッチ・アプリケーションのstdinに渡します。つまり、アプリケーションはSYSIN DDによって割り当てられたデータセットの内容をstdinから読み込むことができます。

● **SYSOUT DD/SYSPRINT DD**

現行のジョブ・ステップにSYSOUT DDまたはSYSPRINT DDが存在する場合、tjclrunが呼び出すバッチ・プログラムの代わりに、このDDが指定するデータセットを解放します。また、バッチ・プログラムがstdoutまたはstderrで出力する内容を読み込んで、バッチ・プログラムに代わってSYSOUT DDまたはSYSPRINT DDによって割り当てられたデータセットに出力します。これは、OpenFrameのデータセットI/O関数を使用せず、stdoutまたはstderrに直接メッセージを出力するほとんどのアプリケーションを、JCLで問題なく実行できるようにするためです。

SYSOUT DDとSYSPRINT DDは同様の意味で取り扱います。2つのDDが両方存在する場合は、常にSYSOUT DDを使用し、SYSPRINT DDは使用しません。

● **JOBLIB DD/STEPLIB DD**

JOBLIB DDとSTEPLIB DDは、tjclrunがEXEC PGM=pgmnameに指定されたバッチ・アプリケーションを検索するPDSデータセット(ライブラリ・パス)を指定します。

区分	説明
JOBLIB DD	JOBLIB DDはINPJCLのJOB文と最初のEXEC文の間に指定できます。また該当のジョブで呼び出されるプログラムを検索するPDS(ライブラリ)を指定します。DD文の連結(concatenation)機能を利用して、複数のPDSからプログラムを検索するように指定できます。

区分	説明
STEPLIB DD	該当のジョブ・ステップのEXEC文の次に位置し、ジョブ・ステップで呼び出すプログラムを検索するPDS(ライブラリ)を指定します。JOBLIBと同様に、DD文の連結機能を使用して、複数のPDSからプログラムを検索するように指定できます。STEPLIB DDが指定されている場合、ジョブ・ステップで呼び出すプログラムを、STEPLIB DDが指定するPDS(ライブラリ)から検索します。この場合、JOBLIB DDは使用されません。

JOBLIB DDは、STEPLIB DDのないジョブ・ステップで使用されます。JOBLIB DDとSTEPLIB DDのいずれも指定していない場合は、OpenFrame環境設定の**tjclrun**サブジェクトの**SYSLIB**セクションに設定された場所から実行するバッチ・プログラムを検索します。

参考

JOBLIBやSTEPLIB DDの連結機能については、メインフレームの『JCL Reference Guide』を参照してください。

● JOBCAT DD/STPCAT DD

JOBCAT DDとSTPCAT DDは、ジョブやジョブ・ステップで使用するデータセット・カタログを指定します。両方ともDD文の連結(concatenation)機能を利用して複数のデータセット・カタログが指定できます。

区分	説明
JOBCAT DD	ジョブ・ストリームのJOB文と最初のEXEC文の間にのみ指定できます。JOBCAT DDは該当のジョブで基本的に使用するデータセット・カタログを指定します。
STPCAT DD	ジョブ・ステップのEXEC文の次に位置し、当該ジョブ・ステップで使用するデータセット・カタログを指定します。

JOBCATとSTPCATが両方指定されている場合は、STPCATが使用されます。JOBCATのみ指定されたジョブ・ステップでは、JOBCATが使用されます。JOBCATとSTPCATのいずれも指定されていない場合は、OpenFrameシステムのマスター・カタログが使用されます。

JOBCATやSTPCATが指定されている場合でも、指定されたカタログにデータセットがカタログ化されていない場合は、マスター・カタログからデータセットのカタログ情報を検索します。

3.2.8. 出力処理

JCL OUTPUT文は、ジョブ・スプールに作成されるSYSOUTデータセットの出力処理方法を設定または制御するためのJCL文です。SYSOUTデータセットを示すDD文は、1つ以上のOUTPUT文を参照して該当の出力の処理方法を指定できます。

参考

OUTPUT文に指定できる大部分のパラメータは、OUTPUT文を使用せずに直接SYSOUTデータセットを示すDD文に指定することもできます。ただし、DD文には固有の用途があり、また複数のDDに重複してコーディングすることを避けるために、OUTPUT文を使用して出力制御を行うことを推奨します。OUTPUT文には指定できるが、SYSOUT DD文には指定できないパラメータもあります。

3.2.9. ジョブ・スプール

tjclrunは、ジョブの実行中にさまざまな目的でジョブ・スプールを使用します。

以下は、tjclrunがジョブ・スプールに作成するデータセットについての説明です。

- INPJCL

tjclrunが実行するジョブ・ストリームが含まれているJCLファイルです。

ほとんどのジョブ・スプール・ファイルはtjclrunによって作成されますが、INPJCLは、tjclrunがジョブをサブミットする際に使用できるように、TJESのSUBMITTER(obmjmsvr)によって事前にジョブ・スプールに作成されます。INPJCLの内容は、ユーザーがサブミットしたJCLファイルの内容と同じです。

- CATPROC

tjclrunがジョブを実行する際、カタログ・プロシージャを処理するときに使用したカタログ・プロシージャの内容を保存するスプール・データセットです。

1つのジョブが複数のカタログ・プロシージャを呼び出すことができるため、CATPROCにはジョブが使用した複数のカタログ・プロシージャの内容が保存されます。これらのカタログ・プロシージャは、tjclrunによって実行された順にCATPROCに保存され、カタログ・プロシージャが呼び出されたジョブ・ステップを示すメッセージが各プロシージャの先頭に追加されます。

- INSDSET

INPJCLに以下の形式で記述されている入カストリーム・データセットをtjclrunが内部で使用するためにジョブ・スプールに作成する一時データセットです。

以下は、入カストリーム・データセットの設定例です。

```
//ddname DD *  
line1  
line2  
...  
lineN  
/*
```

上記例の場合、INSDSETは、line1、line2、...、lineNを内容とする一時データセットであり、当該DDを使用するジョブ・ステップで生成され、ジョブ・ステップの終了時に削除されます。このようなルーチンをINSDSET SHUNTと呼びます。

一時データセットのレコード長は、OpenFrame環境設定の**tjclrun**サブジェクト、**SYSLIB**セクションの**INSDSET_LRECL**キーの設定に従います。この値は、1から4096まで指定することが可能であり、未指定時のデフォルト値は80です。入カストリームがレコードの長さより小さい場合はスペースで埋められ、大きい場合は余分な部分が捨てられます。

参考

1. 入カストリーム・データセットは、非VSAMデータセットと連結されることがあるため、INPJCLから入カストリーム・データセットを抽出してデータセットとしてシャント(SHUNT)しておくと、非VSAM SDSの連結ロジックを使用して一貫した方式で連結を処理することができます。そのため、tjclrunがINSDSET SHUNTを実行しています。
 2. INSDSET_LRECLの設定については、『OpenFrame 環境設定ガイド』の「3.14.3.1. INSDSET_LRECL」を参照してください。
-

● JESJCL

tjclrunがINPJCL、INSPROC、およびカタログ・プロシーヤを解析した結果であるパース・ツリーのテキスト形式で出力したファイルです。問題を分析する際に使用されます。

● JESMSG

tjclrunを実行したランナー・スロットがtjclrunの終了時に該当のジョブ・スプールに記録するデータセットです。実行されたジョブの基本情報とジョブの実行時にレポートされたさまざまな統計情報、そしてジョブ・ステップに関する情報が含まれます。また、TJESシステムにとって重要なジョブ実行に関連する要約情報も含まれています。

● SYSMMSG

SYSMMSGは、tjclrunの主要メッセージ・ログです。SYSMMSGには、ジョブの実行に必要な下位レベルの進行状況や重要な結果メッセージだけでなく、ジョブの処理時に発生したエラー・メッセージも保存されます。

ジョブの実行が異常終了した場合は、原因を分析するために最初にSYSMMSGの内容を検討します。SYSMMSGの内容は、tjclrunがジョブ・ステップを実行した順に時間順で表示されます。

以下は、SYSMMSGに表示される内容です。

- 実行中のジョブ・ステップ名
- ジョブの実行中に割り当てられたデータセットの情報
- tjclrunが呼び出したPGM名とプロセスID(pid)
- tjclrunが呼び出したPGMの終了状態
- tjclrunが現在実行中の動作
- tjclrunの主な動作の成功の可否
- tjclrun内のOpenFrameライブラリのstderrメッセージ

● SYSOUT

SYSOUTデータセットは、以下のようにDD文にSYSOUTパラメータが指定された場合です。SYSOUTデータセットは、他のデータセットとは違ってジョブ・スプールに保存され、一般的に出力データを保存します。

以下は、SYSOUTデータセットの設定例です。

```
//REPORT EXEC PGM=MONTHLY
//SYSIN DD *
2007-07-01,2007-08-01
/*
//SYSOUT DD SYSOUT=*
//RPTOUT DD SYSOUT=*
//RPTERR DD SYSOUT=*
```

SYSOUTデータセットは、ユーザーがバッチ・プログラムのために要求したデータセットとして、ジョブ・スプールに保存されるように特別に指定した場合です。他のジョブ・スプールのデータセットは、TJESが効率的にジョブを実行するために使用するデータセットです。

1つのジョブまたはジョブ・ステップで複数のSYSOUTデータセットを指定できます。上記例では、SYSOUT、RPTOUT、RPTERRがSYSOUTデータセットです。SYSOUTデータセットには、ユーザーが指定したバッチ・プログラムが出力する内容が保存されます。

3.2.10. ジョブ制御

TJESでは、スケジューリングの後、実行中のジョブをSUSPEND(一時停止)、RESUME(再開)、STOP(強制終了)することができます。

TJESでは、ジョブを実行するための複数のサブプロセスに対する制御を**ジョブ制御**といいます。**tjesmgr**または**OpenFrame Manager**の[Batch]メニューなどのユーザー・インターフェースを使用して実行中のジョブを制御することができます。

参考

ジョブ制御コマンドの使用方法については、「[第6章 TJESMGRコマンド](#)」および『[OpenFrame Manager ユーザーガイド](#)』を参照してください。

実行中のジョブに対してジョブ制御コマンドを入力すると、以下の手順で処理されます。

1. TJESがtjclrunにジョブを制御するためのシグナル(SIGUSR1)を送信します。
2. tjclrunは、受信したSIGUSR1に含まれている情報を参照して、要求された動作(SUSPEND/RESUME/STOP)を把握します。
3. UNIXシステムのジョブ制御シグナル(SIGSTOP/SIGCONT/SIGKILL)を、tjclrunがジョブ・ステップで実行したサブプロセスに送信します。
4. サブプロセスは、受信したシグナルに応じて一時停止、再開、または強制終了されます。

5. tjclrunは、サブプログラムに対してジョブ制御を実行した後、その結果をTJESにレポートします。
6. tjclrunは、要求された動作がSUSPENDの場合は、自分自身にSIGSTOPを送信して実行を一時停止し、STOPの場合は実行を終了(exit)します。また、RESUMEの場合は、TJESからtjclrunにSIGCONTを送信して、tjclrunの実行を再開します。実行が再開されたtjclrunは、自身が実行したサブプロセスにSIGCONTを送信します。

参考

tjclrunのジョブ制御は、UNIXシステムのジョブ制御シグナル(SIGSTOP/SIGCONT/SIGKILL)を使用して実装されているため、プロセスの実効ユーザーIDが異なると、サブプロセスにシグナルを送信する際、権限問題によってエラーが発生することがあります。したがって、ジョブ制御を正常に実行するには、tjclrunにroot権限を付与する必要があります。詳細については、[「3.2.13. セキュリティ」](#)のsetuid root tjclrunを参照してください。

3.2.11. ジョブ・レベルのレポート

tjclrunは、ジョブの実行情報をTJESにレポートします。レポートの内容は、ジョブの全般的な情報と特定のジョブ・ステップ情報に分けられます。

以下は、tjclrunがTJESにレポートする情報です。

- **ジョブの開始と終了**

tjclrunがジョブの実行を開始および終了したことをTJESにレポートします。ジョブ終了のレポートには、ジョブの実行が正常に終了したかどうかを示す終了状態と終了コードが含まれます。

- **ジョブ制御の状態**

TJESは、tjclrunが実行しているジョブをSUSPEND/RESUME/STOPさせることができます。tjclrunは、要求されたジョブ制御を実行した後、ジョブの状態をTJESにレポートします。

- **データセットのロック待機**

tjclrunは、ジョブを実行する前にそのジョブで使用するデータセットのロックを要求します。

tjclrunは、ロックを要求したデータセットが他のジョブによってすでにロックされ、使用されている場合、ジョブを処理せずにロックが成功するまで待機します。そのとき、問題の原因となるジョブとロック要求を待っているデータセットをTJESにレポートします。管理者は、レポート情報を検討して、複数のジョブ間でデータセットが共有され、ジョブの処理が遅延される問題を解決することができます。

3.2.12. ステップ・レベルのレポート

以下は、tjclrunのジョブ・ステップ・レベルのレポート情報です。

- **ステップの開始と終了**

tjclrunは、ジョブを構成する各ジョブ・ステップ処理の開始と終了をTJESにレポートしています。

STEP STARTは、現在実行中のジョブ・ステップのパス(ジョブ・ストリームでのロケーション)情報と、ジョブ・ステップのタイプ(PGMまたはPROC)などをレポートします。また、STEP FINISHは、ジョブ・ステップの終了状態、終了コード、実行時間などをレポートします。

● データセットの割り当て

tjclrunは、ジョブ・ステップに指定されたバッチ・プログラムを呼び出す前に、バッチ・プログラムが使用するデータセットを事前に割り当てます(割り当てられるデータセットはJCLのDD文によって指定されます)。

データセットの割り当て情報は、OpenFrameシステムにおいて非常に重要な情報であるため、tjclrunによって割り当てられたddnameとデータセット名は、ジョブ・スプールのSYSMSGとOpenFrame_HOME/log/sysディレクトリのdsalcログに記録されます。

● データセット入出力のレポート

バッチ・プログラムによって使用および変更されたデータセットの入出力情報が反映します。

tjclrunは、バッチ・プログラムの実行中に行われたデータセットの入出力統計情報をTJESに定期的にレポートします。管理者(ユーザー)は、特定のデータセットに対してジョブ・ステップごとに実行された読み書きを監視することができます。

3.2.13. セキュリティ

TACFのサポート機能

tjclrunは以下の2つのTACFセキュリティ機能をサポートします。

1. ジョブ・ステップのEXEC PGMに指定されているプログラムに対し、実行権限があるかどうかをチェックする機能です。
2. DD文に指定されているデータセットに対し、アクセス権限があるかどうかをチェックする機能です。

いずれの場合も、権限がない場合はジョブの実行は異常終了します。

参考

TACFを利用して、特定のTACFユーザーに対してプログラムやデータセットに対する権限を制御する方法については、『OpenFrame TACF管理者ガイド』を参照してください。

TACFの代理ユーザー

ジョブを特定のユーザー権限で実行するために、JCLのJOB文にUSERとPASSWORDパラメータを指定します。そのとき、JCL JOB文のパスワードを暗号化せずに指定するため、JCLファイルを管理する環境によっては、ユーザー・パスワードが公開される恐れがあります。

このようなリスクを防ぐために、TJESでは代理ユーザー(Surrogate User)として指定されたユーザーが、実行ユーザー(Execution User)に代わって、パスワードを指定せずに実行ユーザー権限でジョブをサブミットすることができます。

代理ユーザーが実行ユーザーに代わってジョブをサブミットする場合にも、ジョブは実行ユーザー権限で実行されます。代理ユーザーは、実行ユーザーに代わってジョブをサブミットするだけです。代理ユーザーによるジョブのサブミットは、JCLのJOB文にPASSWORDパラメータが指定されていない場合にのみ有効です。

参考

TACFでジョブを実行するために代理ユーザーを指定する方法は、『OpenFrame TACF管理者ガイド』を参照してください。

setuid root tjcrlun

setuidは、UNIXシステムで実行ユーザーではなく、プログラム・ファイルの所有者権限でプログラムを実行するために、実行プログラムの所有者とファイルの権限を適切に変更して設定することです。setuidされたプログラムであっても、実行ユーザーにプログラムへの実行権限がある場合にのみ実行できます。setuid rootは、プログラムを実行ユーザーではなく、rootユーザー権限でプログラムを実行することです。

プロセスのユーザーとは、UNIXプロセスの実効ユーザーIDを意味します。tjcrlunは、プロセスが実行される間は、tjcrlun.confの設定に関係なく、プロセスの実際のユーザー情報(real userid)を変更しません。UNIXシステムにおけるファイルとシステム・リソースへの権限チェックは、プロセスの実効ユーザーに対して行われます。

ユーザー情報は、プロセスを呼び出したユーザー(オペレーティング・システム・ユーザー)を識別するために使用されます。

以下の2つの事項を考慮して、OpenFrame TJESのインストール後にtjcrlunプログラムにルート実行権限を付与するかどうかを決めます。

1. tjcrlunによって実行されたバッチ・プログラムが、tjcrlunを実行する実効ユーザーではなく、他の実効ユーザーによって実行された場合、tjcrlunは、UNIXのジョブ制御シグナルに基づいたジョブの制御を権限不足によって正常に実行することができません。このような場合は、tjcrlunプログラムにroot実行権限を付与することを考慮する必要があります。

また、管理者がrootユーザーとしてログインし、該当のプロセスに対してジョブ制御シグナルを送信することもできますが、このような作業は時間と手間がかかるため、管理者の負担を増やす要因となります。

このような問題は、tjcrlunによって実行されるバッチ・プログラムに対し、tjcrlunプロセスの実効ユーザーではなく、別のユーザーでsetuidされた場合に発生します。

2. ジョブを実行するtjcrlunおよびジョブ・ステップで実行されるバッチ・プログラムを実行するプロセスをジョブに指定されたユーザー権限で実行するには、OpenFrame環境設定のtjcrlunサブ

ジェクト、**ACCOUNT**セクションの**SETUID**キーの**VALUE**項目を**YES**に設定し、tjclrunプログラムのインストール時にroot実行権限を付与する必要があります。

UNIXオペレーティング・システムで、プログラムの実行中にプロセスを実行するユーザー情報を変更するには、rootユーザー権限が必要であるからです。

このような制約を避けるために、TJESシステムをroot権限で運用するのも1つの方法ですが、システムのセキュリティおよび管理上の理由でroot権限で運用することはお勧めしていません。その代わりに、tjclrunプログラムにsetuid rootを設定すると、tjclrunプログラムの実行ユーザーがrootユーザーでなくても、tjclrunプログラム・ファイルの所有者(root)権限で実行されます。

以下は、setuid rootの設定例です。

```
$ cd $OPENFRAME_HOME/bin
$ su
$ chown 0:0 tjclrun
$ chmod u+s tjclrun
$ ls -l tjclrun
-rwsr-xr--x  root  system  {size}  {date-time}  tjclrun
$ exit
```

ファイル・パーミッションが**-rwsr-xr--x**であり、所有者が**root**に変更されたことを確認します。

参考

詳細については、UNIXガイドの「man chmod」および「man chown」を参照してください。

プログラムが実行された直後のtjclrunはroot権限を持っているため、JCLに指定されたユーザー権限でプロセス・ユーザーを変更することができます。tjclrunは、実行初期の制限されたジョブのみroot権限で実行し、それ以外はJCLに指定されたユーザー権限で実行します。

ジョブのプロセス・アカウントやSTOPなどのジョブ制御を使用しない場合は、tjclrunプログラムに対するsetuidとtjclrun環境設定のsetuid設定は不要です。

参考

IBM AIXの場合、setuid rootが設定されたプログラムを実行するとき、セキュリティ上の理由で共有ライブラリを検索するときに使用する環境変数のLIBPATHが削除されるため、tjclrunが使用するOpenFrame製品の共有ライブラリをロードできない問題が発生します。

この問題は、tjclrunプログラムを使用環境に直接インストールした後、再リンクして解決することができます。これにより、環境変数のLIBPATHに依存せず、リンクを実行するときに検索されたディレクトリから必要な共有ライブラリを見つけることができます。これは、IBM AIX オペレーティング・システムで、外部のユーザーによって変更された共有ライブラリがroot権限を持つプロセスで使用されることを防ぐためのオペレーティング・システム上のポリシーです。

以下は、IBM AIXでtjclrunを再リンクする例です。

```
$ cd $OPENFRAME_HOME/bin
$ cp tjclrun tjclrun.bak
$ cc -q64 -brtl -o tjclrun.new ¥
> ./tjclrun ¥
> -L$OPENFRAME_HOME/lib -L$TB_HOME/client/lib -L$TMAXDIR/lib64 ¥
> -ljcl -licf -ldsio -lspool -ltjes -ljmcliout -lsms -lsaf ¥
> -ldsalc -lpgmdd -lams -lbinfmt -lofcom -llockm -lcli
$ cp tjclrun.new tjclrun
```

OpenFrameのユーザー環境で再リンクを実行した後は、setuid rootを設定します。再リンク時に一緒にリンクするライブラリは(上記例の-ljclから-lcliまで)、ldd tjclrunコマンドによって表示されるOpenFrame製品に含まれているtjclrunが使用するライブラリです。

他のUNIXプラットフォームの場合(Linux)は、ld.so.confファイルなど、trustedライブラリを登録する設定により、共有ライブラリの検索環境変数に依存せず、OpenFrame製品が提供するライブラリを検索することができます。

tjclrunをsetuid root権限でインストールして運用する場合は、OpenFrame環境設定の**tjclrun**サブジェクト、**SYSLIB**セクションの**LIB_PATH**、**BIN_PATH**キーを設定する必要があります。特に、IBM AIXのようなシステムでは必須設定です。それ以外の場合にも、OpenFrame環境設定の**tjclrun**サブジェクト、**SYSLIB**セクションに設定されたパスからのみプログラムや共有ライブラリを検索できるようにするのが、セキュリティやシステムのパフォーマンスの面でメリットがあります。ただし、OpenFrameの設定を変更する権限は厳しく管理する必要があります。

3.2.14. tjclrunの呼び出し

tjclrunは、実行引数と一緒に実行される必要があります。

TJESのランナー・スロットは、ジョブの実行が割り当てられた時点のTJESの状態に基づいて、適切な実行引数と一緒に実行されます。

```
tjclrun RUNMODE JOBID INITINDEX JOBSTREAM JOBPOS [keyword args ...]
```

以下は、tjclrunの引数についての説明です。

引数	説明
RUNMODE	ジョブ(normal run)またはJEM(JOB Emulation)テスト
JOBID	一意のジョブID(例：JOBnnnnn)
INITINDEX	ランナー・スロット・インデックス
JOBSTREAM	JCL文を含むジョブ・ストリームのパス
JOBPOS	ジョブ・ストリーム内に複数のジョブが存在する場合、実行するジョブの位置

- [keyword args]

```
RESTART=* | stepname | stepname.procstepname
```

再開するジョブ・ステップを指定します。アスタリスク(*)は最初のジョブまたはプロシージャ・ステップを表します。stepnameはジョブ・ステップ名、stepname.procstepnameはジョブ・ステップのプロシージャ・ステップを表します。

参考

tjclrunをシェルのコマンドラインで直接実行することも可能です。ただし、この場合、サブミットやスケジューリングなどのジョブ管理段階がスキップされるため、正常なジョブ実行が保証されません。

3.2.15. NICEによるPERFORM処理

メインフレームでは、JCLのJOB文とSTEP文に存在するPERFORMパラメータを使用して、ジョブまたはジョブ・ステップのパフォーマンス・グループを指定し、それらの優先順位を調整しています。

ただしUNIXでは、パフォーマンス・グループは存在しておらず、プロセス・ベースの**NICE**機能を使用してCPUの優先順位を調整しています。OpenFrameでは、PERFORMパラメータとNICE機能をマッピングしてサポートしています。

OpenFrameでPERFORMパラメータの使用やPERFORM値とNICE値のマッピングを設定することができます。OpenFrame環境設定のtjclrunサブジェクト、**PERFORM**セクションにキーとして設定します。詳細については、『OpenFrame 環境設定ガイド』を参照してください。

参考

1. UNIXにてプロセスのNICE値を変更するには、スーパー・ユーザー権限が必要です。そのため、PERFORMを使用する場合には、setuid機能を一緒に使用する必要があります。
 2. NICE値を変更することによって、CPU使用の優先順位を上げることができます。これは、CPU使用時間が長いジョブには有効ですが、I/O処理時間が長いジョブにはほとんど効果がありません。
-

第4章 スプール

本章では、スプール・データセットとスプール・バックアップについて説明します。

4.1. 概要

スプールとは、ジョブの実行に必要な入力およびジョブの実行結果として生成されたスプール・データセットが格納されているボリュームのことを指します。また、スプール・データセットを使用するオペレーションを意味することもあります。

JCLがサブミットされると、TJESではジョブに一意のジョブIDを割り当て、JOBQに登録します。その際、割り当てられたジョブIDでスプール・ボリュームにスプール・スペースが割り当てられます。また、INPJCLというスプール・データセットが生成されたら、JCLをINPJCLにコピーします。その後のジョブの実行からは、INPJCLが入力として使用されます。

tjclrunは、ジョブIDで生成されたスプール・スペースに、JESMSG、JESJCL、SYSMSG、CATPROCなどのスプール・データセットと、JCLに記述されたSYSOUT DDに対するスプール・データセットを作成してジョブの実行に使用します。

作成されたスプール・データセットは、tjesmgrコマンドまたはOpenFrame Managerを使用して確認できます。

4.2. スプール・データセット

スプール・データセットには、ジョブが実行される際に**デフォルトで生成されるデータセット**(INPJCL、SYSMSG、CATPROC、CONVJCL、JESMSG、JESJCL)と、JCLにSYSOUTとして**記述され生成されるデータセット**の2種類があります。

以下は、データセットのTEST.DATASET1をTEST.DATASET2とTEST.DATASET3にコピーした後、TEST.DATASET1を削除するJCLの例です。

<TESTPROC JCL>

```
//TESTPROC JOB CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1)
//STEP01 EXEC IBGPROC
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,KEEP)
//PS010.SYSUT2 DD DSN=TEST.DATASET2,DISP=(NEW,CATLG),
//          DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
//STEP02 EXEC IBGPROC
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,DELETE)
//PS010.SYSUT2 DD DSN=TEST.DATASET3,DISP=(NEW,CATLG),
//          DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
```

このJCLは、カタログ式プロシージャのIBGPROCを使用する2つのステップで構成されています。以下は、IBGPROCの内容と、IEBGENERユーティリティを実行する例を示しています。

<IBGPROC>

```
//PS010   EXEC  PGM=IEBGENER
//STEPLIB DD DSN=SYS1.USERLIB,DISP=SHR
//        DD DSN=SYS1.TEMPLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1  DD DUMMY
//SYSUT2  DD DUMMY
```

JCL TESTPROCをサブミットして実行すると、以下のようなスプール・データセットが生成されます。

```
SP00L LIST :
-----
NO  STEP          DDNAME      SIZE  DSNAME
-----
0   -----      INPJCL       409  INPJCL
1   -----      SYSMSG       5K   SYSMSG
2   -----      CATPROC      444  CATPROC
3   -----      CONVJCL      791  CONVJCL
4   -----      JESMSG       1K   JESMSG
5   -----      JESJCL       3K   JESJCL
6   STEP01/PS010  SYSPRINT  463  oframe.TESTPROC.JOB00040.D000004
7   STEP02/PS010  SYSPRINT  463  oframe.TESTPROC.JOB00040.D000009
-----
```

4.2.1. デフォルト・データセット

本節では、スプール・リストのスプール・データセットについて説明します。

INPJCL

サブミットされたJCLがコピーされています。以降のジョブ実行からは、コピーされたINPJCLを使用します。

```
//TESTPROC JOB CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1)
//STEP01   EXEC  IBGPROC
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,KEEP)
//PS010.SYSUT2 DD DSN=TEST.DATASET2,DISP=(NEW,CATLG),
//           DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
//STEP02   EXEC  IBGPROC
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,DELETE)
```

```
//PS010.SYSUT2 DD DSN=TEST.DATASET3,DISP=(NEW,CATLG),
//              DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
```

SYSMSG

ジョブの実行中にtjclrunが生成したメッセージを保存します。

```
-----
** Cataloged Procedure IBGPROC in SYS1.PROCLIB SCAN start **
-----

** Cataloged Procedure IBGPROC in SYS1.PROCLIB SCAN start **
-----

JOBID=JOB00040   JOBPOS=0   RUNPID=2454
TESTPROC JOB
=====
----- STEP01 EXEC PROC step -----
(JRN0210I) Start parsing Cataloged Procedure ...
----- PS010 EXEC PGM step -----
(JRN0320I) SYSUT1   DD ALLOC - DSN=TEST.DATASET1
(JRN0320I) SYSUT2   DD ALLOC - DSN=TEST.DATASET2
(JRN0320I) STEPLIB  DD ALLOC - DSN=SYS1.USERLIB
(JRN0320I) CONCATED DD ALLOC - DSN=SYS1.TEMPLIB
(JRN0321I) SYSPRINT DD ALLOC - DSN=JOB00040(oframe.TESTPROC.JOB00040.D000004)

----- STEP02 EXEC PROC step -----
(JRN0210I) Start parsing Cataloged Procedure ...
----- PS010 EXEC PGM step -----
(JRN0320I) SYSUT1   DD ALLOC - DSN=TEST.DATASET1
(JRN0320I) SYSUT2   DD ALLOC - DSN=TEST.DATASET3
(JRN0320I) STEPLIB  DD ALLOC - DSN=SYS1.USERLIB
(JRN0320I) CONCATED DD ALLOC - DSN=SYS1.TEMPLIB
(JRN0321I) SYSPRINT DD ALLOC - DSN=JOB00040(oframe.TESTPROC.JOB00040.D000009)

(JRN0210I) Start parsing Cataloged Procedure ...
(JRN0210I) Start parsing Cataloged Procedure ...
=====
----- STEP01 EXEC PROC step -----
(JRN0210I) Start parsing Cataloged Procedure ...
----- PS010 EXEC PGM step -----
EXEC PGM=IEBGENER
(JRN0129I) TACF CHECK UTILITY PROGRAM AUTH OK
(JRN0320I) SYSUT1   DD ALLOC - DSN=TEST.DATASET1
(JRN0320I) SYSUT2   DD ALLOC - DSN=TEST.DATASET2
(JRN0320I) STEPLIB  DD ALLOC - DSN=SYS1.USERLIB
(JRN0320I) CONCATED DD ALLOC - DSN=SYS1.TEMPLIB
(JRN0321I) SYSPRINT DD ALLOC - DSN=JOB00040(oframe.TESTPROC.JOB00040.D000004)
```

```

(JRN0044I) dsalc_report_start OK
(JRN0240I) SYSPRINT DD in STEP01/PS010 open
(JRN0335I) SYSUT1 DD in STEP01/PS010 exported
(JRN0335I) SYSUT2 DD in STEP01/PS010 exported
(JRN0335I) STEPLIB DD in STEP01/PS010 exported
(JRN0335I) SYSPRINT DD in STEP01/PS010 exported
(JRN0056I) coprocess child: argv[0]=IEBGENER
(JRN0058I) pgm pid=2460
(JRN0060I) Entering exec_pipe_loop() ...
(JRN0301I) SYSIN DD to PGM done
(JRN0245I) PGM closed stdout pipe
(JRN0061I) Leaving exec_pipe_loop() ...
(JRN0063I) pipe work done with coprocess child (PGM)
(JRN0080I) PGM pid[2460] EXITED
(JRN0065I) PS010 EXEC PGM step done with RC=0
(JRN0073I) dsalc_report_info OK
(JRN0069I) step res usage: real[ 0.47] user[ 0.05] system[ 0.03] cuser[
0.01] csystem[ 0.01]
(JRN0070I) res usage: cpu_time[ 0.10] proc_time=[ 0.47]
(JRN0345I) SYSUT1 DD DISPOSE COND=NORMAL
(JRN0345I) SYSUT2 DD DISPOSE COND=NORMAL
(JRN0345I) STEPLIB DD DISPOSE COND=NORMAL
(JRN0345I) CONCATED DD DISPOSE COND=NORMAL
(JRN0345I) SYSPRINT DD DISPOSE COND=NORMAL
----- STEP02 EXEC PROC step -----
(JRN0210I) Start parsing Cataloged Procedure ...
----- PS010 EXEC PGM step -----
EXEC PGM=IEBGENER
(JRN0129I) TACF CHECK UTILITY PROGRAM AUTH OK
(JRN0320I) SYSUT1 DD ALLOC - DSNAME=TEST.DATASET1
(JRN0320I) SYSUT2 DD ALLOC - DSNAME=TEST.DATASET3
(JRN0320I) STEPLIB DD ALLOC - DSNAME=SYS1.USERLIB
(JRN0320I) CONCATED DD ALLOC - DSNAME=SYS1.TEMPLIB
(JRN0321I) SYSPRINT DD ALLOC - DSNAME=JOB00040(oframe.TESTPROC.JOB00040.D000009)

(JRN0044I) dsalc_report_start OK
(JRN0240I) SYSPRINT DD in STEP02/PS010 open
(JRN0335I) SYSUT1 DD in STEP02/PS010 exported
(JRN0335I) SYSUT2 DD in STEP02/PS010 exported
(JRN0335I) STEPLIB DD in STEP02/PS010 exported
(JRN0335I) SYSPRINT DD in STEP02/PS010 exported
(JRN0056I) coprocess child: argv[0]=IEBGENER
(JRN0058I) pgm pid=2467
(JRN0060I) Entering exec_pipe_loop() ...
(JRN0301I) SYSIN DD to PGM done
(JRN0245I) PGM closed stdout pipe

```

```

(JRN0061I) Leaving exec_pipe_loop() ...
(JRN0063I) pipe work done with coprocess child (PGM)
(JRN0080I) PGM pid[2467] EXITED
(JRN0065I) PS010 EXEC PGM step done with RC=0
(JRN0073I) dsalc_report_info OK
(JRN0069I) step res usage: real[   0.42] user[   0.04] system[   0.02] cuser[
0.02] csystem[   0.02]
(JRN0070I) res usage: cpu_time[   0.10] proc_time=[   0.42]
(JRN0345I) SYSUT1   DD DISPOSE COND=NORMAL
(JRN0345I) SYSUT2   DD DISPOSE COND=NORMAL
(JRN0345I) STEPLIB  DD DISPOSE COND=NORMAL
(JRN0345I) CONCATED DD DISPOSE COND=NORMAL
(JRN0345I) SYSPRINT DD DISPOSE COND=NORMAL
----- DATA SET UNALLOCATION -----
(JRN0346I) SYSPRINT DD UNALLOC COND=NORMAL
(JRN0346I) STEPLIB  DD UNALLOC COND=NORMAL
(JRN0346I) SYSUT2   DD UNALLOC COND=NORMAL
(JRN0346I) SYSUT1   DD UNALLOC COND=NORMAL
(JRN0346I) SYSPRINT DD UNALLOC COND=NORMAL
(JRN0346I) STEPLIB  DD UNALLOC COND=NORMAL
(JRN0346I) SYSUT2   DD UNALLOC COND=NORMAL
(JRN0346I) SYSUT1   DD UNALLOC COND=NORMAL
(JRN0333I) JOB is enqueued for output processing -  JOBID=JOB00040, ABEND=0

```

以下は、SYSMSGに保存される情報のうち、ステップの実行中に使用されたシステム・リソースを記録するstep res usageとres usageについての説明です。

- step res usage

フィールド	説明
real	ステップの実行時間です。（単位: 秒）
user	ステップがユーザー領域で使用したCPU時間です。（単位: 秒）
system	ステップがシステム領域（カーネル・モード）で使用したCPU時間です。（単位: 秒）
cuser	ステップで生成された子プロセスがユーザー領域で使用したCPU時間です。（単位: 秒）
csystem	ステップで生成された子プロセスがシステム領域（カーネル・モード）で使用したCPU時間です。（単位: 秒）

- res usage

フィールド	説明
cpu_time	ステップが使用した合計CPU時間です。（単位: 秒）

フィールド	説明
proc_time	step res usageのrealフィールドと同じで、該当ステップの総実行時間です。（単位: 秒）
memory size	-1に固定します。

CATPROC

ジョブの実行時にカタログ式プロシージャを使用する場合にプロシージャの内容を保存します。

```

XX----- IBGPROC in SYS1.PROCLIB -----XX
//PS010   EXEC PGM=IEBGENER
//STEPLIB DD DSN=SYS1.USERLIB,DISP=SHR
//         DD DSN=SYS1.TEMPLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1  DD DUMMY
//SYSUT2  DD DUMMY
XX----- IBGPROC in SYS1.PROCLIB -----XX
//PS010   EXEC PGM=IEBGENER
//STEPLIB DD DSN=SYS1.USERLIB,DISP=SHR
//         DD DSN=SYS1.TEMPLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUT1  DD DUMMY
//SYSUT2  DD DUMMY

```

CONVJCL

ジョブの実行で展開されたプロシージャとパラメータの値を置き換えた内容を保存します。カタログ式プロシージャは「XX」で出力され、JCLストリーム内プロシージャは「++」で出力されます。

```

//TESTPROC JOB CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1)
//STEP01   EXEC IBGPROC
XX$DUMMY$ PROC
XXPS010   EXEC PGM=IEBGENER
XXSTEPLIB DD DSN=SYS1.USERLIB,DISP=SHR
XX         DD DSN=SYS1.TEMPLIB,DISP=SHR
XXSYSPRINT DD SYSOUT=*
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,KEEP)
X/SYSUT1   DD DUMMY
//PS010.SYSUT2 DD DSN=TEST.DATASET2,DISP=(NEW,CATLG),
//         DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
X/SYSUT2   DD DUMMY
//STEP02   EXEC IBGPROC
XX$DUMMY$ PROC
XXPS010   EXEC PGM=IEBGENER
XXSTEPLIB DD DSN=SYS1.USERLIB,DISP=SHR

```

```

XX          DD DSN=SYS1.TEMPLIB,DISP=SHR
XXSYSPRINT DD SYSOUT=*
//PS010.SYSUT1 DD DSN=TEST.DATASET1,DISP=(SHR,DELETE)
X/SYSUT1    DD DUMMY
//PS010.SYSUT2 DD DSN=TEST.DATASET3,DISP=(NEW,CATLG),
//          DCB=(RECFM=FB,LRECL=10,BLKSIZE=100)
X/SYSUT2    DD DUMMY

```

参考

JCLの実行に成功した場合にのみCONVJCLにすべての内容が出力されます。実行中に構文エラーが発生した場合にもCONVJCLは作成されますが、一部の内容のみ出力されるか、何も出力されないこともあります。

JESMSG

ジョブの関連情報と各ステップについての情報が保存されます。

```

---< JOB INFO >-----
JOB  ID   : JOB00040
JOB  NAME : TESTPROC      NODENAME : NODE1
JOB  CLASS : A , JOB STATUS : D(R00000) , JOB  PRTY : 5 , RUNNER INDEX : 8
JOB  USER  : ROOT
JCL  PATH  : /home/oframe/OpenFrame/volume_default/SYS1.JCLLIB/TESTPROC
TIME STAMP : SUBM 20200122164142, EXEC 20200122164143, TERM 20200122164145
RES  USAGE : CPU - 0s (p:1s) , 0%
---<STEP INFO>-----
STEP : [STEP01/PS010]
CPU      0s(p: 0s)
SYSUT1   TEST.DATASET1                R:10      W:0
SYSUT2   TEST.DATASET2                R:0       W:10
STEPLIB  SYS1.USERLIB                 R:0       W:0
          SYS1.TEMPLIB                 R:0       W:0
SYSPRINT JOB00040(oframe.TESTPROC.JOB00040.D000004) R:0       W:5
STEP : [STEP02/PS010]
CPU      0s(p: 0s)
SYSUT1   TEST.DATASET1                R:10      W:0
SYSUT2   TEST.DATASET3                R:0       W:10
STEPLIB  SYS1.USERLIB                 R:0       W:0
          SYS1.TEMPLIB                 R:0       W:0
SYSPRINT JOB00040(oframe.TESTPROC.JOB00040.D000009) R:0       W:5

```

以下は、JESMSGに保存されるジョブとステップの情報です。

- ジョブ

フィールド	説明
JOB ID	割り当てられた固有のジョブIDです。
JOB NAME	ジョブ名です。
NODENAME	ジョブが実行されたTJESのノード名です。
JOB CLASS	ジョブ・クラスです。
JOB STATUS	ジョブの現在の状態(戻り値)です。
JOB PRY	実行したジョブのスケジューリング優先順位です。
RUNNER INDEX	ジョブを実行したランナー・スロット番号です。
JOB USER	ジョブを実行したユーザーです。
JCL PATH	ユーザーがサブミットしたJCLファイルのディレクトリとファイル名です。
TIME STAMP	ジョブがサブミットされた時刻、ジョブ実行の開始と終了時刻です。
RES USAGE	ジョブの実行に使用されたCPU時間です。 – CPU – 0s : tjclrunおよびフォークされた子プロセスのCPU使用量の合計(現在はプロセスが終了されたため0です) – (p:2s) : tjclrunがフォークされた時点から終了時までの時間(ジョブの実行時間) – 0% : ジョブの実行時間におけるCPUの使用率

● ステップ

フィールド	説明
STEP	ジョブ・ステップ名です。
CPU	ジョブ・ステップ処理に使用されたCPU時間です。(実際の処理時間)
SYSUT1	ジョブ・ステップで発生したSYSUT1へのI/O数です。
SYSUT2	ジョブ・ステップで発生したSYSUT2へのI/O数です。
STEPLIB	ジョブ・ステップで発生したSTEPLIBへのI/O数です。
SYSPRINT	ジョブ・ステップで発生したSYSPRINTへのI/O数です。

JESJCL

INPJCLに対してJCL文の構文解析ツリーが保存されます。


```

JOB STREAM=[/home/oframe/OpenFrame/spool/JOB00040/INPJCL], JOBPOS=[0]
=====
/JOB[name=TESTPROC, lineno=1]={
  |[key=CLASS]={STR,value=[A]}
  |[key=MSGCLASS]={STR,value=[X]}
  |[key=MSGLEVEL]={SUB,value={
    |[pos=1]={STR,value=[1]}
    |[pos=2]={STR,value=[1]}
  }}
  |----- child statements -----
  /EXEC[name=STEP01, lineno=2]={
    |[pos=1]={STR,value=[IBGPROC]}
  }
  /DD[name=PS010.SYSUT1, lineno=3]={
    |[key=DSN]={STR,value=[TEST.DATASET1]}
    |[key=DISP]={SUB,value={
      |[pos=1]={STR,value=[SHR]}
      |[pos=2]={STR,value=[KEEP]}
    }}
  }
  /DD[name=PS010.SYSUT2, lineno=4]={
    |[key=DSN]={STR,value=[TEST.DATASET2]}
    |[key=DISP]={SUB,value={
      |[pos=1]={STR,value=[NEW]}
      |[pos=2]={STR,value=[CATLG]}
    }}
    |[key=DCB]={SUB,value={
      |[key=RECFM]={STR,value=[FB]}
      |[key=LRECL]={STR,value=[10]}
      |[key=BLKSIZE]={STR,value=[100]}
    }}
  }
  /EXEC[name=STEP02, lineno=6]={
    |[pos=1]={STR,value=[IBGPROC]}
  }
  /DD[name=PS010.SYSUT1, lineno=7]={
    |[key=DSN]={STR,value=[TEST.DATASET1]}
    |[key=DISP]={SUB,value={
      |[pos=1]={STR,value=[SHR]}
      |[pos=2]={STR,value=[DELETE]}
    }}
  }
  /DD[name=PS010.SYSUT2, lineno=8]={
    |[key=DSN]={STR,value=[TEST.DATASET3]}
    |[key=DISP]={SUB,value={
      |[pos=1]={STR,value=[NEW]}
      |[pos=2]={STR,value=[CATLG]}
    }}
  }
}

```



```

|/DD[name=, lineno=5]={
|  | [key=DSN]={STR, value=[SYS1.TEMPLIB]}
|  | [key=DISP]={STR, value=[SHR]}
|}
|/DD[name=SYSPRINT, lineno=6]={
|  | [key=SYSOUT]={STR, value=[*]}
|}
|/DD[name=SYSUT1, lineno=7]={
|  | [pos=1]={STR, value=[DUMMY]}
|}
|/DD[name=SYSUT2, lineno=8]={
|  | [pos=1]={STR, value=[DUMMY]}
|}
}
/PEND[name=, lineno=8]={
}

```

4.2.2. SYSOUTデータセット

主にJCLで記述されたユーティリティまたはアプリケーションで出力として使用されます。以下の例では、STEP01とSTEP02で、IEBGENERメッセージを保存するSYSPRINT文に対してスプール・データセットが作成されています。

<STEP01/PS010 SYSPRINT oframe.TESTPROC.JOB00040.D000004>

```

[2020-01-22T16:41:43.906976] [IEBGENER(2460)          ] [M] [IBG0001M] =====<
IEBGENER START >=====
[2020-01-22T16:41:43.911809] [IEBGENER(2460)          ] [M] [IBG0005M] RECORD
COUNT = 10
[2020-01-22T16:41:43.911817] [IEBGENER(2460)          ] [M] [IBG0006M] TOTAL
RECORD COUNT = 10
[2020-01-22T16:41:43.934682] [IEBGENER(2460)          ] [M] [IBG0002M] =====<
IEBGENER FINISH >=====

```

<STEP02/PS010 SYSPRINT oframe.TESTPROC.JOB00040.D000009>

```

[2020-01-22T16:41:44.490414] [IEBGENER(2467)          ] [M] [IBG0001M] =====<
IEBGENER START >=====
[2020-01-22T16:41:44.493186] [IEBGENER(2467)          ] [M] [IBG0005M] RECORD
COUNT = 10
[2020-01-22T16:41:44.493194] [IEBGENER(2467)          ] [M] [IBG0006M] TOTAL
RECORD COUNT = 10
[2020-01-22T16:41:44.509200] [IEBGENER(2467)          ] [M] [IBG0002M] =====<
IEBGENER FINISH >=====

```

4.3. スプールのバックアップ

JOBQから不要なジョブを削除すると、該当のスプールも削除されます。スプールを保存する必要がある場合は、スプールのバックアップ機能を使用します。

SPOOL BACKUPコマンドを実行すると、OpenFrame環境設定の**tjes**サブジェクト、**SPOOL**セクションの**SPOOL_BACKUP_DIR**キーのVALUE項目に設定されたバックアップ・ディレクトリに現在の日付でディレクトリが作成されます。

このディレクトリにバックアップされたジョブに対し、該当のスプールを圧縮してJOBID_JOBNAME_submitDATE_submitTIME.tar.gz形式のファイルを作成した後、バックアップされたジョブをJOBQとスプールから削除します。バックアップされたスプールを表示する際は、OpenFrame環境設定の**tjes**サブジェクト、**SPOOL**セクションの**SPOOL_UNPACK_DIR**キーのVALUE項目に設定されたディレクトリに圧縮されたJOBID_JOBNAME_submitDATE_submitTIME.tar.gzファイルを解凍した後、そのジョブのスプールを表示することができます。

スプールのバックアップが可能なジョブの状態は、DONE、ERROR、STOP、FLUSHです。

参考

1. スプール・バックアップは、**tjesmgr**コマンドまたは**OpenFrame Manager**の[Batch]メニューを使用して確認することができます。
 2. スプール・バックアップ関連のコマンドについては、[「第6章 TJESMGRコマンド」](#)を参照してください。
 3. OpenFrame環境設定の**tjes**サブジェクトの**SPOOL**セクションの設定方法については、『OpenFrame 環境設定ガイド』を参照してください。
-

第5章 出力処理

本章では、出力の概念および処理方法について説明します。

5.1. 概要

出力処理では、ジョブの実行中に作成されたSYSOUTをジョブの終了後に処理します。

SYSOUTは、出力という単位で処理されます。出力処理は、SYSOUTの内容をプリンターで出力、インターナル・リーダーを使用してJOBQにサブミット、そして指定のエクスターナル・ライターを使用して処理する3つのプロセスで構成されます。

SYSOUTの属性に応じて、実行される処理が決まります。

1. OpenFrameでのプリンター出力とは、物理的なプリンターではなく、外部プリンター・ソリューションに印刷するデータと、印刷に使用されるリソース情報(FORMS、FORMDEF、PAGEDEFなど)を送信することです。外部プリンター・ソリューションにデータと情報を送信するため、OpenFrame環境設定の**print**サブジェクト、**PRINTn**セクションの**COMMAND**、**DRIVER_PATH**キーのVALUE項目に必要なコマンドと共有ライブラリのパスを設定します。
2. インターナル・リーダーを使用してサブミットする場合は、OpenFrame環境設定の**print**サブジェクト、**INTRDR**セクションの**DRIVER_PATH**キーのVALUE項目に必要な共有ライブラリ・パスを設定します。
3. エクスターナル・ライターを使用する場合は、OpenFrame環境設定の**print**サブジェクトの**WRITER**セクションにエクスターナル・ライターの名前と必要な共有ライブラリ・パスを設定します。

参考

printサブジェクトの設定方法については、『OpenFrame 環境設定ガイド』を参照してください。

5.2. 出力

出力は、出力処理の単位です。ジョブの実行が終了すると、そのジョブが作成したスプール・データセットを出力処理する必要があります。

出力は、JCLのSYSOUT文とOUTPUT文によって決まります。通常は、1つのスプール・データセットが1つの出力として処理されますが、以下の例のように1つ以上の出力として処理されることもあります。

以下のようにJCLが構成されると、1つのスプール・データセットが複数の出力処理に使用されます。

```
//OUT01 OUTPUT FORMDEF=FORM1
//OUT02 OUTPUT FROMS=LETTER01, FROMDEF=LTR1, PAGEDEF=LTR4
//PRT DD SYSOUT=A, OUTPUT=(*.OUT01, *.OUT02)
```

上記のJCLでは、PRT DDは、OUT01とOUT02の2つのOUTPUT DDを参照しています。ジョブの実行が終了すると、OUT01のOUTPUT文を参照する出力とOUT02のOUTPUT文を参照する出力が処理されます。

- OUT01を参照する出力が処理されると、FORM1のFORMDEFフォーマットでPRT DDに定義されたスプール・データセットの内容が基本用紙に印刷されます。
- OUT02を参照する出力が処理されると、LTR1のFORMDEFフォーマットとLTR4のPAGEDEFフォーマットで、LETTER01用紙にスプール・データセットの内容が印刷されます。

5.3. 出力処理

ジョブの実行が終了すると、スプール・データセットは出力単位でOUTPUTQに登録されます。登録された出力は、ofrpmsvrによってスケジューリングされ、一般プリンターまたは外部ライターによって処理されるか、あるいは内部リーダーによって再サブミットされます。ofrpmsvrは、OUTPUTQから出力の状態を確認して出力を取り出して、OpenFrame環境設定の**print**サブジェクト、**PRINTn**セクションの**CLASS**キーのVALUE項目と出力クラスを比較してスケジューリングし、OUTPUT DISPOSITIONに従って処理します。

5.3.1. 出力の状態

OUTPUTQに登録された出力の状態を示します。出力の状態は、ofrpmsvrによって処理される際に変更されます。

以下は、出力状態の種類です。

種類	説明
Unable (U)	作成初期の状態です。スケジュールされず、OUTPUTQに待機している状態です。
Ready (R)	OUTPUTQから取り出され、スケジュールされる前の状態です。
Print (P)	スケジューリングされ、ジョブがPRINTERn、内部リーダー、または外部ライターに送信された状態です。
Hold (H)	OUTPUT DISPOSITIONがLEAVEの場合、出力の処理後、OUTPUTQで待機中の状態です。
Cancel (C)	ユーザーによって出力処理が取り消された状態です。
Error (E)	PRINTERn、内部リーダーまたは外部ライターに送信されたジョブにエラーが発生した状態です。

ジョブの実行中に登録される出力の初期状態はUです。ofrpmsvrは、定義的にOUTPUTQをスキャンし、登録された出力からUまたはR状態の出力を取り出してR状態に更新し、ジョブをスケジューリングします。その後、ofrpmsvrは、プリンター、内部リーダー、または外部ライターに出力処理を送信して、P(Print)状態に更新します。OUTPUT DISPOSITIONが「LEAVE」の場合は、出力が正常に処理された後、H状態に変更され、OUTPUTQで待機することになります。

出力処理中にtjesmgrを通したユーザー・コマンドによって出力が「PURGE」されると、その出力はC状態になります。また、プリンター処理中にエラーが発生した場合の出力は、E状態に更新されます。

5.3.2. 出力クラス

出力に付与された1バイトの文字(A-Z、0-9)です。出力クラスはJCLに記述されます。

SYSOUT文にクラスが指定された場合は、OUTPUT文に出力クラスを指定しても、SYSOUT文に指定された値によって上書きされるため、SYSOUT文にはクラスを指定しないでください。

以下は、PRINT DDの出力クラスを「Y」に指定した例です。

```
//OUT1      OUTPUT CLASS=Y,FORM=TEST
//PRINT     DD SYSOUT=(,),OUTPUT=*.OUT1
```

SYSOUT文で出力クラスを指定する場合、PRINT DDの出力クラスがAに指定されます。

```
//PRINT     DD SYSOUT=A
```

出力クラスをアスタリスク(*)に指定した場合は、JOB文のMSGCLASSに指定されたクラスが出力クラスとして指定されます。

以下の例では、PRINT DDの出力クラスが「X」に指定されます。

```
//COPY02    JOB CLASS=I,MSGCLASS=X,MSGLEVEL=(1,1)
...

//PRINT     DD SYSOUT=*
```

出力クラスの優先順位

出力クラスの設定は、以下の順で適用されます。

1. SYSOUT=class
2. OUTPUT CLASS=class
3. JOB MSGCLASS=class

SYSOUT文にクラスが指定された場合は、OUTPUT文にクラスを指定しても、SYSOUT文に指定された値が優先して適用されます。

「SYSOUT=*」または「SYSOUT=(,)」のようにクラスが明示されていない場合は、OUTPUT CLASSパラメータに指定された値が適用されます。SYSOUT文にクラスが指定されておらず、OUTPUT文

にもCLASSパラメータを指定していない場合は、JOB文のMSGCLASSに指定された値が出力クラスとして適用されます。

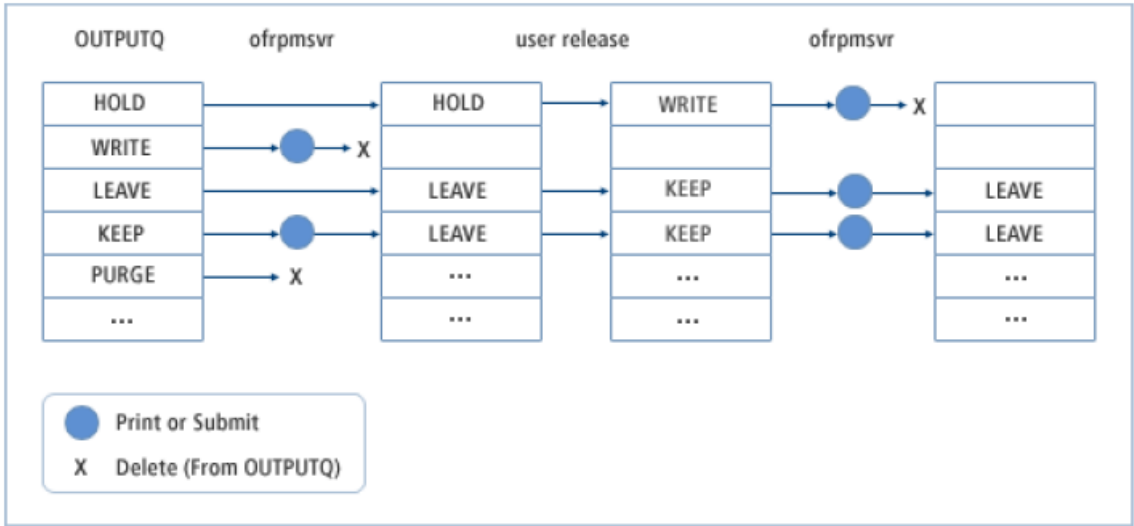
上記のいずれにもクラスを指定していない場合は、OpenFrame環境設定のtjclrunサブジェクト、JOBセクションのMSGCLASSキーのVALUE項目に設定された値が使用されます。

5.3.3. 出力の処理方式

OUTPUT DISPOSITIONは、ジョブの出力を処理する方式です。ジョブが正常に終了したかどうかによって、正常処理と異常処理に分けられます。

以下の図は、ジョブの出力処理方式を示しています。

[図 5.1] ジョブの出力処理方式



以下は、ジョブの出力処理方式についての説明です。

種類	説明
WRITE	出力がスケジューリングされ、印刷される必要があることを示します。正常に処理されたら、出力は削除されます。
HOLD	ユーザーが解除するまで出力はスケジューリングされずにOUTPUTQで待機します。ユーザーによって解除されると、処理方式はWRITEに変更されます。
KEEP	WRITEと同様に、出力がスケジューリングされ、印刷される必要があることを示します。正常に処理されたら、処理方式がLEAVEに変更されます。
LEAVE	ユーザーが解除するまで出力はスケジューリングされずにOUTPUTQで待機します。ユーザーによって解除されると、処理方式はKEEPに変更されます。
PURGE	出力をOUTPUTQから削除します。

以下は、ジョブの出力処理方式を設定する方法です。

1. JCLのOUTPUT文で指定します。

以下は、PRINT DDのOUTPUT DISPOSITIONに正常処理方式としてWRITE、異常処理方式としてPURGEを指定した例です。つまり、JCLが実行され、ジョブが正常に終了した場合は出力を処理し、ジョブが異常終了した場合は出力を処理しません。

```
//OUT1      OUTPUT  OUTDISP=(WRITE,PURGE)
//PRINT     DD  SYSOUT=(A, ), OUTPUT=*.OUT1
```

2. 以下の例で、SUBM DDはINTRDR属性を持ちます。

ジョブの終了後、SUBM DDに定義されたSYSOUTデータは、内部リーダーによってサブミットされます。

```
//SUBM      DD  SYSOUT=(A, INTRDR)
```

3. JCLのOUTPUT文で指定していない場合は、出力クラスに応じてOpenFrame環境設定の設定値が使用されます。

たとえば、OpenFrame環境設定の**tjes**サブジェクト、**OUTCLASS**セクションの出力クラスに応じて出力の処理方式は以下の例のように定義されます。出力クラスがAの場合は「WRITE,WRITE」に指定され、出力クラスがBの場合は「HOLD,PURGE」に指定されます。出力クラスに対応する処理方式が**OUTCLASS**セクションに定義されていない場合は、デフォルト値として「PURGE,PURGE:80」が使用されます。

```
$ ofconfig list -s tjes -sec OUTCLASS
```

SUBJECT	SECTION	KEY	VALUE
tjes	OUTCLASS	A	WRITE,WRITE
		B	HOLD,PURGE

OUTPUTQには、印刷待ちの出力と、内部リーダーを介して処理された後、再びジョブとしてサブミットされる必要がある出力があります。

内部リーダーを介して処理される出力は、出力処理方式がWRITEまたはKEEPです。また、OpenFrame環境設定の**print**サブジェクト、**INTRDR**セクションに登録されている場合は、出力クラスに関係なく、常に内部リーダーによってサブミットされます。

印刷待ちの出力は、出力処理方式がWRITEまたはKEEPの場合は、出力クラス情報を、OpenFrame環境設定の**print**サブジェクト、**PRINTERn**セクションに登録されたクラス情報と比較して、情報が正しければ印刷されます。PRINTERnセクションに登録されたクラス情報にSYSOUT文の出力クラスが含まれている場合、ofrpsvrはPRINTERnセクションに設定されたコマンドと共有ライブラリを使用して、データと印刷リソース情報を外部プリンター・ソリューションに送信します。

5.3.4. 内部リーダー

内部リーダーは、Online CICSまたはバッチ・プログラムがジョブをTJESにサブミットできるようにするモジュールです。OpenFrame上で動作するすべてのジョブは、内部リーダーを介してサブミットできます。

DD文にSYSOUT=(out_class,INTRDR)と記述して、DDに保存された内容が内部リーダーを介して出力されるようにします。

以下は、内部リーダーの使用方法です。

```
#####
## IEBEDT05 - iebedit SYSUT1 JCL and extract RMSTEP1 only.      ##
##           then submit it to intrdr                          ##
#####
//IEBEDT05 JOB CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1)
//IEBEDT EXEC PGM=IEBEDIT
//SYSIN DD *
        EDIT START=,TYPE=INCLUDE
/*
//SYSUT2 DD SYSOUT=(A,INTRDR)
//SYSUT1 DD DATA
内部リーダーを介してサブミットするJCLの内容
/*
//
```

上記のJCLは、SYSUT1 DDに与えられたJCLをSYSUT2にコピーし、SYSUT2をクラスAの内部リーダーを介して出力するジョブです。

上記のように、JCLにユーザーとパスワードが指定されていない場合、内部リーダーは、OpenFrame環境設定の**tjes**サブジェクト、**INTRDR**セクションの**USERNAME**、**PASSWORD**キーのVALUE項目に設定された値を使用してJCLをサブミットします。

5.3.5. 外部ライター

外部ライターは、ジョブの実行後に作成された出力をSMTPなどの他の共有ライブラリを介して処理できるようにするモジュールです。

外部ライターを使用するには、DD文にSYSOUT=(out_class,external_writer_name)の形式で記述し、OpenFrame環境設定の**print**サブジェクト、**WRITER**セクションにexternal_writer_nameに該当するキーとその値を指定する必要があります。現在は、SMTP(SMTP)のみサポートしています。

OpenFrameでSMTP(SMTP)を使用するには、2つのソフトウェアを準備する必要があります。

1. MTA (Mail transport Agent): メールを送受信するためのメール転送エージェントであり、デフォルトとしてSMTP(メール転送プロトコル)ポートである25番ポートを使用します。sendmail、postfixなどがあります。

2. Netcat: TCPやUDPプロトコルを使用するネットワーク接続でデータを読み書きするユーティリティです。OpenFrameでは、Netcatコマンドを使用してSMTPポートの25番ポートに接続し、メール・データを送信します。その後、MTA(postfixなど)を介してメールが転送されます。

以下は、SMTPP(SMTP)外部ライターの使用方法です。

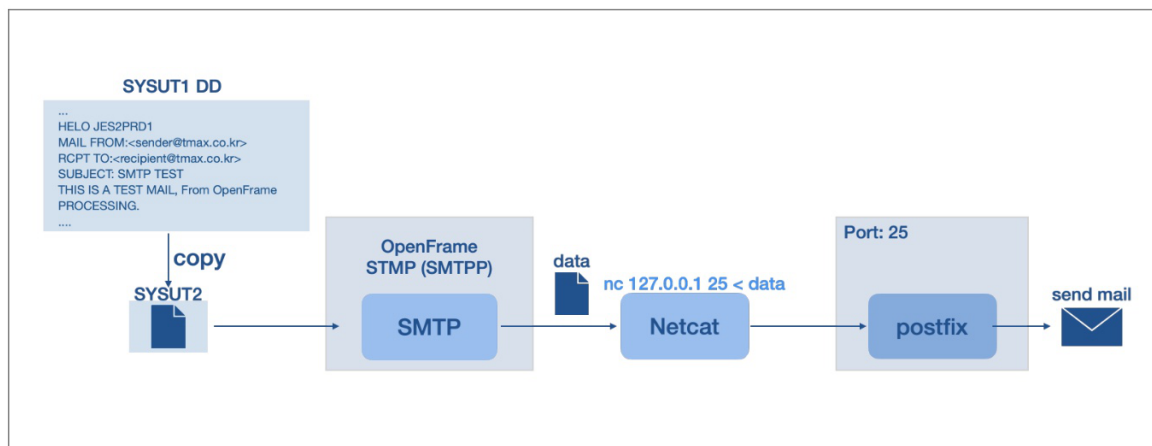
```
//EXTJOB    JOB (MDB114J), '      ',
//          CLASS=A,
//          MSGCLASS=K,
//          MSGLEVEL=(1,1),
//          NOTIFY=&SYSUID
// *
//JCLLIB     JCLLIB ORDER=(SCDC.PROD.JCLLIB)
// *
//JOB LIB    INCLUDE MEMBER=JOB LIB
//JSTEP01    EXEC PGM=IEBGENER
//SYSIN      DD DUMMY (MODE=I)
//SYSPRINT   DD SYSOUT=*
//SYSUT2     DD SYSOUT=(B,SMTPP)
//SYSUT1     DD *
HELO JES2PRD1
MAIL FROM:<sender@tmax.co.kr>
RCPT TO:<recipient@tmax.co.kr>
/*
//          DD *
DATA
SUBJECT: SMTP TEST

THIS IS A TEST MAIL, From OpenFrame
PROCESSING.
/*
// *
```

上記のJCLは、SYSUT1 DDで指定されたJCLをSYSUT2にコピーし、SYSUT2をSYSOUTに指定された外部ライターのSMTPP(SMTP)を介してSMTPサーバーにメールを送信するジョブです。

以下は、SMTP(SMTPP)モジュールの動作構造を示しています。

[図 5.2] SMTPの動作構造



1. JCLのメール関連のデータを加工してファイルに保存します。
2. Netcatユーティリティを使用してローカルホストの25番ポートに接続し、加工したファイルを転送します。SMTPポートである25番ポートは、postfixなどを介してメールを送信するか、メール・サーバー間でメールを送受信するときに使用されます。
3. postfixなどのメール送信エージェントがファイルに書き込まれたSMTPコマンドを実行し、メールを送信します。

第6章 TJESMGRコマンド

本章では、ジョブ実行コマンド、ジョブ出力コマンド、イニシエーター・コマンド、データセット・コマンド、スプール・バックアップ・コマンド、およびその他のTJESMGRコマンドについて説明します。

6.1. 概要

tjesmgrは、ジョブとTJESの管理機能以外に、データセット関連の操作機能とスプールのバックアップ機能などを提供します。**tjesmgr**はTACFまたはOSユーザーとしてログインする必要があります。サブミットするJCLのパラメータにユーザーが登録されていない場合は、ログイン・ユーザーがジョブをサブミットするユーザーになります。

tjesmgrの使用方法は、**tjesmgr**にアクセスした後、**tjesmgr**の実行画面でコマンドを実行するのが一般的ですが、UNIXシステムのコマンドラインで実行することもできます。実行方法は、**tjesmgr -h** コマンドで確認できます。

参考

TJESMGRクラスをTACFにリソースとして登録すると、**tjesmgr**のコマンドの実行権限を制御することができます。詳細については、『OpenFrame TACF 管理者ガイド』を参照してください。

- **tjesmgr**の実行方法

```
$ tjesmgr -i[ USERID[ [GROUPNAME] ]][ /PASSWORD] [command]
```

以下は、ログイン情報を取得する際の優先順位です。

1. ユーザーが[-i]オプションを使用して入力する場合
2. JCL内で呼び出されるプログラムがSACEE情報を利用する場合（JCL内のPGMパラメータによって該当のツールが呼び出されるか、シェル・スクリプトなどの間接的な方法で呼び出される場合）
3. tacflogin情報を利用する場合
4. OpenFrame環境設定から取得する場合
5. ユーザーによるキーボード入力の場合

- **tjesmgr**の構文のヘルプの表示

```
$ tjesmgr -h
```

- tjesmgrバージョン情報の表示

```
$ tjesmgr -v
```

以下は、UNIXコマンドラインでPS(Print Screen)コマンドを実行する例です。

```
$ tjesmgr -iUSERNAME/PASSWORD ps n=TEST
```

以下は、先行コマンドが必要で、コマンドを処理できない場合の例です。

```
$ tjesmgr -iUSERNAME/PASSWORD  
$ PODD JOB000001 di=1
```

PODDコマンドは、PSJOBまたはPOSPOOLコマンドが先に実行された場合に実行できます。そのため、上記の例はコマンドラインで実行できません。詳細については、[「6.3.2. PODD\(Print Output DD Information\)」](#) コマンドを参照してください。

コマンド一覧

以下はコマンド一覧です。

- ジョブ実行コマンド

コマンド	説明
CANCEL	指定されたジョブIDまたはジョブ名に該当するジョブをパーシします。
HOLD	指定されたジョブIDまたはジョブ名に該当するジョブの状態をSTARTからHOLDに変更します。
INQUEUECLASS	指定されたジョブIDまたはジョブ名に該当するジョブのクラスを変更します。
INQUEUEPRIORITY	指定されたジョブIDに該当するジョブの優先順位を変更します。
JEM	指定されたJCLFILEに対して、PGMオペランドに指定されたプログラムが存在するか、DD文に記述されたデータセットが存在するか、またそのデータセットがカタログに登録されているかを論理的にチェックします。処理が完了すると、DONE状態でJOBQに蓄積されます。
NICE	指定されたジョブIDまたはジョブ名に該当するジョブの中央処理装置の使用時間に対する優先順位を設定します。
OUTQUEUEPURGE	指定されたジョブIDまたは出力番号に該当するジョブの出力をOUTPUTQから削除します。

コマンド	説明
OUTQUEUERELASE	指定されたジョブIDまたは出力番号に該当するジョブの出力の処理をHOLDからWRITEに変更します。
PSTART	指定されたプロシージャを実行するステップを持つジョブを作成した後サブミットします。
REMOVE	指定されたジョブIDまたはジョブ名に該当するジョブを削除します。
RESUME	指定されたジョブIDまたはジョブ名に該当するジョブの状態をSUSPENDからRESUMEに変更します。
RUN	入力パラメータに従ってジョブをサブミットします。
SCAN	指定されたJCLファイルに対して構文をチェックします。構文エラーがなければ、DONE状態でJOBQに蓄積されます。
START	指定されたジョブIDまたはジョブ名に該当するジョブの状態をHOLDからSTARTに変更します。
STOP	指定されたジョブIDまたはジョブ名に該当するジョブの状態をWORKINGからSTOPに変更します。
SUSPEND	指定されたジョブIDまたはジョブ名に該当するジョブの状態をWORKINGからSUSPENDに変更します。

● ジョブ出力コマンド

コマンド	説明
PO	各ジョブIDの出力状態を表示します。
PODD	指定されたジョブIDのDD索引またはDD名に該当するスプールを、OpenFrame環境設定の tjesmgr サブジェクト、 DEFAULT_OPTION セクションの VIEWER キーに設定したビューアーを使用して開きます。
POJOB	指定されたジョブIDに該当するジョブの出力情報を表示します。
POLIST	出力一覧の詳細情報を表示します。
POSPool	指定されたジョブIDに該当するジョブのスプール情報を表示します。
PS	入力パラメータの条件を満たすJOBQに存在するジョブのジョブ名、ジョブID、クラス、ステータス、リターンコード、実行ノード、JCL一覧を表示します。
PSHISTORY(PSH)	指定条件に該当するジョブ履歴を表示します。

コマンド	説明
PSIO	指定されたジョブIDに該当するジョブのデータセット入出力回数を表示します。
PSJOB	指定されたジョブIDに該当するジョブの詳細情報を表示します。
PSSTEP	指定されたジョブIDに該当するジョブのステップ情報を表示します。

• イニシエーター・コマンド

コマンド	説明
BOOT	ノード全体のTJESを起動します。
CHANGECLASS(CC)	該当するクラスの状態を変更します。
JCLRUNACTIVE(RA)/ JCLRUNINACTIVE(RI)	コマンドラインで指定されたランナー・スロット状態をActiveからInactiveに、またはInactiveからActiveに変更します。
JCLRUNCLASS(RC)	該当するランナー・スロットのクラスを指定されたクラスに変更します。
NODECLEAR	該当するノードのすべてのジョブを停止させます。
NODESTATUS(NODEST)	ノード全体の状態を表示します。
PSCLASS(PSC)	クラスごとにジョブの状態のカウントを表示します。
PSJCLRUN(PSR)	該当するノードのランナー・スロットの状態を表示します。
SHUTDOWN	ノード全体のTJESを終了します。
SHOWCLASS(SC)	クラスの状態を表示します。

• データセット・コマンド

コマンド	説明
DATASETCOPY(DSCOPY)	ソース・データセットをターゲット・データセット、または指定されたボリュームにコピーします。
DATASETCREATE(DSCREATE)	このコマンドを実行すると、ユーザー入力画面が表示されます。ユーザーから入力された条件に該当する非VSAMデータセットを作成します。
DATASETDELETE(DSDELETE)	指定された非VSAMデータセットを削除します。
DATASETEXPORT(DSEXPORT)	指定された非VSAMデータセットを外部にエクスポートします。

コマンド	説明
DATASETIMPORT(DSIMPORT)	外部にエクスポートしたファイルを読み込んで、データセットに復元します。
DATASETLIST(DSLIST)	指定条件に該当するVTOCに登録されているデータセットの一覧を表示します。
DATASETMOVE(DSMOVE)	データセットの名前を変更するか、ボリュームを変更するか、あるいはこの両方の操作を行います。
DATASETRECATALOG(DSRECAT)	このコマンドを実行すると、ユーザー入力画面が表示されます。指定条件に該当する非VSAMデータセットを再カタログします。
DATASETUNCATALOG(DSUNCAT)	指定されたデータセットをカタログから削除します。
DATATOUCH(DSTOUCH)	指定された非VSAMデータセットの最終変更日を変更します。
GDGCREATE	このコマンドを実行すると、入力画面が表示されます。指定条件に該当するGDGデータセットを作成します。
GDGDELETE	指定されたGDGデータセットを削除します。
PSCATALOG(PSCAT)	指定条件に該当するカタログに登録されているデータセット・リストを表示します。
PSDATASET(PSDS)	指定されたデータセットの詳細情報を表示します。
PSENTRY(PSENT)	指定されたエントリの詳細情報を表示します。
PSLOCK(PSL)	指定されたデータセットのロック情報を表示します。
PSVOLUME(PSV)	ボリュームに関する情報を表示します。
SETCATALOG(SETCAT)	指定されたカタログをデフォルト・カタログにしてデータセット関連コマンドで使います。

● スプール・バックアップ・コマンド

コマンド	説明
SPOOLBACKUP(SPBA)	指定されたスプールをバックアップします。
SPOOLBACKUPLIST(SPBL)	指定された年、月にバックアップされたスプールの日付を表示します。
SPOOLBACKUPSTATUS(SPBASTAT)	バックアップ操作の進捗状況と完了した操作の処理結果を表示します。
SPOOLBACKUPSTOP(SPBASTOP)	実行中のバックアップ処理を中止します。
SPOOLCLEAR(SPCL)	指定された一時ディレクトリに復元されたスプールを削除します。

コマンド	説明
SPOOLPODD(SPPODD)	復元されたジョブのスプールを設定したビューアーで開きます。
SPOOLPOSPPOOL(SPPOSPPOOL)	復元されたジョブのスプール情報を表示します。
SPOOLPS(SPPS)	指定された日付にバックアップされたスプール・リストを表示します。
SPOOLPSJOB(SPPSJ)	復元されたジョブの詳細情報を表示します。
SPOOLPSIO(SPPSIO)	復元されたジョブの入出力情報を表示します。
SPOOLRESTORE(SPRES)	バックアップされたスプールの内容を一時ディレクトリに復元します。

● その他のコマンド

コマンド	説明
CLS	tjesmgr画面を削除します。
CURRENTUSER(CU)	現在ログインしているユーザーのIDを表示します。
EDIT	指定されたファイル・パスを指定のエディターで開きます。
ERROR	指定されたエラー番号に該当するエラー・メッセージを表示します。
HELP	指定されたコマンドの使い方を表示します。コマンドを指定していない場合、すべてのコマンドの使い方を表示します。
LICENSE	指定されたライセンス・ファイルの情報を表示します。
NOTIFY(N)	現在ログインしているユーザーIDにジョブの実行情報を通知します。
PSPRINT(PSP)	OpenFrame環境設定に登録されているプリンター情報を表示します。
SWITCHUSER	ユーザー入力画面が表示され、入力されたユーザーIDでログインします。
TSOLIST(TSL)	現在TSOに接続しているユーザー・リストを表示します。
VERSION	TJESのバージョン情報を表示します。

参考

各コマンドの環境設定例のサブジェクト、セクション、キーの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

6.2. ジョブ実行コマンド

ジョブ実行コマンドは、ジョブをサブミットし、ジョブの状態を変更できるコマンドです。本節では、ジョブ実行コマンドの構文について説明します。

6.2.1. CANCEL

指定されたジョブIDまたはジョブ名に該当するジョブを除去(パージ)します。HOLDまたはSTART状態のジョブに対してのみ有効です。

- 構文

```
CANCEL {<job-ID>|N=<job-name>}
```

6.2.2. HOLD

指定されたジョブIDまたはジョブ名に該当するジョブの状態をSTARTからHOLDに変更します。HOLD状態のジョブはスケジューリングされません。HOLDコマンドは、START状態のジョブに対してのみ有効です。

- 構文

```
HOLD {<job-ID>|N=<job-name>}
```

6.2.3. INQUEUECLASS

指定されたジョブIDまたはジョブ名に該当するジョブのクラスを変更します。HOLDまたはSTART状態のジョブに対してのみ有効です。該当するクラスのランナーがすべて割り当てられていて、すぐにジョブを実行できない場合に主に使用されます。

- 構文

```
INQUEUECLASS(IC) {job-ID|N=<job-name>} C=<class>
```

6.2.4. INQUEUEPRIORITY(IP)

指定されたジョブIDに該当するジョブの優先順位を変更します。priority-numberには、0~15の数字を指定できます。数字が高いほど先に実行されます。

STARTまたはHOLD状態のジョブに対してのみ使用できます。

- 構文

```
INQUEUEPRIORITY(IP) <job-ID> <priority-number>
```

6.2.5. JEM

指定されたJCLFILEに対して、PGMオペランドに指定されたプログラムが存在するか、DD文に記述されたデータセットが存在するか、またそのデータセットがカタログに登録されているかを論理的にチェックします。処理が完了すると、DONE状態でJOBQに蓄積されます。

- 構文

```
JEM <JCLFILE> [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<JCLFILE>	JCL名またはUNIXファイル・パスを指定するか、DATASET(member)形式で指定することができます。 JCL名を指定すると、OpenFrame環境設定のtjesサブジェクト、 PROCLIB セクションの JCLLIB 、 USERLIB キーのVALUE項目に設定されたデータセットからJCLを検索します。ただし、データセット名がJOBnnnnnのようにJOBIDの形式と一致するJCLは使用できません。
[CAT=<catalog-name>]	JCLFILEをDATASET(member)形式で指定する際、データセットがユーザー・カタログに登録されている場合はこの値を指定します。

6.2.6. NICE

指定されたジョブIDまたはジョブ名に該当するジョブに対して、コンピューターの使用時間における相対的な優先順位を設定します。WORKING状態のジョブに対してのみ有効です。

- 構文

```
NICE {<job-ID>|N=<job-name>} <out-number>
```

以下は、入力項目に関する説明です。

項目	説明
<job-ID>	ジョブIDを指定します。
<job-name>	ジョブ名を指定します。
<out-number>	0～20の数字を指定します。数字が小さいほど優先順位が高くなります。この優先順位は、コンピューターの使用時間における優先順位であり、入出力の優先順位ではありません。

6.2.7. OUTQUEUEPURGE

指定されたジョブIDまたは出力番号に該当するジョブの出力をOUTPUTQから削除します。

- 構文

```
OUTQUEUEPURGE(OR) {<job-ID>|ON=<out-number>}
```

以下は、入力項目に関する説明です。

項目	説明
<job-ID>	ジョブIDを指定します。
<out-number>	POLIST または POJOB コマンドによって出力されるOUTPUTIDの数字を指定します。出力についての詳細は、 「5.2. 出力」 を参照してください。

6.2.8. OUTQUEUERELEASE

指定されたジョブIDまたは出力番号に該当するジョブの出力の処理をHOLDからWRITEに変更します。

- 構文

```
OUTQUEUERELEASE(OL) {<job-ID>|ON=<out-number>}
```

以下は、入力項目に関する説明です。

項目	説明
<job-ID>	ジョブIDを指定します。
<out-number>	POLIST または POJOB コマンドによって出力されるOUTPUTIDの数字を指定します。出力についての詳細は、 「5.2. 出力」 を参照してください。

6.2.9. PSTART

プロシージャをサブミットします。PSTARTを使用してプロシージャをサブミットすると、そのプロシージャを呼び出すステップを持つジョブがサブミットされ、RUNコマンドと同様にJOBQにサブミットされたジョブが登録されます。このとき、プロシージャはカタログ・プロシージャである必要があります。

- 構文

```
PSTART(PRS) <procedure-name> [C=<class>] [NODE=<node-name>]  
[<procedure-parameters>]
```

以下は、入力項目に関する説明です。

項目	説明
<procedure-name>	実行するプロシージャ名を指定します。 プロシージャ名を指定すると、そのプロシージャを呼び出すステップを持つJCLが生成され、そのJCLはtjesmgrを介してサブミットされます。OpenFrame環境設定の tjes サブジェクト、 PROCLIB セクションの PROC00 キーのVALUE項目に設定されたデータセットからプロシージャを検索します。
[C=<class>]	プロシージャを実行するJCLのジョブ・クラスを指定します。
[NODE=<node-name>]	– ノード名を指定した場合：指定したノードでジョブが実行されるようにサブミットします。 – ノード名を指定していない場合：OpenFrame環境設定の tjesmgr サブジェクト、 DEFAULT_OPTION セクションの DEFAULT_RUNNING_NODE キーの設定値に従ってジョブが実行されるようにサブミットします。
[<procedure-parameter>]	プロシージャで使用する記号パラメータを指定します。 JCL構文と同様に<key>=<value>の形式で指定し、コンマで区切って複数のパラメータを指定することができます。

6.2.10. REMOVE

指定されたジョブIDまたはジョブ名に該当するジョブを削除します。DONE、ERROR、STOP、FLUSH状態のジョブに対してのみ有効です。

- 構文

```
REMOVE  {<job-ID>|N=<job-name>}
```

6.2.11. RESUME

指定されたジョブIDまたはジョブ名に該当するジョブの状態をSUSPENDからRESUMEに変更します。つまり、一時停止されたジョブを再開します。SUSPEND状態のジョブに対してのみ有効です。

- 構文

```
RESUME  {<job-ID>|N=<job-name>}
```

6.2.12. RUN

入力パラメータに従ってジョブをサブミットします。サブミットしたジョブは、START状態でJOBQに蓄積されます。PSコマンドを使用してJOBQに蓄積されたジョブを確認することができます。

- 構文

```
RUN(R) <JCLFILE> [NODE=<node-name>] [STEP=<restart-step>] [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<JCLFILE>	JCL名またはUNIXファイル・パスを指定するか、DATASET(member)形式で指定することができます。 JCL名を指定すると、OpenFrame環境設定の tjes サブジェクト、 PROCLIB セクションの JCLLIB 、 USERLIB キーのVALUE項目に設定されたデータセットからJCLを検索します。ただし、データセット名がJOBnnnnnのようにJOBIDの形式と一致するJCLは、tjesmgrを使用してサブミットできません。
[NODE=<node-name>]	– ノード名を指定した場合：指定したノードでジョブが実行されるようにサブミットします。 – ノード名を指定していない場合：OpenFrame環境設定の tjesmgr サブジェクト、 DEFAULT_OPTION セクションの DEFAULT_RUNNING_NODE キーのVALUE項目に設定された値に従ってジョブを現在のノードでのみ実行できるようにするか、またはすべてのノードで実行できるようにするかが決まります。
[STEP=<restart-step>]	指定されたステップからジョブを実行します。IF文の分岐先内のステップは、restart-stepで指定できません。
[CAT=<catalog-name>]	JCLFILEをDATASET(member)形式で指定する際、データセットがユーザー・カタログに登録されている場合はこの値を指定します。

6.2.13. SCAN

指定されたJCLFILEの構文をチェックします。構文エラーがない場合は、DONE状態でJOBQに蓄積されます。

- 構文

```
SCAN(SCN) <JCLFILE> [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<JCLFILE>	JCL名またはUNIXファイル・パスを指定するか、DATASET(member)形式で指定することができます。 JCL名を指定すると、OpenFrame環境設定の tjes サブジェクト、 PROCLIB セクションの JCLLIB 、 USERLIB キーのVALUE項目に設定されたデータセットからJCLを検索します。ただし、データセット名がJOBnnnnnのようにJOBIDの形式と一致するJCLは使用できません。
[CAT=<catalog-name>]	JCLFILEをDATASET(member)形式で指定する際、データセットがユーザー・カタログに登録されている場合はこの値を指定します。

6.2.14. START

指定されたジョブIDまたはジョブ名に該当するジョブの状態をHOLDからSTARTに変更します。STARTコマンドは、HOLD状態のジョブに対してのみ有効です。

- 構文

```
START {<job-ID>|N=<job-name>}
```

6.2.15. STOP

指定されたジョブIDまたはジョブ名に該当するジョブの状態をWORKINGからSTOPに変更します。つまり、実行中のジョブを中止します。WORKINGまたはSUSPEND状態のジョブに対してのみ有効です。

- 構文

```
STOP {<job-ID>|N=<job-name>}
```

6.2.16. SUSPEND

指定されたジョブIDまたはジョブ名に該当するジョブの状態をWORKINGからSUSPENDに変更します。つまり、実行中のジョブを一時停止します。WORKING状態のジョブに対してのみ有効です。

- 構文

```
SUSPEND {<job-ID>|N=<job-name>}
```


6.3. ジョブ出力コマンド

ジョブ出力コマンドは、ジョブをサブミットした後、ジョブの進捗状況と実行結果、および出力関連情報を確認するためのコマンドです。本節では、ジョブ出力コマンドの構文について説明します。

6.3.1. PO(Print Output status)

各ジョブの出力状態を表示します。

- 構文

```
PO
```

- 表示情報

以下の情報が表示されます。

情報	説明
JOBID	ジョブIDです。
OUTPUT STATUS	出力状態です。

6.3.2. PODD(Print Output DD Information)

指定されたジョブIDのDD索引またはDD名に該当するスプールを、OpenFrame環境設定のtjesmgrサブジェクト、**DEFAULT_OPTION**セクションの**VIEWER**キーのVALUE項目に設定したビューアーを使用して開きます。PODDコマンドを使用するには、tjesmgr内のシェルで実行されるPSJOBまたはPOSPOOLコマンドを先に実行する必要があります。

- 使用方法

```
PODD <job-ID> { DI=<dd-index> | DN=<dd-name> }
```

関連環境設定

スプールを開くビューアーを確認します。

```
$ ofconfig list -n NODE1 -s tjesmgr -sec DEFAULT_OPTION -k VIEWER -l
```

```
=====
```

```
SUBJECT      : tjesmgr
SECTION      : DEFAULT_OPTION
KEY          : VIEWER
TYPE         : 2
DEFAULT_VALUE : vi &FILEPATH
VALUE(M)*    : vim -w&ROWCOUNT -R &FILEPATH
```

```

VALUE(D)      : vim -w&ROWCOUNT -R &FILEPATH
AVAIL_VALUE   :
DESCRIPTION   : Viewing program name for tjesmgr command PODB.
                  &ROWCOUNT will be replaced window size.
                  &FILEPATH will be replaced viewing filepath.
                  -w&ROWCOUNT option is valid only for `old vi' not `vim'.
=====

```

6.3.3. POJOB(Print Output of JOB)

指定されたジョブIDに該当するジョブの出力情報を表示します。

- 構文

```
POJOB(POJ) <job-ID>
```

以下は、POJOBの入力項目に関する説明です。

項目	説明
<job-ID>	ジョブIDを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
OUTPUTID	出力IDです。
DDID	DDに付与されたIDです。
DDNAME	JCLに記述されたDD名です。
CLS	出力クラスです。
ST	出力ステータスです。
FORMS	印刷関連のオプション情報です。
CHARS	印刷関連のオプション情報です。
FORMDEF	印刷関連のオプション情報です。
PAGEDEF	印刷関連のオプション情報です。
WRITER	印刷関連のオプション情報です。

参考

POJOBの表示情報は、POLISTと同じです。

6.3.4. POLIST(Print Output detail LIST)

出力リストの詳細情報を表示します。

- 構文

```
POLIST [<job-id>] [N=<job-name>] [S=<output-status>] [C=<outclass>]
```

- 表示情報

以下の情報が表示されます。

情報	説明
OUTPUTID	出力IDです。
JOBID	ジョブIDです。
JOBNAME	ジョブ名です。
DDID	DDに付与されたIDです。
DDNAME	JCLに記述されたDD名です。
CLS (OUTPUT CLASS)	出力クラスです。
ST (OUTPUT STATUS)	出力ステータスです。
FORMS	印刷関連のオプション情報です。
CHARS	印刷関連のオプション情報です。
FORMDEF	印刷関連のオプション情報です。
PAGEDEF	印刷関連のオプション情報です。
WRITER	印刷関連のオプション情報です。

参考

DDID、FORMS、CHARS、FORMDEF、PAGEDEF、WRITERは、印刷関連の情報です。詳細については、「[第5章 出力処理](#)」と「[6.6. スプール・バックアップ・コマンド](#)」を参照してください。

6.3.5. POSPOOL(Print Output SPOOL of JOB)

指定されたジョブIDに該当するジョブのスプール情報を表示します。

- 構文

```
POSP00L <job-ID>
```

以下は、入力項目についての説明です。

項目	説明
<job-ID>	ジョブIDを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
NO	DDINDEXです。PODDコマンドの実行時に入力パラメータとして使用されます。
STEP	スプールが生成されたステップ名です。
DDNAME	スプールのDD名です。
SIZE	スプールのサイズです。
DSNAME	スプールのデータセット名です。

関連環境設定

SPOOL_LISTは、スプールの表示条件を設定します。

```
$ ofconfig list -n NODE1 -s tjesmgr -sec DEFAULT_OPTION -k SPOOL_LIST -l
=====
SUBJECT       : tjesmgr
SECTION       : DEFAULT_OPTION
KEY           : SPOOL_LIST
TYPE          : 2
DEFAULT_VALUE : ALL
VALUE(M)*     : ALL
VALUE(D)      : ALL
AVAIL_VALUE   : ALL,EXIST,GTZERO
DESCRIPTION   : tjesmgr command(POSP00L) display option.
                  ALL : display all spool list.
                  EXIST : display existing spool list.
                  GTZERO : display spool list whose size are greater than
zero.
=====
```

6.3.6. PS(Print Screen)

入力パラメータの条件を満たすJOBQに存在するジョブのJOBNAME、JOBID、CLASS、STATUS、RC、NODE、JCLなどを表示します。

- 構文

```
PS [{ N=<job-name> | [C=<class>] [S=<job-status>] [RC=<return-code>]
    [NODE=<node-name>] |
    [SUBM=<submit_date>|<submit_date(from)-submit_date(to)>] |
    <from-job-ID> [C=<class>] [S=<job-status>] [RC=<return-code>]
    [NODE=<node-name>] | <job-status> |
    DESC N=<job-name>|RC=<return-code> }]
```

- 表示情報

以下の情報が表示されます。

情報	説明
JOBNAME	ジョブ名です。
JOBID	ジョブIDです。
CLASS	ジョブ・クラスです。
STATUS	ジョブの現在の状態です。以下のように1バイトの文字で表示されます。 – W : Working – D : Done – E : Error – F : Flush – S : Start – H : Hold – P : Suspend – T : Stop
RC	ジョブのリターンコードです。
NODE	ジョブが実行されたノード名です。
SUBMIT-TIME	ジョブがサブミットされた時間です。SUBMオプションを使用する場合にのみ表示されます。
START-TIME	ジョブの開始時刻です。
END-TIME	ジョブの終了時刻です。
JCL	JOB文が記述されているJCL名です。

- 例

CLASS、STATUS、RETCODE、NODENAMEの順に条件を与えます。

– 正しい例)

```
PS c=A s=W rc=0
```

– 正しくない例)

```
PS s=W c=A → 解析エラーが発生
```

DESCコマンドは、ジョブ・リストを逆順に表示します。

– 使用例)

```
PS DESC
```

6.3.7. PSHISTORY(PSH)

ジョブ履歴を表示します。

このコマンドを実行すると、条件検索ウィンドウが表示されます。指定した条件に該当するジョブ履歴ログが表示されます。

- 構文

```
PSHISTORY(PSH)
```

- 入力パラメータ

以下は、入力パラメータについての説明です。

パラメータ	説明
USERID	検索するユーザーIDを指定します。
DATE	検索する日付を指定します。
OPERATION	検索する操作を指定します。
JOBID	検索するジョブIDを指定します。

6.3.8. PSIO(Print Screen Input/Output information)

指定されたジョブIDに該当するジョブのデータセット入出力回数を表示します。

- 構文

```
PSIO <job-ID>
```

以下は、入力項目に関する説明です。

項目	説明
<job-ID>	ジョブIDを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
[STEP]	ステップ名です。
DDNAME	JCLに記述されているDD名です。
DSNAME	データセット名です。
RTV (Retrieved count)	ステップごとにデータセットが検索された回数です。
INS (Inserted count)	ステップごとにデータセットが追加された回数です。
UPD (Updated count)	ステップごとにデータセットが更新された回数です。
DEL (Deleted count)	ステップごとにデータセットが削除された回数です。

6.3.9. PSJOB(Print Screen JOB detail information)

指定されたジョブIDに該当するジョブの詳細情報を表示します。ジョブの状態がWORKINGの場合は、詳細情報を1秒単位で更新して表示します。

- 構文

```
PSJOB(PSJ) <job-ID>
```

- 表示情報

以下は、PSJOBの表示情報です。

情報	説明
JOBID	ジョブIDです。
NODE NAME	ジョブが実行されたノード名です。
JOB NAME	ジョブ名です。
JOB CLASS	ジョブ・クラスです。
JOB STATUS	ジョブの現在の状態です。 DONE、ERROR、FLUSHの場合は、括弧内にジョブの戻り値が入ります。
JOB PRY	ジョブの優先順位です。
JCLRUNNER INDEX	ジョブが実行されたランナー・スロット・インデックスです。ランナー・スロットに関する情報は、PSJCLRUNから確認できます。
JOB WAIT	WORKING状態のジョブが待機している理由を表示します。

情報	説明
	– ALREADY LOCK : JCLによって割り当てられるデータセットがロックされています。 – JOBNAME DUP : 同じ名前のジョブが現在実行中です。関連設定は、OpenFrame環境設定のtjesサブジェクト、SCHEDULINGセクションのDUPL_JOBNAMEキーのVALUE項目に設定します。 – JCLHOLD : JCL内で該当のジョブにJCLHOLDが設定されています。
JOB USER	ジョブをサブミットしたユーザーです。
JCL PATH	ジョブのJCLパスです。
TIME STAMP	ジョブのサブミット、開始、終了時刻です。
RES USAGE	ジョブのリソース使用量です。 – PROCESS : プロセスの総使用時間です。 – CPU : WORKINGプロセスのCPU使用時間の合計です。ジョブがWORKING状態の場合は、tjclrunと現在のWORKINGプロセスのCPU使用時間の合計を表示し、ジョブがWORKING状態でない場合は、ジョブのCPU使用時間の合計を表示します。 – CPU ?% : WORKINGプロセスのCPU占有率です。 – MEM ?Kb : WORKINGプロセスのメモリ・サイズです。ジョブがWORKING状態である場合にのみ表示されます。
STEP LIST	ジョブのステップごとの情報を表示します。 – [???]: ステップ名です。 – START : ステップの開始時刻です。 – RC : ステップのリターン・コードです。 – CPU : ステップのCPU使用時間です。 – / ??s : ステップの総実行時間です。
SPOOL LIST	ジョブのスプール・リストです。 POSPOOLコマンドと同じです。
OUTPUT PROCESS-ING STATUS	ジョブの出力処理に関する情報を表示します。

以下は、WORKING状態のジョブの表示情報です。

情報	説明
[STEPNAME] CURRENT PROCESSING	現在処理中のステップです。
DDNAME JOBID(DSNAME) R:? W:?	– R : データセットの読み込み試行回数です。 – W : データセットの書き込み試行回数です。

6.3.10. PSSTEP(Print Screen STEP)

指定されたジョブIDに該当するジョブのステップ情報を表示します。ジョブがWORKING状態である場合は、現在処理中のステップ情報を1秒間隔で更新して表示します。

- 構文

```
PSSTEP <job-ID>
```

- 表示情報

以下の情報が表示されます。

情報	説明
STEP LIST	ジョブの各ステップ情報を表示します。 – [???] : ステップ名です。 – START : ステップの開始時刻です。 – RC : ステップのリターンコードです。 – CPU : ステップのCPU使用時間です。 – / ??s : ステップの総実行時間です。

6.4. イニシエーター・コマンド

本節では、TJES環境でジョブを実行するためのイニシエーター・コマンドの構文について説明します。

6.4.1. BOOT

すべてのノードのTJESをブートします。ブートされたノードではジョブを実行することができます。つまり、ランナー・スロットにジョブを割り当てることができます。

- 構文

```
BOOT [NODE=<node-name>]
```

以下は、入力項目に関する説明です。

項目	説明
[NODE=<node-name>]	ノード名を指定した場合は、指定したノードのみブートします。

6.4.2. CHANGECLASS(CC)

クラスの状態を変更します。

- 構文

```
CHANGECLASS(CC) C=<class> HOLD = {Y|N}
```

関連環境設定

クラス状態の初期値を指定します。OpenFrame環境設定に指定されていない場合のデフォルト値はSTARTであり、CHANGECLASSコマンドを出力する際、Nに該当する値です。

```
$ ofconfig list -n NODE1 -s tjcs -sec JOBCLASS -l
=====

SUBJECT      :  tjcs
SECTION      :  JOBCLASS
KEY          :  A
TYPE         :  2
DEFAULT_VALUE :  START
VALUE(M)*    :  START
VALUE(D)     :  START
AVAIL_VALUE  :  START,HOLD
DESCRIPTION  :  Specifies the properties of JOB CLASS.
-----

SUBJECT      :  tjcs
SECTION      :  JOBCLASS
KEY          :  B
TYPE         :  2
DEFAULT_VALUE :  START
VALUE(M)*    :  START
VALUE(D)     :  START
AVAIL_VALUE  :  START,HOLD
DESCRIPTION  :  Specifies the properties of JOB CLASS.
=====
```

6.4.3. JCLRUNACTIVE(RA)/JCLRUNINACTIVE(RI)

ランナー・スロットの状態をACTIVEからINACTIVEに、またはINACTIVEからACTIVEに変更します。変更情報は、PSJCLRUNコマンドを使用して確認できます。

- 構文

```
JCLRUNACTIVE(RA)/JCLRUNINACTIVE(RI) I=<index>,,, [NODE=<node-name>]
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	複数のインデックスを指定するか、あるいは範囲を指定することができます。 – 形式 <pre>I=<index>,,,</pre>
[NODE=<node-name>]	ノード名を指定していない場合は、自ノードのランナー・スロット状態を変更します。

- 例

以下の例は、JCLRUNACTIVEとJCLRUNINACTIVEコマンドを使用する際、範囲を指定する方法を示しています。

```
$ RA I=0,1,2,3
$ RI I=0-9
```

関連環境設定

現在のノードを検索する際に使用します。

```
$ ofconfig list -n NODE1 -s tjes -sec NODEINFO -k NODENAME -l
=====
SUBJECT      : tjes
SECTION      : NODEINFO
KEY          : NODENAME
TYPE         : 2
DEFAULT_VALUE : NODE1
VALUE(M)*    : NODE1
VALUE(D)     : NODE1
AVAIL_VALUE  :
DESCRIPTION  : TMAX node name. (recommend to use HOSTNAME)
=====
```

6.4.4. JCLRUNCLASS(RC)

ランナー・スロットのクラスを指定されたクラスに変更します。

- 構文

```
JCLRUNCLASS(RC) I=index,,, CS=<classes> [NODE=<node-name>]
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	複数のインデックスを指定するか、あるいは範囲を指定することができます。 – 形式 <pre>I=<index>,,,</pre>
[NODE=<node-name>]	ノード名を指定していない場合は、自ノードのクラス設定を変更します。

- 例

以下は、JCLRUNCLASSコマンドを使用して範囲とクラスを指定する方法です。

```
$ JCLRUNCLASS I=0,1,2,3 CS=A
$ RC I=0-9 CS=AB
```

関連環境設定

現在のノードを検索する際に使用します。

```
$ ofconfig list -n NODE1 -s tjcs -sec NODEINFO -k NODENAME -l
=====
SUBJECT      :  tjcs
SECTION      :  NODEINFO
KEY          :  NODENAME
TYPE         :  2
DEFAULT_VALUE :  NODE1
VALUE(M)*    :  NODE1
VALUE(D)     :  NODE1
AVAIL_VALUE  :
DESCRIPTION  :  TMAX node name. (recommend to use HOSTNAME)
=====
```

6.4.5. NODECLEAR

ノードのすべてのジョブを停止させます。ノードに問題が発生し、ノード内のジョブを制御できなくなった場合に使用します。

- 構文

```
NODECLEAR NODE=<node-name>
```

6.4.6. NODESTATUS(NODEST)

すべてのノードの状態を表示します。

- 構文

```
NODESTATUS(NODEST)
```

- 表示情報

以下の情報が表示されます。

情報	説明
NODENAME	ノード名です。
OBMJINIT_STATUS	obmjinitサーバーの状態です。 – Active – Downed
BOOT_STATUS	TJESのブート状態です。 – Booted – Not booted

6.4.7. PSCLASS(PSC)

クラスごとにジョブの状態(NODE、CLASS、HOLD、START、WORKING、SUSPEND、DONE、STOP、ERROR、FLUSH)のカウンタを表示します。

- 構文

```
PSCLASS(PSC)
```

6.4.8. PSJCLRUN (PSR)

ノードのランナー・スロットの状態を表示します。

- 構文

```
PSJCLRUN(PSR) [NODE=<node-name>]
```

以下は、入力項目に関する説明です。

項目	説明
[NODE=<node-name>]	ノード名を指定していない場合は、自ノードのランナー・スロットの状態を表示します。

- 表示情報

以下の情報が表示されます。

情報	説明
IDX	ランナー・スロット・インデックスです。以下のコマンドで使用されます。 – JCLRUNACTIVE(RA) – JCLRUNINACTIVE(RI) – JCLRUNclass(RC)
SLOTNAME	OpenFrame環境設定の tjes サブジェクト、 INITDEF セクションの INIT{nnnn} キーに定義されているランナー・スロット名です。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
STATUS	ランナー・スロットの状態です。 – ACTIVE – INACTIVE – DOWNED
class	ランナー・スロットに割り当てられたクラスです。
JOBID, JOBNAME, JOBclass, PID	ランナー・スロットに割り当てられたジョブの情報です。 ランナー・スロットのクラスまたは状態をRA、RI、RCまたはSHUTDOWNコマンドを使用して変更する場合、ランナー・スロットにジョブが割り当てられていると、ジョブの終了後に変更されるというメッセージが表示されます。 STATUS will be [DOWNED] after JOB finished

関連環境設定

現在のノードを検索する際に使用します。

```
$ ofconfig list -n NODE1 -s tjes -sec NODEINFO -k NODENAME -l
```

```
=====
SUBJECT      : tjes
SECTION      : NODEINFO
KEY          : NODENAME
TYPE         : 2
DEFAULT_VALUE : NODE1
VALUE(M)*    : NODE1
VALUE(D)     : NODE1
AVAIL_VALUE  :
DESCRIPTION  : TMAX node name. (recommend to use HOSTNAME)
=====
```

ランナー・スロットのデフォルト設定を行います。現在のノードを検索する際に使用します。

```
$ ofconfig list -n NODE1 -s tjes -sec INITDEF -l
```

```
=====
SUBJECT      : tjes
SECTION      : INITDEF
KEY          : INIT0000
TYPE         : 2
DEFAULT_VALUE : INITA,A,ACTIVE
VALUE(M)*    : INITA,A,ACTIVE
VALUE(D)     : INITA,A,ACTIVE
AVAIL_VALUE  :
DESCRIPTION  : INITXXX= name, multi class, ACTIVE/INACTIVE
-----
```

```

SUBJECT      : tjes
SECTION      : INITDEF
KEY          : INIT0001
TYPE         : 2
DEFAULT_VALUE : INITB,B,ACTIVE
VALUE(M)*    : INITB,B,ACTIVE
VALUE(D)     : INITB,B,ACTIVE
AVAIL_VALUE  :
DESCRIPTION  : INITXXX= name, multi class, ACTIVE/INACTIVE
-----
```

..

..

中略

..

..

6.4.9. SHUTDOWN

すべてのノードのTJESをシャットダウンします。これにより、空のランナー・スロットがDOWNED状態に変更され、追加的なジョブ・スケジューリングが制限されます。また、WORKING状態のランナー・スロットは、ジョブの終了と同時にDOWNED状態に変更されます。

- 構文

```
SHUTDOWN [NODE=<node-name>]
```

以下は、入力項目に関する説明です。

項目	説明
[NODE=<node-name>]	ノード名を指定した場合は、指定したノードのみシャットダウンされます。

参考

コマンドについての詳細は、[「6.4.8. PSJCLRUN \(PSR\)」](#)を参照してください。

6.4.10. SHOWCLASS(SC)

クラスの状態を表示します。クラスがHOLD状態の場合、そのクラスにサブミットされたジョブは常にHOLD状態になります。

- 構文

```
SHOWCLASS(SC)
```

- 表示情報

以下の情報が表示されます。

情報	説明
CLASS	クラス名です。
HOLD	クラスがHOLD状態であるかどうかを示します。(Y、N)

関連環境設定

クラス状態の初期値を設定します。OpenFrame環境設定に指定されていない場合のデフォルト値はSTARTであり、SHOWCLASSコマンドの出力時のNに該当する値です。

```
$ ofconfig list -n NODE1 -s tjес -sec JOBCCLASS -l
```

```
=====
```


SUBJECT	:	tjes
SECTION	:	JOBCLASS
KEY	:	A
TYPE	:	2
DEFAULT_VALUE	:	START
VALUE(M)*	:	START
VALUE(D)	:	START
AVAIL_VALUE	:	START,HOLD
DESCRIPTION	:	Specifies the properties of JOB CLASS.

SUBJECT	:	tjes
SECTION	:	JOBCLASS
KEY	:	B
TYPE	:	2
DEFAULT_VALUE	:	START
VALUE(M)*	:	START
VALUE(D)	:	START
AVAIL_VALUE	:	START,HOLD
DESCRIPTION	:	Specifies the properties of JOB CLASS.
=====		

6.5. データセット・コマンド

本節では、データセットを表示、作成、削除、コピーするためのコマンドについて説明します。

参考

USERCATALOGについては、tjesmgrの「[SETCATALOG\(SETCAT\)](#)コマンド」と、『OpenFrame データセットガイド』の「第4章 統合カタログ」を参照してください。

6.5.1. DATASETCOPY(DSCOPY)

from-dataset-nameをto-dataset-nameにコピーし、from-dataset-nameを検索するためのカタログとボリューム情報、to-dataset-nameのボリュームと登録されるカタログをオプションとして指定できます。

- 構文

```
DATASETCOPY(DSCOPY) <from-dataset-name> [CAT=<from-catalog-name>]
[VOL=<from-volume-serial>]
<to-dataset-name> [TOV=<to-volume-serial>]
[TOC=<to-catalog-name>] [FORCE]
```

以下は、入力項目に関する説明です。

項目	説明
<from-dataset-name>	ソース・データセット名を指定します。
[CAT=<from-catalog-name>]	ユーザー・カタログを指定します。未指定の場合は、エイリアス、マスター・カタログ、ボリュームの順にデータセットを検索します。
[VOL=<from-volume-serial>]	ソース・データセットのボリューム情報を指定します。未指定の場合は、デフォルト・ボリュームを検索します。
<to-dataset-name>	ターゲット・データセット名を指定します。<to-dataset-name>とTOV=<to-volume-serial>のいずれかを指定する必要があります。
TOV=<to-volume-serial>	コピーしたデータセットが位置するボリューム情報を指定します。未指定の場合は、デフォルト・ボリュームにデータセットをコピーします。
TOC=<to-catalog-name>	コピーしたデータセットを登録するカタログを指定します。
[FORCE]	PDSメンバーをコピーする際に、コピーしたデータセットがすでに存在する場合、強制的に上書きします。未指定の場合は、上書きせずにスキップします。このオプションはソース・データセットがPDSメンバーである場合にのみ適用されます。

参考

デフォルト・カタログについての詳細は、[「6.5.18. SETCATALOG\(SETCAT\)」](#) コマンドを参照してください。

6.5.2. DATASETCREATE(DSCREATE)

指定された非VSAMデータセットを作成します。

● 構文

```
DATASETCREATE(DSCREATE)
```

● 入力パラメータ

以下は、入力パラメータに関する説明です。

パラメータ	説明
NAME	作成する非VSAMデータセット名を指定します。(必須パラメータ)
MEMBER	作成するPDSメンバー名を指定します。
VOLUME	非VSAMデータセットが格納されるボリュームを指定します。デフォルト値は、UNITまたはSMSルールに従います。

パラメータ	説明
	未指定の場合は、デフォルトとして指定されたボリュームが使用されます。
UNIT	データセットのUNITを指定します。デフォルト値は、VOLUMEに従います。 指定されたUNITとVOLUMEが一致しない場合は、エラーが発生します。
DSORG	データセットのDSORGを指定します。
RECFM	データセットのレコード・フォーマットを指定します。
KEYLEN	データセットのキー長を指定します。ISAMデータセットの場合に指定できます。
LRECL	データセットの最大レコード長を指定します。
KEYPOS	データセットのキー位置を指定します。ISAMデータセットの場合に指定できます。
BLKSIZE	データセットのブロック・サイズを指定します。(デフォルト値：4096)
TRK/CYL/blklgth	トラック、シリンダー、またはブロックの長さを指定します。 TRK、CYL、またはブロックの長さに該当する自然数を指定できます。内部的に、TRKは48KB、CYLは768KBで処理されます。
PRIMARY	データセットを割り当てる際、最初の割り当てサイズを指定します。(単位：KB) 以下は、デフォルト値の割り当てルールです。 – PRIMARYとTRK/CYL/blklgthのいずれも設定していない場合：該当するデータセットのデバイスのSpaceLimit値の設定に従います。 – PRIMARYのみ設定し、TRK/CYL/blklgthは設定していない場合：TRK/CYL/blklgthをデフォルト値の1024バイトとみなして、PRIMARYの値に乗算します。 – PRIMARYは設定せず、TRK/CYL/blklgthのみ設定した場合：PRIMARYをデフォルト値の4096バイトとみなして、TRK/CYL/blklgthの値に乗算します。
SECONDARY	データセットの次の割り当てサイズを指定します。(単位：KB)
EXPIRATION	データセットの有効期限を指定します。(デフォルト値:99991231)
CATALOG	ユーザー・カタログを指定します。

パラメータ	説明
	ユーザー・カタログを指定していない場合は、マスター・カタログまたはエイリアスに従います。作成関連のコマンドは、tjesmgrのSETCATALOGコマンドのオプションに従いません。
NOCATALOG	データセットを作成したが、カタログ化していない場合に指定します。
DATA CLASS	SMSのデータ・クラスを指定します。
MGMT CLASS	SMSのマネジメント・クラスを指定します。
STOR CLASS	SMSのストレージ・クラスを指定します。

参考

VSAMデータセットの作成については、『OpenFrame ユーティリティリファレンスガイド』を参照してください。

6.5.3. DATASETDELETE(DSDELETE)

指定された非VSAMデータセットを削除します。

- 構文

```
DATASETDELETE(DSDELETE) [FORCE] <dataset-name> [CAT=<catalog-name>]
[VOL=<volume-serial>]
```

以下は、入力項目に関する説明です。

項目	説明
[FORCE]	削除する非VSAMデータセットのファイルが実際に存在していない場合にもエラーを発生させず、カタログにある情報のみ削除します。
<dataset-name>	削除されるデータセット名を指定します。
[CAT=<catalog-name>]	カタログに登録されている場合、そのカタログ名を指定します。
[VOL=<volume-serial>]	カタログに登録されていないデータセットを削除する場合は、ボリュームを指定します。

6.5.4. DATASETEXPORT(DSEXPORT)

指定された非VSAMデータセットを外部にエクスポートします。

- 構文

```
DATASETEXPORT(DSEXPORT) <from-dataset-name> <to-filepath>  
[CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<from-dataset-name>	外部にエクスポートする非VSAMデータセット名を指定します。
<to-filepath>	非VSAMデータセットをエクスポートする絶対ファイル・パス(UNIX)を指定します。
[CAT=<catalog-name>]	エクスポートする非VSAMデータセットがデフォルト・カタログに登録されていない場合に指定します。

6.5.5. DATASETIMPORT(DSIMPORT)

外部にエクスポートしたファイルを読み込んで、非VSAMデータセットとして復元します。

- 構文

```
DATASETIMPORT(DSIMPORT) <from-filepath>
```

以下は、入力項目に関する説明です。

項目	説明
<from-filepath>	非VSAMデータセットが格納されている絶対ファイル・パス(UNIX)を指定します。

- 入力パラメータ

以下は、入力パラメータに関する説明です。入力パラメータを使用して外部にエクスポートするときのデータセット名と形式を変更することができます。すでに登録されている場合は、既存のデータセットに上書きします。

パラメータ	説明
NAME	変更する非VSAMデータセット名を指定します。
VOLUME	非VSAMデータセットが格納されるボリュームの位置を指定します。
RECFM	データセットのレコード・フォーマットを指定します。
LRECL	データセットの最大レコード長を指定します。(デフォルト値: 80)
BLKSIZE	データセットのブロック・サイズを指定します。(デフォルト値: 4096)
EXPIRATION	データセットの有効期限を指定します。(デフォルト値: 99991231)
CATALOG	ユーザー・カタログを指定します。

パラメータ	説明
	ユーザー・カタログを指定していない場合は、マスター・カタログまたはエイリアスに従います。作成関連のコマンドは、tjesmgrの SETCATALOG コマンドのオプションに従いません。

6.5.6. DATASETLIST(DSLIST)

VTOCに登録されているデータセット・リストを表示します。VTOCに登録されているデータセットはすべて非VSAMデータセットです。

- 構文

```
DATASETLIST(DSLIST) [<dataset-prefix>] [VOL=<volume-serial>]
```

以下は、入力項目に関する説明です。

項目	説明
[<dataset-prefix>]	指定した接頭辞で始まるすべてのデータセットが表示されます。
[VOL=<volume-serial>]	データセットを検索するボリュームを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
VTOC name	検索したデータセットです。
VOLUME	データセットが格納されているボリュームです。
DSORG	データセットの構造です。
RECFM	データセットのレコード・フォーマットです。
LRECL	データセットのレコード長です。

6.5.7. DATASETMOVE(DSMOVE)

データセット名またはボリュームを変更するか、データセット名とボリュームの両方を変更します。ソース・データセットをターゲット・データセット、または入力されたボリュームに移動します。

- 構文

```
DATASETMOVE(DSMOVE) <from-dataset-name> [CAT=<catalog-name>]
                        [VOL=<from-volume-serial>]
                        {<to-dataset-name> | TOV=<to-volume-serial>}
```

以下は、入力項目に関する説明です。

項目	説明
<from-dataset-name>	ソース・データセット名を指定します。
[CAT=<catalog-name>]	ユーザー・カタログを指定します。未指定の場合は、エイリアス、マスター・カタログ、ボリ्यूムの順にデータセットを検索します。
[VOL=<from-volume-serial>]	ソース・データセットのボリ्यूム情報を指定します。未指定の場合は、デフォルト・ボリ्यूムを検索します。
<to-dataset-name>	ターゲット・データセット名を指定します。<to-dataset-name>とTOV=<to-volume-serial>のいずれかを指定する必要があります。
TOV=<to-volume-serial>	ソース・データセットを移動させるボリ्यूム情報を指定します。<to-dataset-name>とTOV=<to-volume-serial>のいずれかを指定する必要があります。

6.5.8. DATASETRECATALOG(DSRECAT)

指定された非VSAMデータセットを再カタログします。ファイルのみ存在しており、カタログに登録されていない非VSAMデータセットをカタログに登録するか、すでにカタログに登録されている非VSAMデータセットのカタログ内容を変更する場合に使用します。

- 構文

```
ATASETRECATALOG(DSRECAT)
```

- 入力パラメータ

以下は、入力パラメータに関する説明です。

パラメータ	説明
NAME	再カタログする非VSAMデータセット名を指定します。(必須パラメータ)
CATALOG(S)	再カタログするデータセットが登録されているカタログを指定します。(デフォルト値: マスター・カタログ)
VOLUME	再カタログする非VSAMデータセットが格納されているボリ्यूムの位置を指定します。
DSORG	データセットのDSORGを指定します。
RECFM	データセットのレコード・フォーマットを指定します。
KEYLEN	データセットのキー長を指定します。ISAMデータセットの場合にのみ指定できます。

パラメータ	説明
LRECL	データセットの最大レコード長を指定します。
KEYPOS	データセットのキー位置を指定します。ISAMデータセットの場合にのみ指定できます。
BLKSIZE	データセットのブロック・サイズを指定します。(デフォルト値: 4096)
EXPIRATION	データセットの有効期限を指定します。(デフォルト値: 99991231)
CATALOG	ユーザー・カタログを指定します。 ユーザー・カタログを指定していない場合は、マスター・カタログまたはエイリアに従います。

6.5.9. DATASETUNCATALOG(DSUNCAT)

指定されたデータセットをカタログから削除します。

- 構文

```
DATASETUNCATALOG(DSUNCAT) <dataset-name> [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<dataset-name>	カタログから削除するデータセット名を指定します。
[CAT=<catalog-name>]	削除するデータセットが登録されているカタログを指定します。

6.5.10. DATATOUCH(DSTOUCH)

指定された非VSAMデータセットの最終変更日を変更します。

- 構文

```
DATATOUCH(DSTOUCH) <dataset-name> [VOL=<volume-serial>]  
[CAT=<catalog-name>] [DATE=<yyyymmdd>]
```

以下は、入力項目に関する説明です。

項目	説明
<dataset-name>	変更日を変更する非VSAMデータセットを指定します。
VOL=<volume-serial>	非VSAMデータセットが格納されているボリュームを指定します。
[CAT=<catalog-name>]	ユーザー・カタログを指定します。

項目	説明
[DATE=<yyyymmdd>]	変更日を指定します。

6.5.11. GDGCREATE

指定された条件に合わせてGDS(世代別データセット)を作成します。

- 構文

```
GDGCREATE
```

- 入力パラメータ

以下は、入力パラメータに関する説明です。

パラメータ	説明
NAME	作成するGDG名を指定します。(必須パラメータ)
LIMIT	GDSのリミットを指定します。(デフォルト値: 255)
EXPIRATION	作成するGDGの有効期限を指定します。
CATALOG	GDGが登録されるユーザー・カタログを指定します。 ユーザー・カタログを指定していない場合は、マスター・カタログまたはエイリアに従います。 作成関連のコマンドは、tjesmgrの SETCATALOG コマンドのオプションに従いません。
EMPTY	GDSの作成時にGDGのLIMIT値を超える場合に、GDGエントリからGDSアソシエーション情報を削除する範囲を指定します。 – 0: NOEMPTY。GDGエントリから一番古いGDSアソシエーション情報のみを削除します。(デフォルト値) – 1: EMPTY。GDGエントリから最新のGDSを除く他のGDSアソシエーション情報を削除します。
SCRATCH	EMPTY/NOEMPTYオプションの処理によってカタログのGDGエントリからアソシエーション情報が削除されたGDSに対して行う処理を指定します。 – 0: NOSCRATCH: GDSを一般の非VSAMデータセットとして扱い、再びカタログに登録します。(デフォルト値) – 1: SCRATCH: GDSを物理的に削除します。

6.5.12. GDGDELETE

指定されたGDGを削除します。GDGメンバーが存在する場合はエラーが発生するため、GDGメンバーを事前に削除する必要があります。

- 構文

```
GDGDELETE <dataset-name> [CAT=<catalog-name>] [FORCE]
```

以下は、入力項目に関する説明です。

項目	説明
<dataset-name>	削除するGDGを指定します。
[CAT=<catalog-name>]	GDGが登録されているカタログを指定します。
[FORCE]	関連付けられたGDSが存在する場合、一緒に削除します。

6.5.13. PSCATALOG(PSCAT)

指定されたカタログに登録されているデータセット・リストを表示します。

- 構文

```
PSCATALOG(PSCAT) <entry-prefix> [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<entry-prefix>	指定された接頭辞で始まるすべてのデータセットを表示します。
[CAT=<catalog-name>]	データセットを検索するカタログを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
CATALOG NAME	検索したデータセットです。
VOLUME	データセットが格納されているボリュームです。
ENTTYPE	データセットのタイプです。 データセットのタイプには、非VSAM、CLUSTER、AIX、PATH、GDG、UCAT、ALIASなどがあります。
VSAM	VSAMデータセットの種類です。

情報	説明
	VSAMデータセットの場合にのみ、KSDS、RRDSなどのデータセットを表示します。
NVSM	非VSAMデータセットの種類です。 非VSAMデータセットの場合にのみ、PDS、SDS、GDSなどのデータセットを表示します。

6.5.14. PSDATASET(PSDS)

指定されたデータセットの詳細情報を表示します。

- 構文

```
PSDATASET(PSDS) <dataset-name> [CAT=<catalog-name>]
[MEM=<member-prefix>] [VOL=<volume-serial>]
```

以下は、入力項目に関する説明です。

項目	説明
<dataset-name>	詳細を表示するデータセット名を指定します。
[CAT=<catalog-name>]	データセットを検索するカタログを指定します。
[MEM=<member-prefix>]	MEMを指定すると、データセットのメンバーを検索して表示します。
[VOL=<volume-serial>]	カタログ化されていないデータセットの詳細を確認する場合に使用します。指定されたボリュームからデータセットを検索して表示します。

- 表示情報

以下の情報が表示されます。括弧内の情報は、VSAMデータセットの場合です。

情報	説明
Data Set Name	データセット名です。
Data Set Type	データセットのタイプです。
Catalog Name	データセットが登録されているカタログ名です。
Management Class	データセットの作成時に参照したマネジメント・クラスです。
Creation Date	データセットの作成日です。
Data Set Owner	データセット・ファイルの所有者です。
Expiration Date	データセットの有効期限です。

情報	説明
Storage Class	データセットの作成時に参照したストレージ・クラスです。
Volume Serial	データセットが格納されているボリュームです。
Device Type	データセットのデバイス・タイプです。 テープ・データセットである場合は、「TAPE Device」と表示されます。
Data Class	データセットの作成時に参照したデータ・クラスです。
Organization	データセットのDSORGです。
Record Format (AVGLRECL)	データセットのRECFMです。(表示されるデータセットの平均長)
KEYLEN	データセットのキー長です。
Record Length (MAXLRECL)	データセットの長さです。(表示されるデータセットの最大長)
KEYPOS	データセットのキー位置です。
BLKSIZE (CISIZE)	データセットのブロック・サイズです。(表示されるデータセットのCISIZE)
(AIXRKP)	データセットのAIXのKEYPOSです。
Primary Space	データセットの最初の割り当てサイズです。
Number of Extents	データセットの割り当て時の拡張回数です。
Secondary Space	データセットの次の割り当てサイズです。
Data Set Size	データセットの実際のサイズです。 テープ・データセットは圧縮されているが、圧縮されていないデータ・サイズです。
Last Access Date	データセットが最後に検索または変更された日付です。
Last Access Time	データセットが最後に検索または変更された時刻です。
Associations	表示されるデータセットと関連のあるデータセットを検索します。 主にVSAMデータセットを検索するときに使用されます。

以下は、データセット・メンバーが存在する場合の表示情報です。

情報	説明
Name	メンバー名です。
Owner	メンバー・ファイルの所有者です。
Size	メンバー・ファイルのサイズです。
Last Access Date	メンバーを最後に検索または変更した日付と時刻です。

6.5.15. PSEENTRY(PSENT)

指定されたエントリの詳細情報を表示します。PSDATASETコマンドと違って、GDG、PATH、ALIASなど、カタログに登録されているエントリ情報まで表示します。

- 構文

```
PSEENTRY(PSENT) <entry-name> [CAT=<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
<entry-name>	検索するエントリ名を指定します。
[CAT=<catalog-name>]	エントリを検索するカタログを指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
Entry Name	エントリ名です。
Entry Type	エントリのタイプです。
Catalog Name	エントリが登録されているカタログ名です。
Creation Date	エントリの作成日です。
Entry Owner	エントリの所有者です。
Expiration Date	エントリの有効期限です。
GDG Limit	エントリがGDGの場合、作成可能なGDG数です。
Associations	表示されるエントリと関連のあるエントリです。

6.5.16. PSLOCK(PSL)

指定されたデータセットのロック情報を表示します。

- 構文

```
PSLOCK(PSL) [<job-ID> | DSN=<dataset-prefix> [CAT=<catalog-name>]]
```

以下は、入力項目に関する説明です。

項目	説明
<job-ID>	指定されたジョブIDでロックされているデータセット・リストを表示します。

項目	説明
DSN=<dataset-prefix>	データセットの名前またはデータセット名の接頭辞を指定します。
[CAT=<catalog-name>]	検索するデータセットがカタログされている場合、そのカタログ名を指定します。

- 表示情報

以下の情報が表示されます。

項目	説明
オプション(1、2カラム)	– 1 : ロックを要求している状態は「R」が表示されます。 – 2 : ロックを設定または要求したプロセスが実行中の場合はアスタリスク(*)が表示されます。プロセスのノードが異なる場合は表示されません。
PID	ロックを設定または要求したプロセスIDです。
JOBID	ロックを設定または要求したジョブIDです。
NODEID	ロックを設定または要求したノードです。
LOCKTYPE	要求したロックのタイプです。 – Exclusive : データセットを変更するために使用します。他のジョブとロックを共有できません。 – Shared : データセットを検索するために使用します。他のジョブとロックを共有できます。
CATALOG_DSN	要求したロック名です。 USERCATALOG_NAME:DATASET_NAMEの形式でロック名を指定します。

6.5.17. PSVOLUME(PSV)

ボリュームの情報を表示します。

- 構文

```
PSVOLUME (PSV)
```

- 表示情報

以下の情報が表示されます。

情報	説明
Volume	ボリューム・シリアル名です。
Total(MB)	ボリューム全体のサイズです。
Usage(%)	現在のボリュームの使用率です。
Free(MB)	使用可能なボリューム・サイズです。
Tablespace	ボリューム・シリアルに該当するデータベース上の表領域の名前です。
Volume Path	ボリュームのUNIXファイル・パスです。

6.5.18. SETCATALOG(SETCAT)

指定されたカタログをデフォルト・カタログにして、データセット関連コマンドで使します。

• 構文

```
SETCATALOG(SETCAT) [<catalog-name>]
```

以下は、入力項目に関する説明です。

項目	説明
[<catalog-name>]	指定されたカタログをデフォルト・カタログにして、データセット関連コマンドで使します。カタログを指定していない場合は、デフォルト・カタログを空の文字列に変更します。 以下は、カタログの優先順位です。 1. CATパラメータで指定されたユーザー・カタログ 2. SETCATで指定されたユーザー・カタログ 3. データセットのエイリアスによるユーザー・カタログ 4. マスター・カタログ

• 例

以下のように指定してPSCATまたはPSDSを実行すると、CATパラメータが指定されていないため、TAPECATをデフォルト・カタログにして情報を表示します。

```
$ SETCAT TAPECAT
```

6.6. スプール・バックアップ・コマンド

本節では、スプール・バックアップ・コマンドの構文について説明します。

6.6.1. SPOOLBACKUP(SPBA)

指定されたスプールをバックアップします。バックアップされたスプールは、OpenFrame環境設定のtjesサブジェクト、SPOOLセクションのSPOOL_BACKUP_DIRキーのVALUE項目に設定されたディレクトリに圧縮ファイルで格納されます。バックアップは、obmjspbkサーバーによって実行され、バックアップが実行中の場合はSPBAコマンドが実行されません。

スプールをバックアップできるジョブの状態は、DONE、ERROR、STOP、FLUSHであり、それ以外のジョブはスキップされ、次のジョブのバックアップを実行します。

- 構文

```
SPOOLBACKUP(SPBA) { DT=<yyyymmddhhmiss> | <start-job-ID>,<end-job-ID> }
```

以下は、入力項目に関する説明です。

項目	説明
DT=<yyyymmddhhmiss>	ジョブをサブミットした時刻を指定します。指定した時間より前にサブミットされたジョブのスプールをバックアップします。
<start-job-ID>,<end-job-ID>	バックアップするジョブIDの範囲を指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
Spool backup command submitted.	バックアップ・コマンドが正常に実行されたことを示します。
Spool backup server is busy!	現在バックアップが実行されているため、バックアップ・コマンドを実行できないことを示します。

関連環境設定

バックアップされたスプールを格納するディレクトリです。

```
$ ofconfig list -n NODE1 -s tjes -sec SPOOL -k SPOOL_BACKUP_DIR -l
```

```
=====
SUBJECT      : tjes
SECTION      : SPOOL
KEY          : SPOOL_BACKUP_DIR
TYPE         : 2
DEFAULT_VALUE : $(OPENFRAME_HOME)/spbackup
VALUE(M)*    : $(OPENFRAME_HOME)/spbackup
VALUE(D)     : $(OPENFRAME_HOME)/spbackup
```



```

AVAIL_VALUE      :
DESCRIPTION      : Directory where spool archive.
=====

```

バックアップされたスプールを必要に応じて復元するディレクトリです。

```

$ ofconfig list -n NODE1 -s tjes -sec SP00L -k SP00L_UNPACK_DIR -l
=====

SUBJECT          : tjes
SECTION          : SP00L
KEY              : SP00L_UNPACK_DIR
TYPE             : 2
DEFAULT_VALUE    : $(OPENFRAME_HOME)/spunpack
VALUE(M)*        : $(OPENFRAME_HOME)/spunpack
VALUE(D)         : $(OPENFRAME_HOME)/spunpack
AVAIL_VALUE      :
DESCRIPTION      : Directory where archived spool is extracted temporarily.
=====

```

6.6.2. SPOOLBACKUPLIST(SPBL)

指定された年、月にバックアップされたスプールの日付を表示します。

- 構文

```
SPOOLBACKUPLIST(SPBL) DATE={ <yyyymm>|<yyyy> }
```

以下は、入力項目に関する説明です。

項目	説明
DATE={ <yyyymm> <yyyy> }	検索する年、月を指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
BACKUP_DATE	スプールがバックアップされた日付です。

6.6.3. SPOOLBACKUPSTATUS(SPBASTAT)

バックアップ操作の進捗状況と完了した操作の処理結果を表示します。

- 構文

SPOOLBACKUPSTATUS(SPBASTAT)

- 表示情報

現在実行中かまたは最後に実行したバックアップの状態と情報を表示します。

```
server state:<state>, attempt:<count>, done:<count>, success rate:<percent>
```

情報	説明
server state:<state>	現在のサーバー状態です。 – WORKING : バックアップが実行中の場合 – IDLE : WORKING以外の状態
attempt:<count>	スプール・バックアップが試行された回数です。
done:<count>	スプール・バックアップが成功された回数です。
success rate:<percent>	スプール・バックアップの成功率(パーセント)です。

6.6.4. SPOOLBACKUPSTOP(SPBASTOP)

現在実行中のバックアップを終了します。バックアップが実行中の場合にのみ有効です。

- 構文

```
SPOOLBACKUPSTOP (SPBASTOP)
```

6.6.5. SPOOLCLEAR(SPCL)

指定された一時ディレクトリに復元されたスプールを削除します。つまり、SPOOLRESTOREによって復元されたスプールを削除します。

- 構文

```
SPOOLCLEAR (SPCL)    I=<index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSで出力されたスプール・インデックス番号を指定します。

6.6.6. SPOOLPODD(SPPODD)

復元されたジョブのスプールを事前に設定したビューアーで開きます。

- 構文

```
SPOOLPODD(SPPODD)  I=<index> SPDI=<dd-index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSで出力されたスプール・インデックス番号を指定します。
SPDI=<dd-index>	確認するDD索引を指定します。

参考

SPOOLPODDコマンドはPODDコマンドと同様な機能です。詳細については、[「6.3.2. PODD\(Print Output DD Information\)」](#) コマンドを参照してください。

6.6.7. SPOOLPOSPPOOL(SPPOSPPOOL)

復元されたジョブのスプール情報を表示します。SPOOLPOSPPOOLコマンドの実行結果を確認するには、SPOOLPSとSPOOLRESTOREコマンドが先に実行される必要があります。

- 構文

```
SPOOLPOSPPOOL(SPP0SP00L)  I=<index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSで出力されたスプール・インデックス番号を指定します。

- 表示情報

POSPPOOLと同じ情報が表示されます。

6.6.8. SPOOLPS(SPPS)

指定された日付にバックアップされたスプール・リストを表示します。

- 構文

```
SPOOLPS(SPPS) { DATE=<yyyymmdd> [N=<job-name-prefix>] | N=<job-name-prefix> }
```

以下は、入力項目に関する説明です。

項目	説明
DATE=<yyyymmdd>	バックアップされた日付を指定します。

項目	説明
N=<job-name-prefix>	バックアップされたジョブから検索するジョブ名の接頭辞を指定します。

- 表示情報

以下の情報が表示されます。

情報	説明
NO	スプール・リスト番号です。 スプール・リスト番号は、 SPOOLRESTORE 、 SPOOLPSJOB 、 SPOOLPSIO などで使用されます。
JOBNAME	ジョブ名です。
JOBID	ジョブIDです。
submit_TIME	ジョブがサブミットされた日付です。
BACKUP_DATE	ジョブがバックアップされた日付です。

6.6.9. SPOOLPSJOB(SPPSJ)

復元されたジョブの詳細情報を表示します。SPOOLPSJOBの実行結果を確認するには、SPOOLPSとSPOOLRESTOREコマンドが先に実行される必要があります。

- 構文

```
SPOOLPSJOB(SPPSJ)    I=<index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSJOBで出力されるスプール・インデックス番号を指定します。

- 表示情報

PSJOBの表示情報と同じです。ただし、出力に関する情報は表示されません。

6.6.10. SPOOLPSIO(SPPSIO)

復元されたジョブの入出力情報を表示します。SPOOLPSIOコマンドの実行結果を確認するには、SPOOLPSとSPOOLRESTOREコマンドが先に実行される必要があります。

- 構文

```
SPOOLPSIO(SPPSIO)  I=<index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSで出力されたスプール・インデックス番号を指定します。

- 表示情報

各ステップごとにデータセットの読み書き回数を表示します。

6.6.11. SPOOLRESTORE(SPRE)

バックアップされたスプールの内容を確認するために一時ディレクトリに復元します。

SPOOLRESTOREの実行結果を確認するには、SPOOLPSコマンドが先に実行される必要があります。

- 構文

```
SPOOLRESTORE(SPRE)  I=<index>
```

以下は、入力項目に関する説明です。

項目	説明
I=<index>	SPOOLPSで出力されたスプール・インデックス番号を指定します。

関連環境設定

スプールがバックアップされるディレクトリを指定します。

```
$ ofconfig list -n NODE1 -s tjes -sec SPOOL -k SPOOL_BACKUP_DIR -l
```

```
=====
```

SUBJECT	: tjes
SECTION	: SPOOL
KEY	: SPOOL_BACKUP_DIR
TYPE	: 2
DEFAULT_VALUE	: \$(OPENFRAME_HOME)/spbackup
VALUE(M)*	: \$(OPENFRAME_HOME)/spbackup
VALUE(D)	: \$(OPENFRAME_HOME)/spbackup
AVAIL_VALUE	:
DESCRIPTION	: Directory where spool archive.

```
=====
```

スプールの内容を確認するために一時的に復元するディレクトリを指定します。

```
$ ofconfig list -n NODE1 -s tjes -sec SPOOL -k SPOOL_UNPACK_DIR -l
=====
SUBJECT      : tjes
SECTION      : SPOOL
KEY          : SPOOL_UNPACK_DIR
TYPE         : 2
DEFAULT_VALUE : $(OPENFRAME_HOME)/spunpack
VALUE(M)*    : $(OPENFRAME_HOME)/spunpack
VALUE(D)     : $(OPENFRAME_HOME)/spunpack
AVAIL_VALUE  :
DESCRIPTION  : Directory where archived spool is extracted temporarily.
=====
```

6.7. その他のコマンド

本章では、その他のコマンドの使用方法について説明します。

6.7.1. CLS

CLSコマンドを実行すると、tjesmgr画面が削除されます。

- 構文

```
CLS
```

6.7.2. CURRENTUSER

現在ログインしているユーザーIDを表示します。

- 構文

```
CURRENTUSER (CU)
```

6.7.3. EDIT

入力したfilepathを指定したエディターで開きます。

- 構文

```
EDIT <filepath>
```

以下は、入力項目に関する説明です。

項目	説明
<filepath>	絶対パスを指定します。

関連環境設定

ファイルを開くエディターを指定します。

```
$ ofconfig list -n NODE1 -s tjesmgr -sec DEFAULT_OPTION -k EDITOR -l
=====
SUBJECT      : tjesmgr
SECTION      : DEFAULT_OPTION
KEY          : EDITOR
TYPE         : 2
DEFAULT_VALUE : vi &FILEPATH
VALUE(M)*    : vim -w&ROWCOUNT &FILEPATH
VALUE(D)     : vim -w&ROWCOUNT &FILEPATH
AVAIL_VALUE  :
DESCRIPTION  : Editing program name for tjesmgr command EDIT.
               &ROWCOUNT will be replaced window size.
               &FILEPATH will be replaced editing filepath.
               -w&ROWCOUNT option is valid only for `old vi' not `vim'.
=====
```

6.7.4. ERROR

入力したerror-numberに該当するエラー・メッセージを表示します。

- 構文

```
ERROR <error-number>
```

6.7.5. HELP

指定されたコマンドの使用方法を表示します。未指定の場合は、すべてのコマンドの使用方法を表示します。

- 構文

```
HELP    [<tjesmgr_command>]
```

6.7.6. LICENSE

指定されたライセンス・ファイルの情報を表示します。

- 構文

```
LICENSE { <product> | <abs-filepath> | <filename> }
```

以下は、入力項目に関する説明です。

項目	説明
<product>	TJES、TACF、HIDB、OSC、OSIのいずれかの製品を指定します。
<abs-filepath>	ライセンス・ファイルの絶対パスを指定します。
<filename>	ライセンス・ファイル名のみ指定した場合は、OpenFrame環境設定に登録されているライセンス・ディレクトリから指定したライセンス・ファイルを検索します。

関連環境設定

ライセンス・ディレクトリです。

<ofsys.conf>

```
$ ofconfig list -n NODE1 -s ofsys -sec DIRECTORY -k LICENSE_DIR -l
```

```
=====
SUBJECT       : ofsys
SECTION       : DIRECTORY
KEY           : LICENSE_DIR
TYPE          : 2
DEFAULT_VALUE : ${OPENFRAME_HOME}/license
VALUE(M)*     : ${OPENFRAME_HOME}/license
VALUE(D)      : ${OPENFRAME_HOME}/license
AVAIL_VALUE   :
DESCRIPTION   :
=====
```

6.7.7. NOTIFY(N)

現在ログインしているユーザーIDにジョブの実行情報を通知します。JOB文やJES JCL文(MSP)/JES2 JCL文(MVS)のNOTIFYパラメータに特定のユーザーIDを指定してジョブを実行すると、ジョブの終了後そのユーザーIDにメッセージを残します。このユーザーIDでtjesmgrに接続したユーザーは、tjesmgrの実行画面で各コマンドの実行後にメッセージを確認したり、NOTIFYコマンドを使用してメッセージを残したすべてのジョブの実行情報（JOBNAME、JOBID、JOBの終了状態）を確認することができます。

- 構文

```
NOTIFY(N)
```

- 表示情報

以下の情報が表示されます。

情報	説明
DATE	メッセージを残した日付です。
TIME	メッセージを残した時間です。
MESSAGE	ジョブの実行情報です。

6.7.8. PSPRINT(PSP)

OpenFrame環境設定に登録されているプリンター情報を表示します。

- 構文

```
PSPRINT(PSP)
```

- 表示情報

以下の情報が表示されます。

情報	説明
PRINTER_NAME	プリンター名です。
CLASS	出力クラスです。
PRINTER_COMMAND	プリンターを実行するコマンドです。
DRIVER_PATH	プリンター・ライブラリのパスです。

関連環境設定

プリンターを設定します。

```
$ ofconfig list -n NODE1 -s print -sec PRINTER1 -l
=====
SUBJECT      : print
SECTION     : PRINTER1
KEY         : CLASS
TYPE        : 2
DEFAULT_VALUE : AB
VALUE(M)*   : AB
VALUE(D)    : AB
AVAIL_VALUE :
DESCRIPTION : Output class used for this printer.
-----
SUBJECT      : print
SECTION     : PRINTER1
KEY         : COMMAND
TYPE        : 2
```

DEFAULT_VALUE	:	cat
VALUE(M)*	:	cat
VALUE(D)	:	cat
AVAIL_VALUE	:	
DESCRIPTION	:	Printer's command.

SUBJECT	:	print
SECTION	:	PRINTER1
KEY	:	DRIVER_PATH
TYPE	:	2
DEFAULT_VALUE	:	\${OPENFRAME_HOME}/lib/libdfltprt.so
VALUE(M)*	:	\${OPENFRAME_HOME}/lib/liblrsprt.so
VALUE(D)	:	\${OPENFRAME_HOME}/lib/liblrsprt.so
AVAIL_VALUE	:	
DESCRIPTION	:	Dynamic loading library path for printer.
=====		

6.7.9. SWITCHUSER

ユーザー入力画面が表示され、入力されたユーザーIDでログインします。

- 構文

```
SWITCHUSER (SU)
```

- 入力パラメータ

以下は、入力パラメータに関する説明です。

パラメータ	説明
ID	ログインするユーザーIDを指定します。
GROUP NAME	ログインするユーザー・グループを指定します。
PASSWORD	ログインするユーザー・パスワードを指定します。

6.7.10. TSOLIST(TSL)

現在TSOに接続しているユーザー・リストを表示します。

- 構文

```
TSOLIST(TSL) [ ID=<userid> | LU=<luname> ]
```

- 表示情報

以下の情報が表示されます。

情報	説明
USER_ID	TSOユーザーのIDです。
LU_NAME	TSOユーザーのLU名です。
JOBID	TSOユーザーの接続ジョブIDです。
LOGIN_TIME	TSOユーザーが接続した時間情報です。

6.7.11. VERSION

TJESのバージョン情報を表示します。

- 構文

```
VERSION
```


第7章 環境設定

本節では、TJESを実行するために必要なTmaxとTJESの環境設定について説明します。

参考

TJES環境設定の詳細については、『OpenFrame 環境設定ガイド』の**tjes**、**tjesmgr**、**print**、**tjclrun**、**rc**サブジェクトを参照してください。

7.1. Tmaxの設定

OpenFrameでTJESを実行するには、Tmaxのインストールおよび環境設定を行う必要があります。OpenFrameをインストールするとき、Tmaxの環境はデフォルト値で登録されます。以下の手順でデフォルト値を変更することができます。

1. Tmaxサーバーを停止します。

```
$ tmdown
```

2. \$TmaxDIR/config/XXX.mファイルを開いて環境を再設定します。

3. 変更した設定を適用します。

```
$ cfl -i $TMAXDIR/config/XXX.m
```

4. Tmaxサーバーを再起動します。

```
$ tmboot
```

参考

上記例のXXX.m環境ファイルの変更方法については、『Tmax インストールガイド』を参照してください。

7.1.1. Tmaxサーバー

TJESを運用するためには、以下のサーバーが起動している必要があります。

- TJESサーバー

サーバー名	役割
obmjschd	ジョブ・スケジューラー

サーバー名	役割
obmjinit	ジョブ・イニシエーター
obmjmsvr	ジョブ・マネージャー
obmjhist	ジョブの履歴
obmjspbk	ジョブ・スプールのバックアップ
ofrpmsvr	プリンター・マネージャー
obmtsmgr	TSOジョブの処理
obmtscmd (オプション)	TSOコマンド処理（このサーバーは、ISPFサービスを使用する環境でのみ提供されます）

- TJES関連サーバー

サーバー名	役割
ofrsasvr	セキュリティ管理
ofrlhsrv	ロック・ハンドラー割り当て
ofruisvr	ユーザー・インターフェース・サーバー
ofrcmsvr	コンソール管理

OpenFrameのインストール時に、TJESサーバーとTJES関連サーバーは自動的に登録されます。

以下は、Tmax環境に登録されているTJESサーバーの例です。

```

obmj schd      SVGNAME = svg_domain, MIN = 1, MAX = 1, SVRTYPE=UCS,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

obmj init      SVGNAME = svg_node1, MIN = 1, MAX = 1, SVRTYPE=UCS,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

obmj msrv      SVGNAME = svg_node1,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

obmj hist      SVGNAME = svg_domain, MIN = 1, MAX = 1,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

obmj spbk      SVGNAME = svg_domain, MIN = 1, MAX = 1,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

ofrp msrv      SVGNAME = svg_domain,
                MIN = 1, MAX = 1, SVRTYPE=UCS,
                CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

```

```
obmtsmgr    SVGNAME = svg_domain,    MIN = 1, MAX = 1, SVRTYPE=UCS,
            CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"

obmtscmd    SVGNAME = svg_node1,
            CLOPT="-o $(SVR)$(DATE).out -e $(SVR)$(DATE).err"
```

上記例の**SVGNAME=svg_domain**は、1つのノードでのみ起動される必要があることを示しています。また、**MIN=1**、**MAX=1**は、1つのサーバーのみ起動される必要があることを示します。

したがって、obmjschdは1つのノードで1つのみ起動されるサーバーであり、obmjinitは各ノードごとに1つずつ起動されるサーバーです。また、obmjmsvrは各ノードごとに複数起動できるサーバーです。使用環境の負荷状況に応じてMIN値を調整してください。未指定の場合は、MIN = 1です。

参考

TJES関連サーバーの設定については、『OpenFrame Baseガイド』を参照してください。

以下は、Tmaxサーバーの起動と正常に起動したかどうかを確認する方法です。

- Tmaxサーバーの起動

```
$ tmboot
```

- Tmaxサーバーの状態を確認

```
$ tmdadmin
$$1 node (tmadm): si
```

以下は、Tmaxサーバーが正常に動作しているかどうかを表示する画面です。statusのRDYはサーバーが起動中であることを、NRDYはサーバーが終了したことを示します。

```
-----
  clh   svrname   (svri)  status    count    qcount    qpcount    emcount
-----
    0   obmjmsvr  ( 11)   RDY        0         0         0         0
    0   obmjschd  ( 12)   RDY        0         0         0         0
    0   obmjinit  ( 13)   RDY        1         0         0         0
    0   obmjhist  ( 14)   RDY        0         0         0         0
    0   obmjspbk  ( 15)   RDY        0         0         0         0
    0   ofrpmsvr  ( 16)   RDY        0         0         0         0
    0   obmtsmgr  ( 17)   RDY        0         0         0         0
    0   obmtscmd  ( 18)   RDY        0         0         0         0
-----
```

- Tmaxサーバーの停止

```
$ tmdown
```

tmadminを使用してTmaxサーバーの状態を確認する際、サーバーの状態がNRDYの場合は、サーバー・ログからその原因を確認することができます。Tmaxによって作成されるサーバー・ログ・ファイルは、\$TmaxDIR/log/ulogディレクトリに格納されています。

参考

Tmaxについての詳細は、『Tmax 管理者ガイド』を参照してください。

7.1.2. マルチノードの設定

TJESをマルチノードで運用するには、以下のような追加設定が必要です。また、同一ドメインに属するすべてのノードのTmax設定ファイルは同じである必要があります。

以下は、NODE1とNODE2でマルチノードのTJESを実装する例です。

1. ノードの追加

次のようにマルチノードとして構成するノードを追加します。

```
*NODE
DEFAULT:
    DOMAINNAME = "domofb"

NODE1
    HOSTNAME = "tmaxof1",
    IP = "192.168.1.1",
    TMXHOME = "$OPENFRAME_HOME/tmax",
    TMXDIR = "$OPENFRAME_HOME/tmax",
    APPDIR = "$OPENFRAME_HOME/server",
    TLOGDIR = "$OPENFRAME_HOME/tmax/log/tlog",
    ULOGDIR = "$OPENFRAME_HOME/tmax/log/ulog",
    SLOGDIR = "$OPENFRAME_HOME/tmax/log/slog",
    CLHOPT = " -o $OPENFRAME_HOME/tmax/log/clh.log -e
/$OPENFRAME_HOME/tmax/log/clh.err",
    TPORTNO = 9400, SHMKEY = 94000, RACPORT = 9450

NODE2
    HOSTNAME = "tmaxof2",
    IP = "192.168.1.2",
    TMXHOME = "$OPENFRAME_HOME/tmax",
    TMXDIR = "$OPENFRAME_HOME/tmax",
    APPDIR = "$OPENFRAME_HOME/server",
    TLOGDIR = "$OPENFRAME_HOME/tmax/log/tlog",
    ULOGDIR = "$OPENFRAME_HOME/tmax/log/ulog",
    SLOGDIR = "$OPENFRAME_HOME/tmax/log/slog",
    CLHOPT = " -o $OPENFRAME_HOME/tmax/log/clh.log -e
/$OPENFRAME_HOME/tmax/log/clh.err",
    TPORTNO = 9400, SHMKEY = 94000, RACPORT = 9450
```


2. [SVRGROUP]セクションの変更

SVRGROUPセクションには、新しいノードにサーバー・グループを追加、同じサーバーを起動するためのCOUSINを設定、フェイルオーバーのためにsvg_domainのBACKUPを設定します。

```
svg_domain
    NODENAME = "NODE1", BACKUP="svg_domain2"
svg_domain2
    NODENAME = "NODE2"
svg_node1
    NODENAME = "NODE1", COUSIN="svg_node2"
svg_node2
    NODENAME = "NODE2"
```

3. [ROUTING]セクションの追加およびルーティングするサービスの指定

マルチノードを構成するために、いくつかのサービスはTmaxのデータ駆動型ルーティング機能を使用します。そのとき、ルーティングに使用されるデータはノード名です。[SERVICE]セクションには、変更が必要なサービスのみ表示します。

```
*ROUTING
rt_nodename FIELD = "FIELD/FB_NODENAME",
    RANGES = "'NODE1':svg_node1,'NODE2':svg_node2,*:svg_node1"

*SERVICE
OBMJMSVRJDETAIL SVRNAME = obmjmsvr, ROUTING = rt_nodename
OBMJINITCTLNODE SVRNAME = obmjinit, ROUTING = rt_nodename
OBMJINITCTLJOB SVRNAME = obmjinit, ROUTING = rt_nodename
OBMJINITCTLINIT SVRNAME = obmjinit, ROUTING = rt_nodename
OBMJINITRUN SVRNAME = obmjinit, ROUTING = rt_nodename
OBMJINITSTATUS SVRNAME = obmjinit, ROUTING = rt_nodename
OBMJINITCHECK SVRNAME = obmjinit, ROUTING = rt_nodename
```


索引

B

BOOTコマンド, 91

C

CANCELコマンド, 77, 82

CANCELログ, 25

CATPROC, 37, 41

CATPROCデータセット, 56

CHANGECLASS(CC)コマンド, 92

CHANGEログ, 22

CLSコマンド, 120

ColdBoot, 7

CONVJCLデータセット, 56

CURRENTUSERコマンド, 120

D

DATASETCOPY(DSCOPY)コマンド, 99

DATASETCREATE(DSCREATE)コマンド, 100

DATASETDELETE(DSDELETE)コマンド, 102

DATASETEXPORT(DSEXPORT)コマンド, 102

DATASETIMPORT(DSIMPORT)コマンド, 103

DATASETLIST(DSLIST)コマンド, 104

DATASETMOVE(DSMOVE)コマンド, 104

DATASETRECATALOG(DSRECAT)コマンド, 105

DATASETUNCATALOG(DSUNCAT)コマンド, 106

DATATOUCH(DSTOUCH)コマンド, 106

DONE, 12

E

EDITコマンド, 120

ERROR, 13

ERRORコマンド, 121

EXEC COND, 34

EXEC PGM STEP, 33, 34

EXEC PROC STEP, 34

EXECUTEログ, 21

F

FINISHログ, 22

FLUSH, 13

G

GDGCREATEコマンド, 107

GDGDELETEコマンド, 108

H

HELPコマンド, 121

HOLD, 12

HOLDコマンド, 77

I

IF-THEN/ELSE/ENDIF, 34

INITIATORコマンド

BOOT, 91

CHANGECLASS(CC), 92

JCLRUNACTIVE(RA), 93

JCLRUNCLASS(RC), 94

JCLRUNINACTIVE(RI), 93

NODECLEAR, 95

NODESTATUS(NODEST), 95

PSCCLASS(PSC), 95

PSJCLRUN (PSR), 95

SHOWCLASS(SC), 98

SHUTDOWN, 98

INPJCL, 41

INPJCLデータセット, 52

INQUEUECLASSコマンド, 77

INQUEUEPRIORITY(IP)コマンド, 77

INSDSET, 41

INSPROC, 37

J

JCLRUNACTIVE(RA)コマンド, 93

JCLRUNCLASS(RC)コマンド, 94

JCLRUNINACTIVE(RI)コマンド, 93

JEMコマンド, 78

JESJCL, 42

JESJCLデータセット, 58

JESMSG, 42
JESMSGデータセット, 57
JESST, 9
JOB CLASS, 17
JOB COND, 34, 36
JOBCAT DD/STEP CAT DD, 40
JOBID, 15
JOBLIB DD/STEPLIB DD, 39
JOBQ, 9, 16

L

LICENSEコマンド, 121

N

NICEコマンド, 78
NICEログ, 24
NODECLEARコマンド, 95
NODESTATUS(NODEST)コマンド, 95
Normalデータセット, 38
NOTIFY(N)コマンド, 122

O

obmjhist, 3, 128
obmjinit, 3, 128
obmjmsvr, 3, 128
obmj schd, 3, 127
obmj spbk, 3, 128
obmtscmd, 3, 128
obmtsmgr, 128
ofrcmsvr, 128
ofrlhsvr, 128
ofrpmsvr, 3, 128
ofrsasvr, 128
ofruisvr, 128
OpenFrame Manager, 5
OpenFrame/Base, 4
OpenFrame/TACF, 4
OUTPUT, 63
OUTPUT CLASS, 65
OUTPUT STATUS, 64
OUTPUTQ, 10
OUTQUEUEPURGEコマンド, 79

OUTQUEUERELEASEコマンド, 79

P

PODD, 19
PODDコマンド, 83
POJOB, 19
POJOBコマンド, 84
POLISTコマンド, 85
POSPPOOL, 19
POSPPOOLコマンド, 85
POコマンド, 83
PSCATALOG(PSCAT)コマンド, 108
PSCCLASS(PSC)コマンド, 95
PSDATASET(PSDS)コマンド, 109
PSENTRY(PSENT)コマンド, 111
PSHISTORY(PSH)コマンド, 88
PSIO, 19
PSIOコマンド, 88
PSJCLRUN (PSR)コマンド, 95
PSJOB, 19
PSJOBコマンド, 89
PSLOCK(PSL)コマンド, 111
PSPRINT(PSP)コマンド, 123
PSSTEPコマンド, 91
PSTARTコマンド, 79
PSVOLUME(PSV)コマンド, 112
PSコマンド, 86

R

REMOVEコマンド, 80
REMOVEログ, 23
RESUMEコマンド, 80
RESUMEログ, 24
RUNNER CLASS, 18
RUNコマンド, 81

S

SCANコマンド, 81
SETCATALOG(SETCAT)コマンド, 113
setuid, 46
setuid root tjclrun, 46
SHOWCLASS(SC)コマンド, 98

SHUTDOWNコマンド, 98
SPOOL, 15, 51
SPOOLBACKUP, 25
SPOOLBACKUP(SPBA)コマンド, 114
SPOOLBACKUPLIST, 25
SPOOLBACKUPLIST(SPBL)コマンド, 115
SPOOLBACKUPSTATUS(SPBASTAT)コマンド, 115
SPOOLBACKUPSTOP(SPBASTOP)コマンド, 116
SPOOLCLEAR, 26
SPOOLCLEAR(SPCL)コマンド, 116
SPOOLPODD, 26
SPOOLPODD(SPPODD)コマンド, 116
SPOOLPOSPPOOL, 26
SPOOLPOSPPOOL(SPPOSPPOOL)コマンド, 117
SPOOLPS, 25
SPOOLPS(SPPS)コマンド, 117
SPOOLPSIO, 26
SPOOLPSIO(SPPSIO)コマンド, 118
SPOOLPSJOB, 26
SPOOLPSJOB(SPPSJ)コマンド, 118
SPOOLRESTORE, 25
SPOOLRESTORE(SPRE)コマンド, 119
START, 12
STARTコマンド, 82
STEPログ, 24
STOP, 13
SUBMITログ, 21
SUSPEND, 12
SUSPENDコマンド, 82
SUSPENDログ, 23
SWITCHUSERコマンド, 124
SYSIN DD, 39
SYSMSG, 42
SYSMSGデータセット, 53
SYSOUT, 43
SYSOUT DD/SYSPRINT DD, 39
SYSOUTデータセット, 38

T

TACFの代理ユーザー, 45
textrun, 5
tjclrun, 4

tjclrunのジョブの実行, 30
tjclrunの呼び出し, 48
tjesmgr, 5, 71
tjesmgrコマンド
 PODD, 19
 POJOB, 19
 POSPOOL, 19
 PSIO, 19
 PSJOB, 19
TJESでのジョブ状態, 12
 DONE, 12
 ERROR, 13
 FLUSH, 13
 HOLD, 12
 START, 12
 STOP, 13
 SUSPEND, 12
 WORKING, 12
TJESのコンポーネント
 obmjhist, 3
 obmjinit, 3
 obmjmsvr, 3
 obmjmschd, 3
 obmjspbk, 3
 obmtscmd, 3
 ofrpmsvr, 3
 OpenFrame Manager, 5
 OpenFrame/Base, 4
 OpenFrame/TACF, 4
 textrun, 5
 tjclrun, 4
 tjesmgr, 5
 TJESシステム・テーブル, 5
 外部スケジューラー, 5
 スプール, 5
 ランナー・スロット, 4
TJESのシャットダウン, 8
TJESのブート, 7
TJESのリソース, 15
TJESシステム・テーブル, 5
 JESST, 9
 JOBQ, 9
 OUTPUTQ, 10

TSOLIST(TSL)コマンド, 124

V

VERSIONコマンド, 125

W

WarmBoot, 7

WORKING, 12

あ

一時データセット, 38

か

外部スケジューラー, 5

外部ライター, 68

さ

削除段階, 19

出力処理, 63

出力クラス, 65

出力クラスの優先順位, 65

出力段階, 18

出力状態の種類, 64

出力の処理方式, 66

出力処理方式の種類, 66

出力の状態, 64

実行段階, 18, 33

その他のコマンド

CLS, 120

CURRENTUSER, 120

EDIT, 120

ERROR, 121

HELP, 121

LICENSE, 121

NOTIFY(N), 122

PSPRINT(PSP), 123

SWITCHUSER, 124

TSOLIST(TSL), 124

VERSION, 125

サブミット段階, 15

ジョブのバックアップ, 25

ジョブの処理手順

削除, 19

サブミット, 15

実行, 18

出力, 18

スケジューリング, 17

ジョブの実行, 33

ジョブの開始と終了, 44

ジョブ・ステップ終了コード, 26

ジョブ・スプール, 41

CATPROC, 41

INPJCL, 41

INSDSET, 41

JESJCL, 42

JESMSG, 42

SYSMSG, 42

SYSOUT, 43

ジョブ・レベルのレポート, 44

ジョブ・ログ, 20

CANCEL, 25

CHANGE, 22

EXECUTE, 21

FINISH, 22

NICE, 24

REMOVE, 23

RESUME, 24

STEP, 24

SUBMIT, 21

SUSPEND, 23

ジョブ出力コマンド

PO, 83

PODD, 83

POJOB, 84

POLIST, 85

POSPOOL, 85

PS, 86

PSHISTORY(PSH), 88

PSIO, 88

PSJOB, 89

PSSTEP, 91

ジョブ制御, 43

ジョブ制御の状態, 44

ジョブ実行コマンド

CANCEL, 77, 82

HOLD, 77
INQUEUECLASS, 77
INQUEUEPRIORITY(IP), 77
JEM, 78
NICE, 78
OUTQUEUEPURGE, 79
OUTQUEUERELEASE, 79
PSTART, 79
REMOVE, 80
RESUME, 80
RUN, 81
SCAN, 81
START, 82
SUSPEND, 82
ジョブ終了コード, 27
 ジョブ・ステップ終了コード, 26
スケジューリング段階, 17
ステップの開始と終了, 44
ステップ・レベルのレポート, 44
スプール, 5
スプールのバックアップ, 62
スプールのバックアップ・コマンド, 25
スプール・データセット, 51
 CATPROC, 56
 CONVJCL, 56
 INPJCL, 52
 JESJCL, 58
 JESMSG, 57
 SYSMSG, 53
スプール・バックアップ・コマンド
 SPOOLBACKUP(SPBA), 114
 SPOOLBACKUPLIST(SPBL), 115
 SPOOLBACKUPSTATUS(SPBASTAT), 115
 SPOOLBACKUPSTOP(SPBASTOP), 116
 SPOOLCLEAR(SPCL), 116
 SPOOLPODD(SPPODD), 116
 SPOOLPOSPPOOL(SPPOSPPOOL), 117
 SPOOLPS(SPPS), 117
 SPOOLPSIO(SPPSIO), 118
 SPOOLPSJOB(SPPSJ), 118
 SPOOLRESTORE(SPRE), 119
スペシャルDD, 39
 JOB CAT DD/STEP CAT DD, 40

JOB LIB DD/STEPLIB DD, 39
SYSIN DD, 39
SYSOUT DD/SYS PRINT DD, 39

た

データセットのロック待機, 44
データセットの割り当て, 45
データセット・コマンド
 DATASETCOPY(DSCOPY), 99
 DATASETCREATE(DSCREATE), 100
 DATASETDELETE(DSDELETE), 102
 DATASETEXPORT(DSEXPORT), 102
 DATASETIMPORT(DSIMPORT), 103
 DATASETLIST(DSLIST), 104
 DATASETMOVE(DSMOVE), 104
 DATASETRECATALOG(DSRECAT), 105
 DATASETUNCATALOG(DSUNCAT), 106
 DATATOUCH(DSTOUCH), 106
 GDGCREATE, 107
 GDGDELETE, 108
 PSCATALOG(PSCAT), 108
 PSDATASET(PSDS), 109
 PSEENTRY(PSENT), 111
 PSLOCK(PSL), 111
 PSVOLUME(PSV), 112
 SETCATALOG(SETCAT), 113
データセット入出力のレポート, 45

な

内部リーダー, 68
入カストリーム・データセット, 38

ら

ランナー・スロット, 4
リターンコードのチェック, 36
ロック段階, 33

